

HANA: HANA stands **for High Performance Analytics Appliance.**

HANA is the “SAP Database”. Before that SAP used Third party Database i.e., Oracle, SQL, SyBase,DB2, etc.

Challenges in Traditional Databases:

- ☐ Design to perform with limited RAM capacity.
- ☐ Data stored in HDDs[Hard Disk Drives].
- ☐ Row based data storage.
- ☐ Complex Data Model[Unnecessary Tables in Old Database].
- ☐ Slow reporting
- ☐ Slower transaction processing
- ☐ High data foot print. [The same data is stored in multiple tables]
- ☐ Reactive Data Model. [Some transactions & reports should be run in the background. Reactive means NOT LIVE]

Question: What is the meaning of S/4?

Advantages of HANA

- ✓ HANA Database will work on **In-Memory processing** or **In-Memory Computing**.
 - In HANA Database, the data will be stored in RAM also. So while retrieving the data. The System will retrieve from RAM itself.
 - The result is the extraction of data is very fast.
 - Based on HDD's capacity, we can purchase RAM capacity same as or equal to HDD's capacity.
 - When we are on the server, then replicable data will again come & seat in RAM.
 - Suppose, when we shutdown the server, then the data which is stored in RAM will go to back HDD(Database).
- ✓ Simplified Data Model(Unnecessary tables have been eliminated).
- ✓ Faster Reporting.
- ✓ Faster Transaction Processing.
- ✓ Less Data Foot Print.
- ✓ Proactive Data Model.
 - HANA supports **OLAP**(Online Analytical Processing) & **OLTP**(Online Transactional Processing)



Advantages of HANA

- ✓ HANA data supports both **ROW based** & **COLUMN based** data storage.
 - While saving the transactions the data will be stored in row based & in the backend system will also store the data in column based tables will help to extract the data very faster.
 - Row based will help to save the transaction data very faster & Column based will help to extract the data very faster.

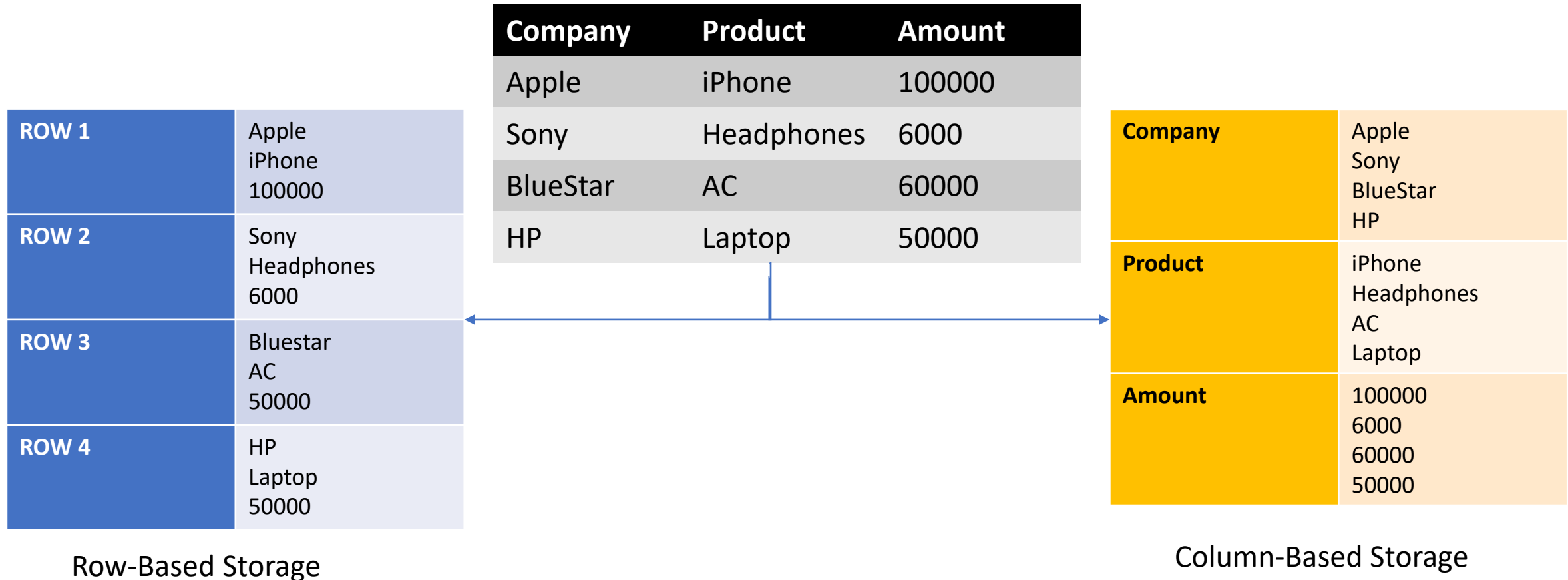
Question:

1. Can you explain the role of in-memory computing technology in S/4HANA and how it differs from traditional database systems?
2. How does S/4HANA enable companies to better leverage the Internet of Things (IoT) and other emerging technologies?
3. Can you explain how S/4HANA's real-time data processing capabilities can improve decision making?

Let's discuss more about storage type of database tables in SAP HANA

In SAP HANA DB we have two types of tables to store the data.

- ✓ **Row Based**
- ✓ **Columnar Based**



How to decide whether we should create Row or Column?

Row Store:

- ✓ The application must only process one record at a time (many selects and/or updates of single records).
- ✓ Typically, the application requires access to the entire record.
- ✓ Because the columns mostly contain distinct values, the compression rate would be low.
- ✓ Aggregations and fast searching are not required.
- ✓ The table has a few rows (for example, configuration tables).

Column Store:

- ✓ Calculations are typically performed on single or a small number of columns.
- ✓ The table is searched using the values of a few columns.
- ✓ The table has a large number of columns.
- ✓ The table has a large number of rows, so columnar operations (aggregate, scan, and so on) are required.
- ✓ Because the majority of the columns contain only a few distinct values (in comparison to the number of rows), high compression rates can be achieved.

Storage type of Database would give this information whether the table is Row or Column based.

To check: Go To – > SE11 -> provide the Tables -> Click on Display

Database table ☒ View ☐
Data type
Type Group
Domain
Search help
Lock object
Display Change Create

Dictionary: Display Table
Transparent Table **MARA** Partly active
Short Description General Material Data
Attributes Delivery and Maintenance **Fields** Input Help/Check Currency/Quantity Fields
Srch Help Predefined Type

Dictionary: Display Technical Settings
Revised<->Active
Name **MARA** Transparent Table
Short Descript. General Material Data
Last Changed SAP 31.01.2023
Status Actv. Saved
General Properties **DB-Specific Properties**
Storage Type
☒ Column Store ☐ Row Store ☐ Undefined

Table to get the information about all Row store and Column Store?

There is a standard table – **DD09L** in which we would get all the technical settings related information

Go To – SE11 – Provide the table.

1

☒ Database table ☐ View

☐ Data type

☐ Type Group

☐ Domain

☐ Search help

☐ Lock object

2

Dictionary: Display Table

Transparent Table **DD09L** Active

Short Description DD: Technical settings of tables

Attributes Delivery and Maintenance **Fields** Input Help/Check Currency/Quantity Fields

3

Data Browser: Table DD09L Select Entries 1

Table: DD09L

TABNAME	AS4LOCAL	AS4VERS	PUFFERUNG	SCHFELDANZ	PROTOKOLL	SPEICHPUFF	AS4USER	AS4DATE	AS4TIME	TRANSP	Storage Type	Short Descript.	LY ROWORCOLST	SH
MARA	A	0000		000			SAP	18.11.2020	19:52:31		R	Row Store		
											C	Column Store		
												Not Defined		

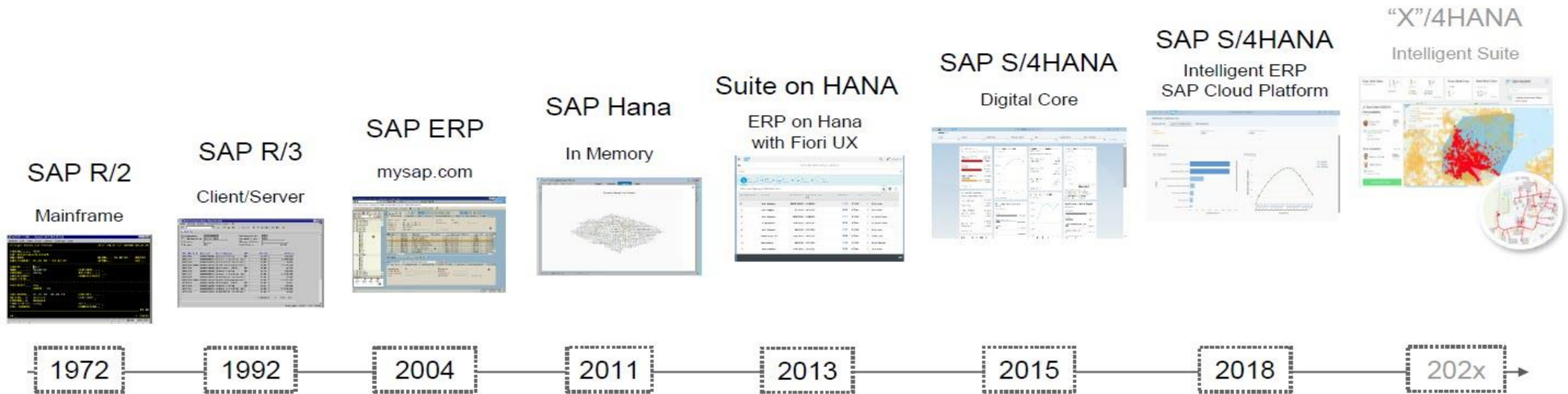
Questions:

- What is the main difference between Row-Based and Column-Based data storage?
- In which type of data storage is the data arranged in rows?
- In which type of data storage is the data arranged in columns?
- What is the advantage of Column-Based data storage over Row-Based data storage?
- How does Column-Based data storage improve the performance of analytical queries?
- How does Column-Based data storage reduce the memory footprint of the database?
- Can you give an example of a business scenario where Column-Based data storage would be more beneficial than Row-Based data storage?
- How does SAP S/4HANA utilize Column-Based data storage in its Simplification Database (SAP HANA)?

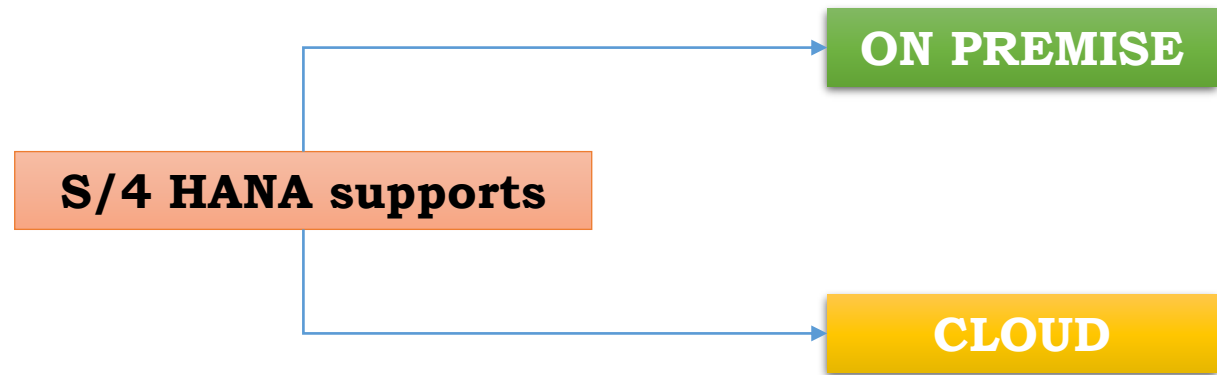
Version in S/4 HANA

S stands for “**SIMPLE**” or “**SUITE**”.
and **4** stands for “**4 Generation**”.

S/4 Application will suite or work only with HANA Database.



For more details: <https://blogs.sap.com/2020/05/30/sap-s-4hana-on-premise-release-version-updates-1511-current-version/>



ON PREMISE

Means servers will be maintained by clients

CLOUD

Means servers will be maintained by third party vendor.[e.g. AWS, SAP CLOUD]

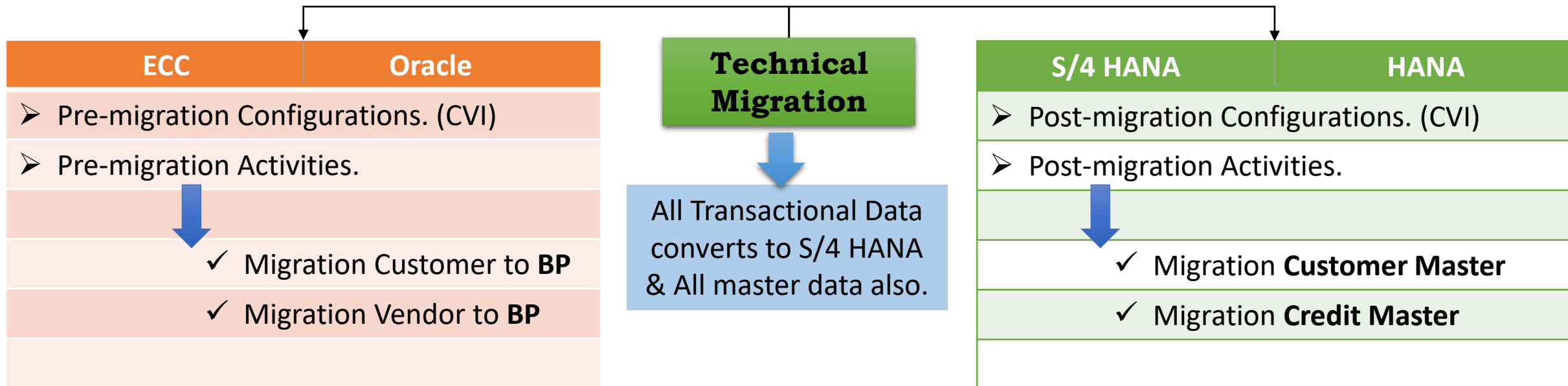
Types of projects in S/4 HANA

Migrations

→ Migrations means converting ECC to S/4 HANA

Green Field Implementation

→ Green Field Implementation means NEW implementation in S/4 HANA.



SD data changes in S/4 HANA

1. Customer Master, Vendor Master, Contact Person, and Credit Master has been converted to BP[**Business Partner**].
2. The **BP role** will be different whether it is customer master, Vendor Master, Credit Master, etc.
3. Status Tables has been eliminated(**VBUK & VBUP**).
4. Pricing Table eliminated **KONV** and replaced with **PRCD_ELEMENTS**.
5. User define condition table numbers almost double.
6. The Access Number in the access sequence has been increased.
7. Document Pricing procedure & Customer pricing procedure length has been increased from **1 to 2**.
8. Group Condition Routine length has been increased from **2 to 7**.
9. Condition counter length has been increased from **2 to 3**.
10. Document category length has been increased from **1 to 4**.
11. The length material number has been increased from **18 to 40**.
12. Credit Management concept has been moved to FSCM[**Finance Supply Chain Management**].
13. The traditional foreign trade from **SD & MM** has been to GTS[**Global Trade Services**] – In path billing concepts.
14. Rebate Process has been converted to “**Settlement Management**”..
15. Billing Output has been moved to BRF[**Business Rule Framework**] which is optional.



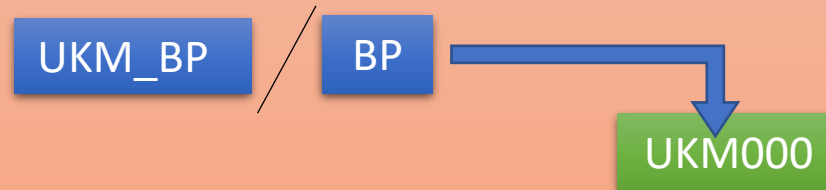
SD data changes in S/4 HANA

16. Advance ATP Tables has been changed from **VBBS** to **VBBE[OMENG/VMENG]**.

17. The T-Code **VKM3** has been eliminated, Now we are using **VKM1** & **VKM4** T-Codes to release the credit block document.

The future eliminated **UKM_MY_DCDS**.

18. The T-Code to maintain **credit limits**.



19. The T-Code for MRP Live is **MD01N**.

Migration Process

1. Basis Consultant will apply "SAP Notes" in ECC System.
After applying SAP notes then the system is ready for migration

ECC/Oracle

1. Pre-migration configuration related to CVI.
2. If CVI link in success.
3. Check upgradation check reports.
4. Check consistency check report.
5. Migrate Customer/Vendors to BP.

Pre-Migration Server

Technical
Migration

Post-Migration Server

S/4 HANA/HANA

1. Post Migration configuration related to BP.
2. Migrate condition master.
3. Cr. Management Configurations.
4. Migrate credit Master.
5. Settlement management configuration
6. BRF+

Migration Process Error Messages(Return Code)

If **Errors < 7** then We can proceed technical & Post-migration.

If **Errors = 7** then We can proceed only up to technical migration.

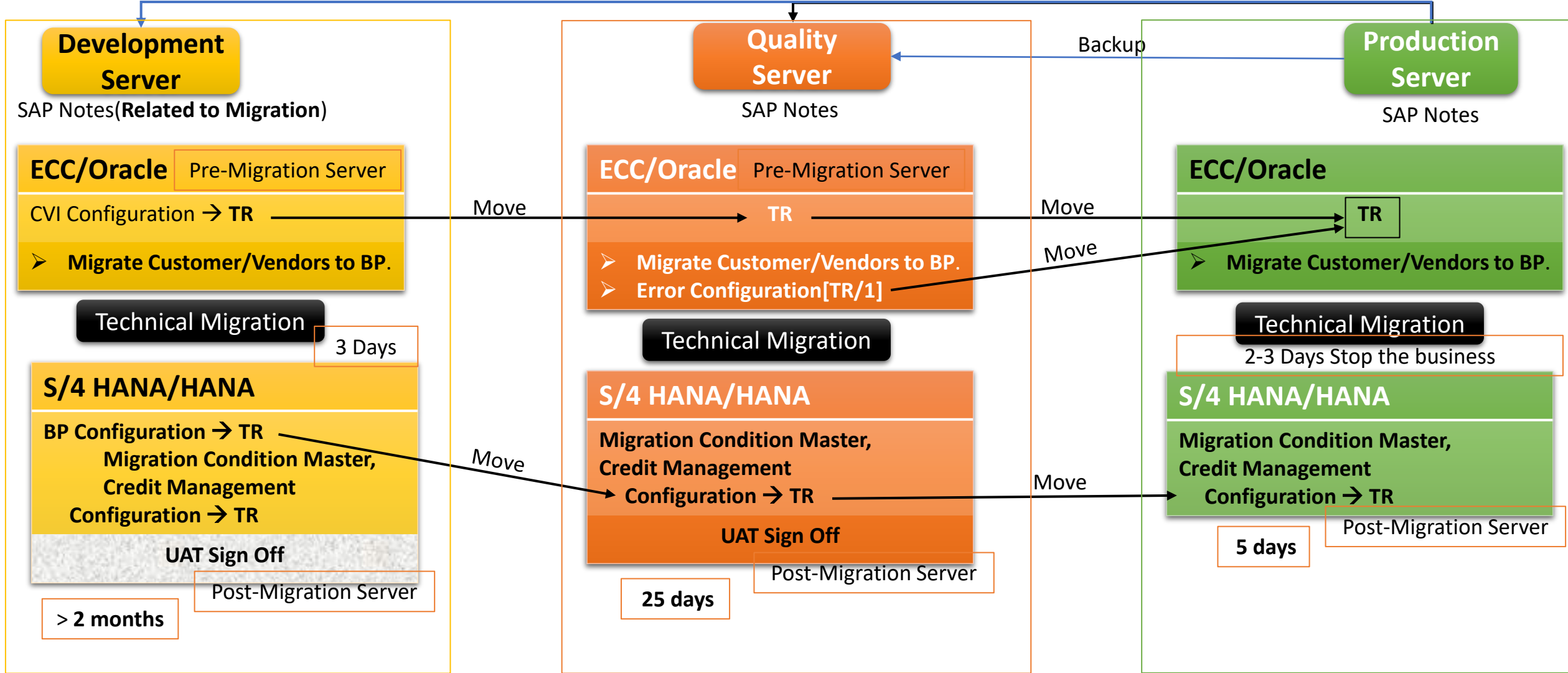
If **Errors > 7** then We can't proceed from here.

Questions:

- 1.What is the technical migration process for SAP S/4HANA?
- 2.What are the system requirements for technical migration to SAP S/4HANA?
- 3.How can you convert custom code during the technical migration process?
- 4.How do you prepare for a technical migration to SAP S/4HANA?
- 5.What is the role of SAP Solution Manager in the technical migration process?
- 6.How do you ensure data integrity during the technical migration process?
- 7.How do you handle the impact of technical migration on existing interfaces and integrations?
- 8.What are the risks and challenges involved in technical migration to SAP S/4HANA?
- 9.What are the benefits of technical migration to SAP S/4HANA?
- 10.How do you measure the success of a technical migration to SAP S/4HANA?



Migration Process Understanding



Migration Process Understanding

Technical Migration done by ABAP & BASIS Consultants.

- Convert all configuration Data, transaction Data & Master Data & Other Than Condition Master & Credit Master will be migrated into S/4 HANA.
- The Time Period for total migration project is **3 to 4 Months**.

Questions:

- 1.What is the technical migration process for SAP S/4HANA?
- 2.What are the system requirements for technical migration to SAP S/4HANA?
- 3.How can you convert custom code during the technical migration process?
- 4.How do you prepare for a technical migration to SAP S/4HANA?
- 5.What is the role of SAP Solution Manager in the technical migration process?
- 6.How do you ensure data integrity during the technical migration process?
- 7.How do you handle the impact of technical migration on existing interfaces and integrations?
- 8.What are the risks and challenges involved in technical migration to SAP S/4HANA?
- 9.What are the benefits of technical migration to SAP S/4HANA?
- 10.How do you measure the success of a technical migration to SAP S/4HANA?



Green Field Implementation

Other than normal implementation, we'll do the below configurations related to S/4 HANA.

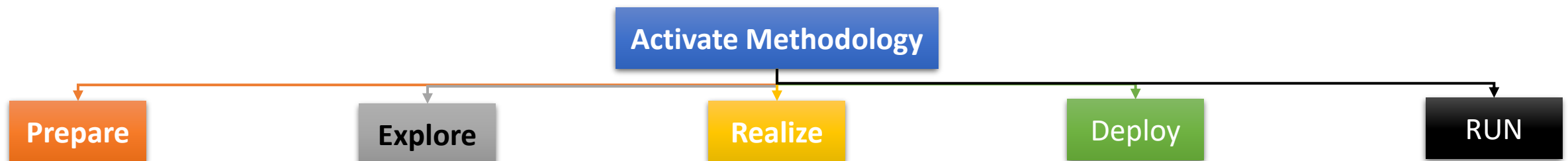
- 1) BP Configurations
- 2) Credit Management Configurations.
- 3) Settlement Management Configurations.
- 4) BRF+ Configurations.

Green Field Implementation Project :

First **"Basis Consultant"** apply **"SAP Notes"**.

→ We follow **"Activate Methodology"**.

→ Activate Methodology will further classified into



Activate Methodology**Prepare**

We have to prepare a model company-structure with relevant best practices & with some sample master data & transaction data. Then we give demo to client in prepare phase.

Explore

Here we do **"Fit & Gap Analysis"** Which process will fit to present client & which process will need to go for Gap or New configurations.

Realize

In this phase, We map the clients business process into S/4 HANA[Configuration]
→ In realization will also do automated testing with **"pre-defined test scripts"**
→ After testing prepare **"User Manual Documents"** & **"Configuration Documents"**
→ Once this is done then go to deploy

Deploy

- In deploy, We have some activities,
- a) User training.
 - b) UAT Sign-Off[User has to test all the scenarios & if everything is fine, then users will give UAT sign-off]
 - c) After UAT sign-off, we will move the TR from quality server to production server.
 - d) Cut Over activities[Uploading the master data & open transaction data from legacy to SAP]
 - e) **GO-LIVE**

RUN

**Post
Production
Support**

CRM[Customer Relationships Management]

Business Partner[BP]

Advantages of BP

1. BP Categories

→ We can create BP as

- a) Person[**Individual**]
- b) Organization[**Customer & Vendor**]
- c) Groups[**Group of Companies**] e.g. TATA Group, Reliance Group, etc.

2. BP is roles bases

→ We can differentiate customer, vendors based on BP Roles.

3. If the customer and the vendor are the same, there is no need to create the customer and the vendor separately; instead, we create the BP once and extend the roles to the customer and the vendor.

4. Address, role, bank data relationship, and so on are **time dependent**.

5. Simple to integrate with other systems such as **CRM** and **GTS [Global Trade Services]**.

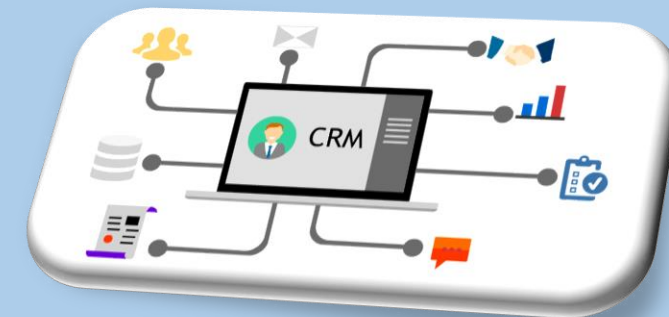
6. It is software that enables businesses to support and define import and export trade processes in SAP-ERP.

7. All T-codes used to create customer and vendor masters have been replaced with a single T-code, **BP**.

8. The table for BP is **BUT000**.

9. The table for customer link is **CVI_CUST_LINK**.

10. The table for vendor link is **CVI_VEND_LINK**.



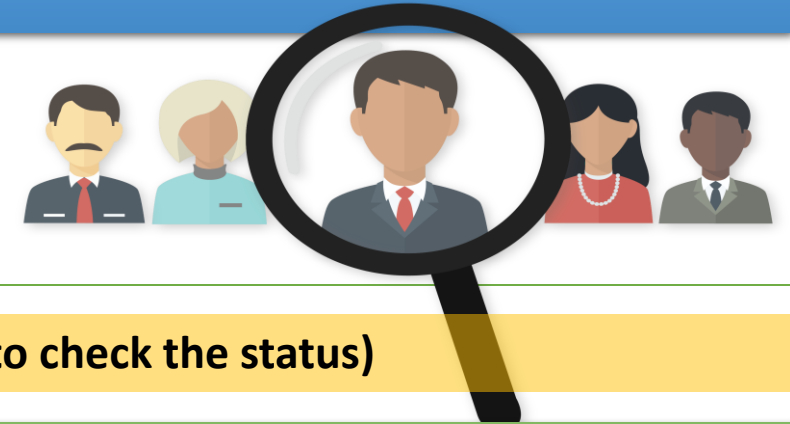
The Standard BP Roles

BP Roles	
Customer FI	FLCU00
Customer SD	FLCU01
Vendor FI	FLVN00
Vendor Purchase	FLVN01
Credit Master	UKM000

1. Customer = BP
Vendor = BP
2. Customer = BP
Vendor $\neq$ BP
3. Customer $\neq$ BP
Vendor = BP
4. Customer $\neq$ BP
Vendor $\neq$ BP

The BP approach design will be taken by client & most of clients will go for 2nd case.

So, Based on this decision, our configurations will be changed.



Scenario while migration customer & vendor to BP(We have to check the status)

Tables	BUT000	CVI_CUST_LINK	
	✓	✓	OK
	✓	✗	OK
	✗	✓	NOT OK

→ In case of 2nd scenario, do the help of adapter & we develop Z-Program & Forcedly updates

PARTNER_GUID & customer information's in **CVI_CUSST_LINK** Table.

→ In case of 3rd scenario, we have to re-run the migration **COCKPIT** & still if the data isn't uploaded in **BUT000** Table. Then we have to raise the issue to SAP.

Green Field Implementation Project

BP configurations in green field implementation project:

1. The Master Data Direction for green field project is

BP → Customer

BP → Vendor

2. Before BP configurations, We have to finalize the BP approach with a client. i.e.,

1. Customer = BP

Vendor = BP

2. Customer = BP

Vendor ≠ BP

3. Customer ≠ BP

Vendor = BP

4. Customer ≠ BP

Vendor ≠ BP

3. In S/4 HANA, All T-Codes related to customer & Vendor have been eliminated

e.g.

{ XD01,XD02,XD03,XD05,XD06,
VD01,VD02, VD03, VD05, VD06,
FD01, FD02, FD03, FD05, FD06. }

Customer Master

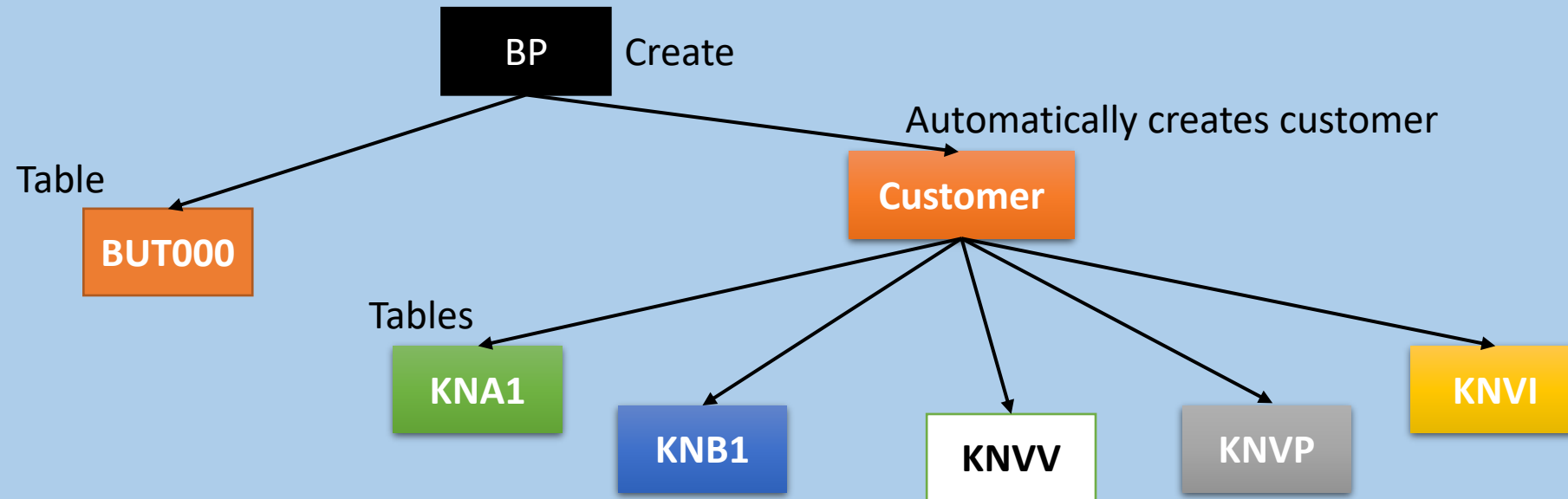
{ XK01, XK02, XK03,MK01,MK02,MK03,FK01,FK02,FK03. }

Vendor Master



Green Field Implementation Project

- There is no change in **customer masters Tables & Vendor master Tables** in S/4 HANA.
- In S/4 HANA System we create We create only **BP** & System Automatically Customers & Vendors in Background.



& Also automatically creates Vendors.

- There is no change in the partner determination configurations.

Code Push Down Approach

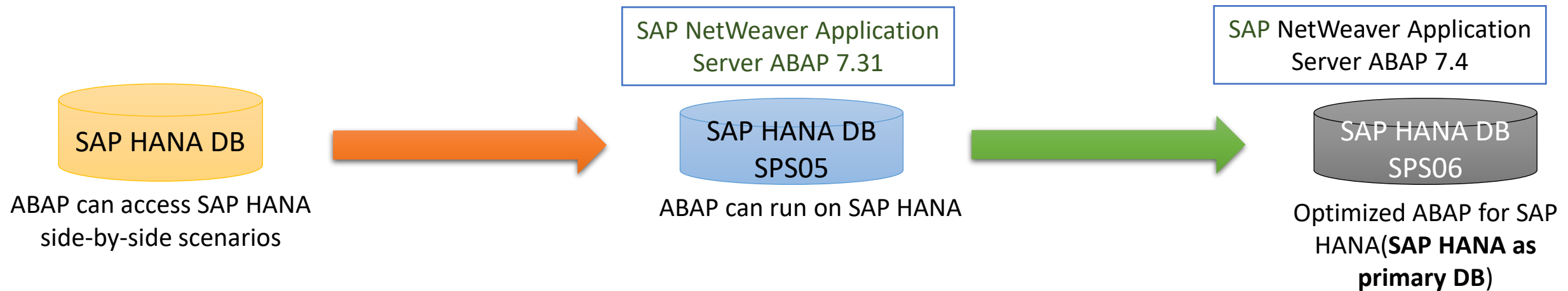
2010 saw the release of SAP HANA, an in-memory, relational database management system.

The use of SAP HANA in-

memory database has been a cornerstone of SAP's new products for a long time and will continue to be so in the future. ABAP has a mature development model and application architecture as the technical foundation of SAP's traditional products.

The CTO of SAP once said, "There is no future in SAP without ABAP, and there is no future with ABAP alone."

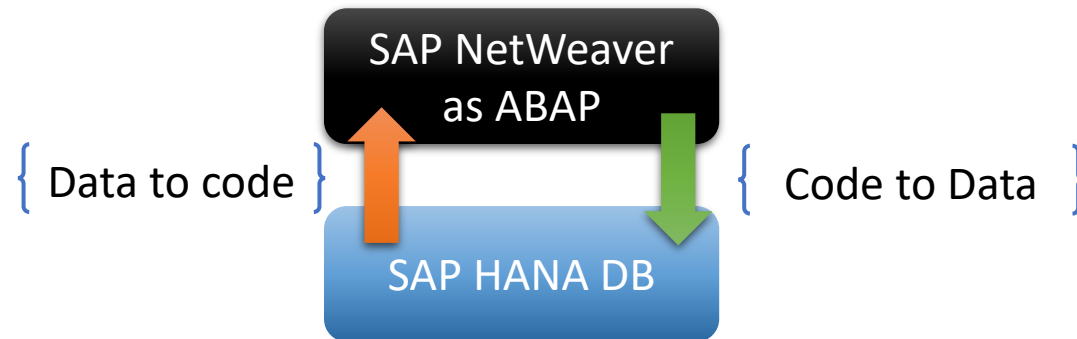
We discovered that ABAP goes hand in hand with the evolution of the SAP HANA database.



Code Push Down Approach

SAP HANA and ABAP have also become the technical architecture of many SAP products. During its evolution, SAP proposed the concept of "**Code Pushdown**."

When compared to the traditional approach, "**Code Pushdown**" is a data-centric framework that means you should execute intensive, expensive computations in the database layer as much as possible to fully utilize HANA's computing power. If the database can perform the calculation logic, we do not recommend putting it in the NetWeaver ABAP application layer. "**Code Pushdown**" enables fast data retrieval and reduces application execution time.



"Code Pushdown" Realization

The "**Code Pushdown**" paradigm can be achieved primarily in four ways:

1. Optimizations that are visible
2. Components should be reused.
3. ABAP advanced SQL
 - **Open SQL enhancement**
 - **CDS view**
4. HANA by **SAP AMDP** and **Native SQL** are examples of native features.

On the other hand, with increased levels of complexity in "**Code Pushdown**" efforts and increased utilization of SAP HANA capabilities.

When possible, most ABAPers embrace code pushdown and may get hands-on or have already worked with it.

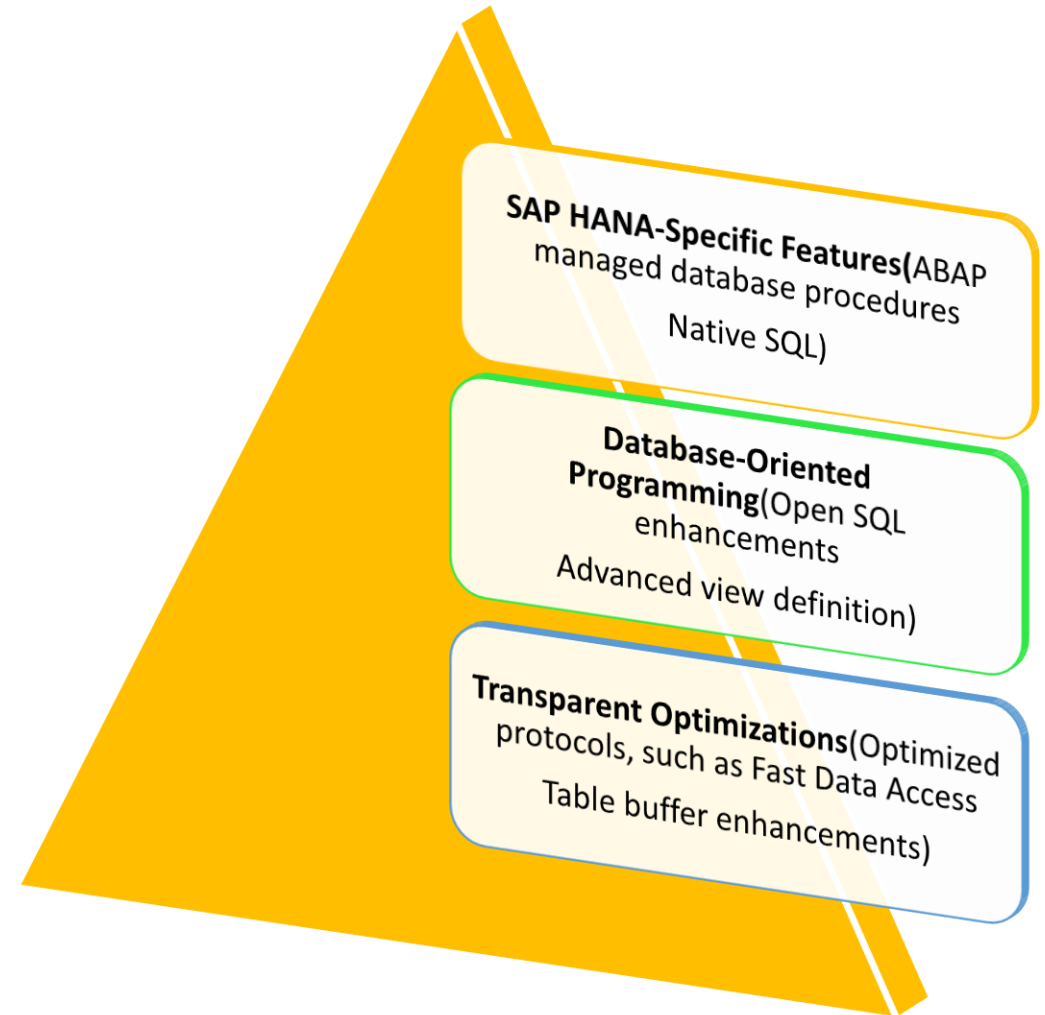


Table Enhancement on ABAP on HANA

Some of the new features in tables in SE11:

☐ Deactivation of Secondary Indexes:

**Do not create this
secondary index
on SAP HANA OB**

Dictionary: Display Index

Index Name: SCUSTOM NAM

Short Description: Search for customer using name

Last changed: SAP 30.03.2016 Original language: DE

Status: Active Saved Package: SAPBC_DATA

Index does not exist in database system HDB

☒ Non-unique index

☐ Index on all database systems

☒ Database-specific Index

☐ Create Index for Selected Database Systems

☐ Selection List

☒ Exclusion List

Database Systems

Index	DBName	Short Description
F	HDB	P HANA Database
M		
N		

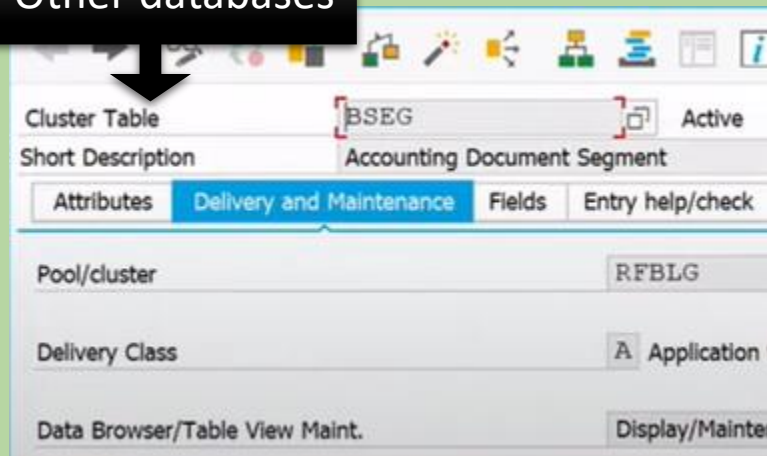
SAP S/4H (1) 100 airdithana INS

Table Enhancement on ABAP on HANA

Some of the new features in tables in SE11:

❑ **De-Pooling and De-Clustering** : There is no cluster & pooled tables in S4-HANA.

In Other databases



In HANA databases

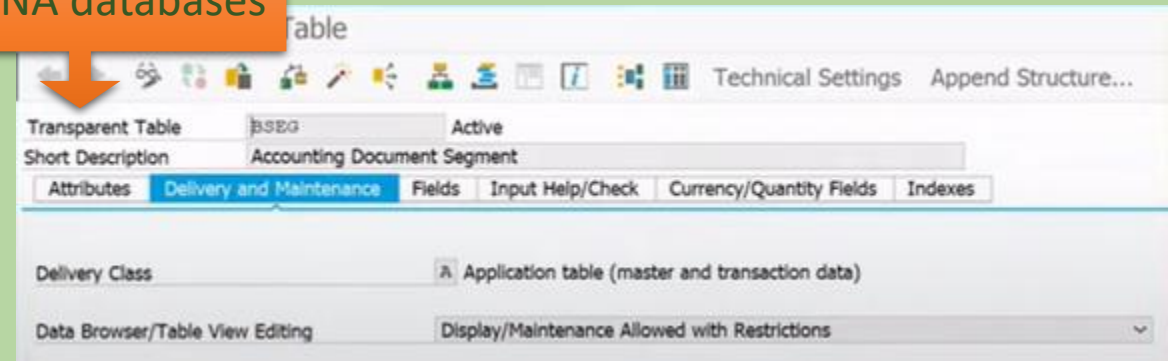


Table Enhancement on ABAP on HANA

Some of the new features in tables in SE11:

❑ Empty Aggregate Tables:

MKPF for document header information and **MSEG** for document item data MARC,MARD and MCHB(**Material Master Date**)

 MSSA containing only aggregated actual stock quantities.

MATDOC : MSEG + MKPF [MSSA not available you have to calculate on fly]

Table	Table description	DDL Source of CDS View for redirect	View to read the content of the database table (w/o redirect to compatibility view)	View to read the master data attributes only
MKPF	Material document header	NSDM_DDL_MKPF	NSDM_V_MKPF	-
MSEG	Material document item	NSDM_DDL_MSEG	NSDM_V_MSEG	
MARC	Plant Data for Material	NSDM_DDL_MARC	NSDM_V_MARC	V_MARC_MD

Table Enhancement on ABAP on HANA

Some of the new features in tables in SE11:

❑ **Some fields have their new replacements:** Go to the table in SE11> Extra > Replacement Object

The screenshot shows the SAP SE11 interface. The 'Extras' menu is open, highlighting the 'Replacement Object...' option. The main window displays the 'MKPF' table (Header: Material Document) with the 'Fields' tab selected. A dialog box titled 'Display Replacement Object for MKPF' is open, showing a table with two rows: 'Table' (MKPF) and 'Replacement Object' (NSDM_E_MKPF). A green arrow points from the 'Replacement Object' field to the 'Table' field, indicating a replacement relationship. The dialog also includes a green checkmark and a red X button.

Field	Key	Initi...	Data element	Data Type	Length	Decim...	Coord
MANDT	✓	✓	MANDT	CLNT	3	0	
MBLNR	✓	✓	MBLNR	CHAR	10	0	
MJAHR	✓	✓	MJAHR	NUMC	4	0	
VGART	□	□	VGART	CHAR	2	0	
BLART	□	□	BLART	CHAR	2	0	
BLAUM	□	□	BLAUM	CHAR	2	0	
BLDAT	□	□	BLDAT	DATS	8	0	

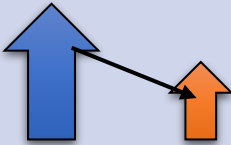
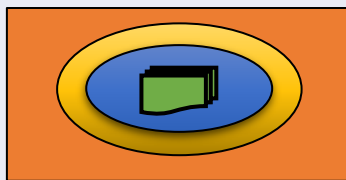
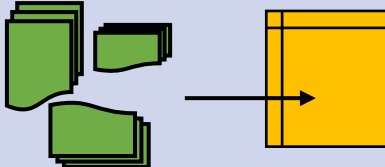
Field	Table	Replacement Object
MANDT	MKPF	
MBLNR		NSDM_E_MKPF
MJAHR		
VGART		
BLART		
BLAUM		

Performance Guidelines for SAP HANA Database ABAP Development

One of the most frequently asked questions consultants get is:

"How do these guidelines change in the context of SAP HANA?"

Let us first take a look at the current guidelines. They are typically summarized in the **"5 golden rules"**:

Understanding	Rule	Details/Examples
	Keep the result sets small	➤ Do not retrieve rows from the database and then discard them on the application server with CHECK or EXIT, as is common in SELECT loops. ➤ WHERE clause should be as specific as possible.
	Minimize amount of transferred data	➤ Use SELECT with a field list instead of SELECT * to transfer only the columns you need. ➤ Instead of transferring all rows to the application server, use aggregate functions (COUNT, MIN, MAX, SUM, AVG).
	Minimize the number of data transfers	➤ Instead of nested SELECT loops, use JOINS and / or sub-queries. ➤ Instead of using multiple SELECTs or SELECT SINGLES, use SELECT... FOR ALL ENTRIES. ➤ Use INSERT, UPDATE, MODIFY, and DELETE commands.
	Minimize the search overhead	➤ Create and implement necessary secondary indexes.
	Keep load away from the database	➤ Avoid reading data multiple times. ➤ If possible, use table buffering rather than bypassing it. ➤ Sorting Data in ABAP Programs

Performance Guidelines for SAP HANA Database ABAP Development

So, the short answer to the pervious question is

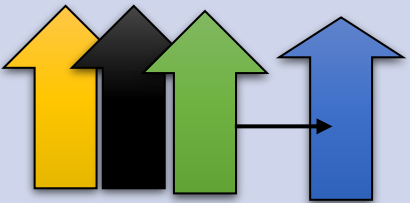
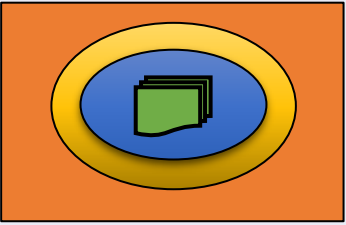
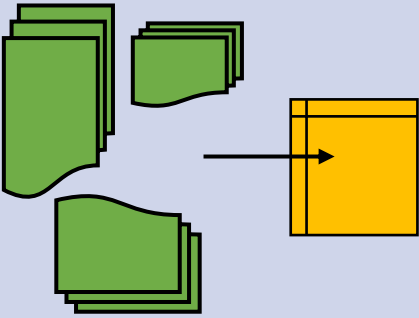
- ✓ All existing (standard and custom) ABAP code runs unchanged on SAP HANA.
- ✓ As a general recommendation, all existing guidelines are still valid for SAP HANA.

However, **the priorities of some rules are shifting**, i.e. some aspects are less important due to the nature of the In-Memory Column Store, but there are also certain non-optimal coding patterns with a greater impact on SAP HANA. Furthermore, there are completely **new opportunities for SAP HANA performance tuning** that were previously unavailable, such as pushing complex operations to the database.

The recommendations are therefore based on measurements and experiences with **SAP Business Suite** and **SAP NetWeaver AS ABAP 7.4** running on **SAP HANA SPS5**.

They will be included in the standard **ABAP 7.4 documentation**, as well as supported by standard tools such as the ABAP code inspector.



Understanding	Additions in the context of SAP HANA
	Every database access, as with all database systems, incurs a performance overhead for connection handling, SQL parsing, execution plan determination, and so on. The following existing SAP HANA guidelines should be prioritized higher: ➤ When changing many data records, array operations should be preferred over single operations for modifying operations (INSERT, UPDATE, DELETE). ➤ If possible, avoid nested SELECT loops by changing the nested single SELECT statement to an appropriate SQL construct (e.g. FOR ALL ENTRIES, JOIN, sub-query, etc.) ➤ avoiding repeated SQL access to the same data ➤ Making use of the ABAP table buffer (see existing guidelines for the ABAP table buffer)
	In most cases, secondary indexes are not required for good search performance in SAP HANA. All existing non-unique secondary database indices on columnar tables are removed during migration or are not created during installation for all AS ABAP systems starting with SAP NetWeaver 7.4 to reduce main memory consumption and improve insert performance. Secondary indexes can still be useful in some cases. This is especially true for highly selective queries on fields other than the primary key. Indexes on single fields that are most selective can significantly improve these queries. SAP Note 1794297 describes how to locate and create these indexes.
	It is advantageous to move data-intensive calculations into the database when using SAP HANA. However, it is not recommended to repeat the same operations, such as in different user contexts or different dialogue steps for the same user. On the application server, results should be meaningfully buffered. In this regard, the following SAP HANA recommendation should be considered. Sorting data: In general, the recommendations for sorting remain the same as before, i.e. if the database does not use the same index for sorting as for selection, then it may be more efficient to sort in the application server in some situations, particularly if all data to be sorted must be fetched from the application server anyway. However, if the sorting is part of determining the result set (e.g., select top n customers based on revenue) or is part of a larger calculation logic (e.g., within a procedure), it should be performed in SAP HANA.

Client

Client is **3 digit** numeric value. Client is used segregate the data within system. Each client in a system will have it's own Master data , Transaction Data and User Data.

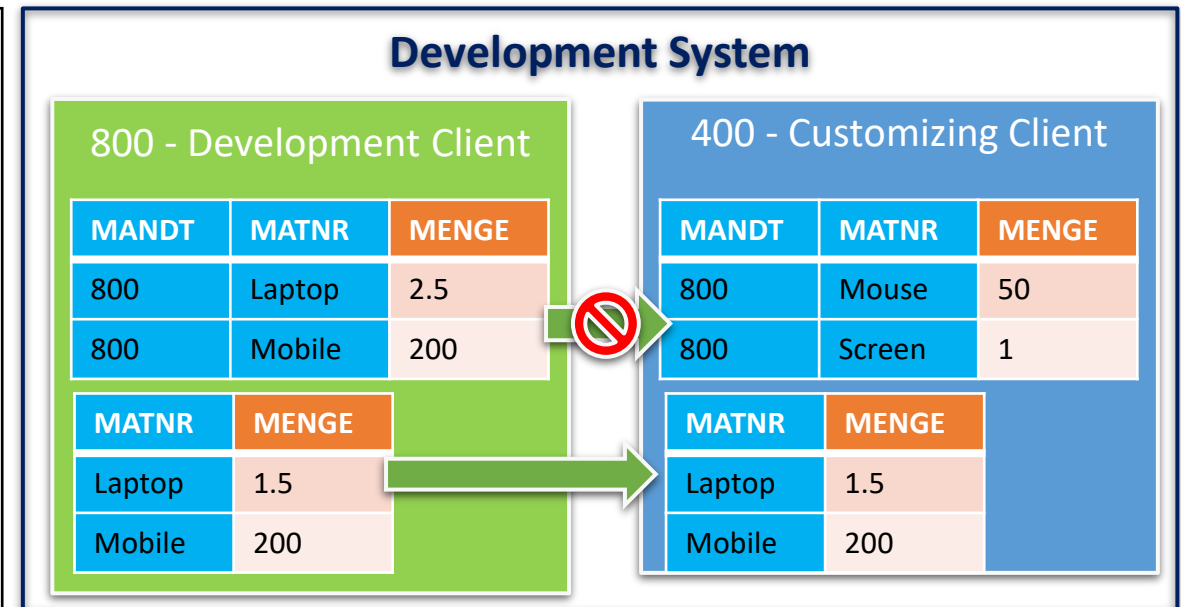
Client number can any number between **000 to 999**.

Client Dependent Data: If MANDT field is a part of key field, Then table data is accessible within the client

Cross-Client Data: If MANDT field isn't a part of key field, Then table data is accessible across the system.

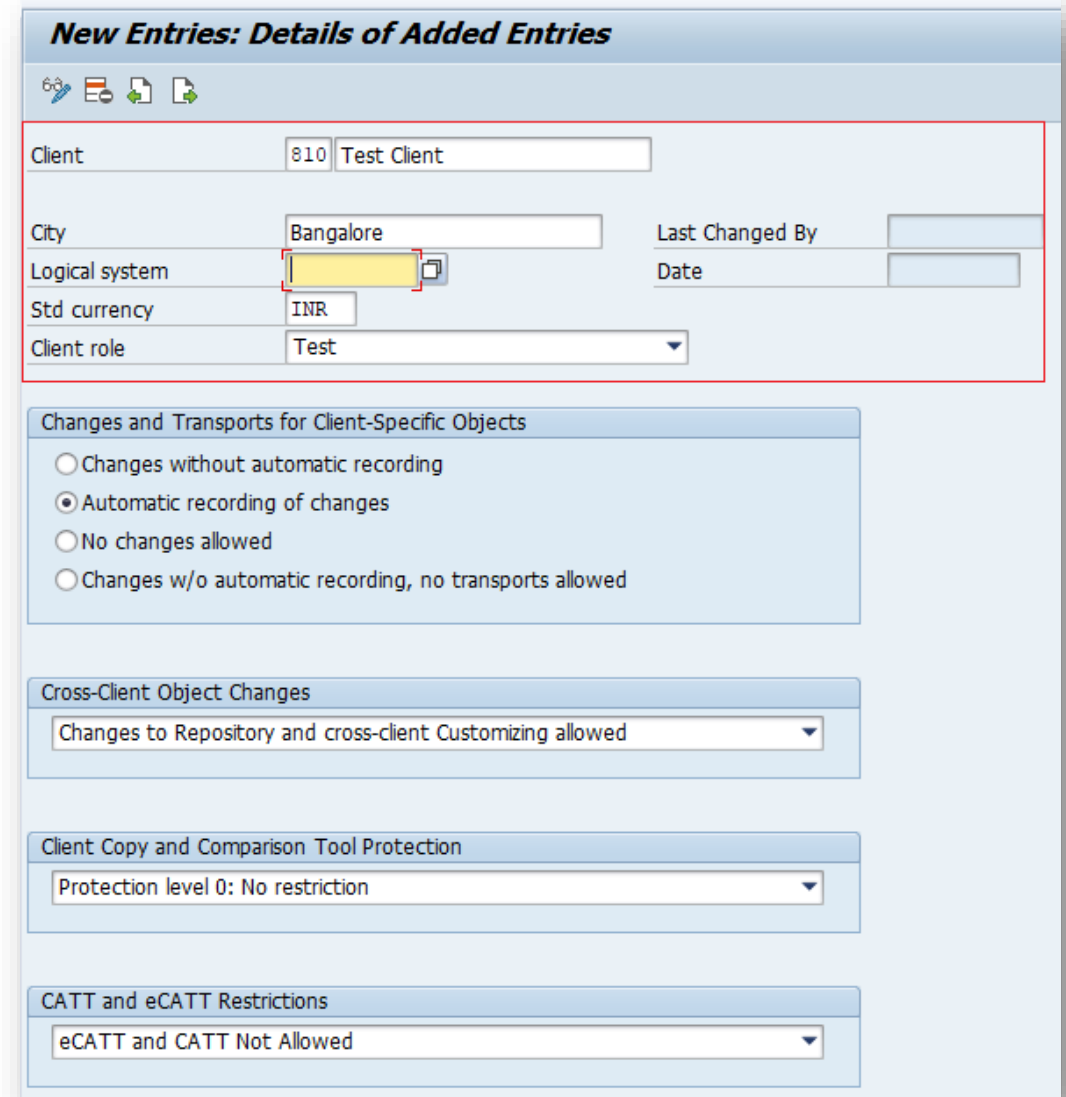
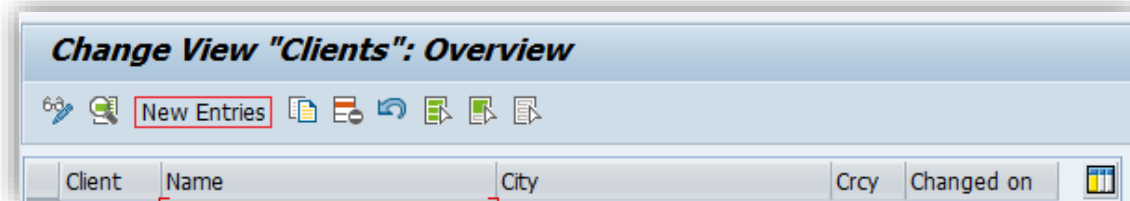
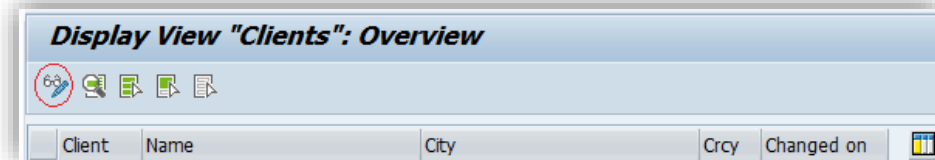
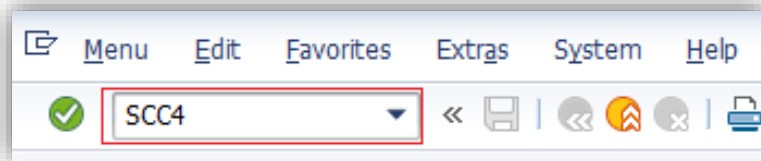
- 1. Master Data:** Data will be used frequently and updated rarely(Ex: Customer Master Data, Vendor Master Data, Material Master Data).
- 2. Transaction Data:** Transaction data can be updated frequently (Purchase Order, Sales Order, Delivery Data, Invoice etc...)
- 3. User Data:** User Details(Ex: Login Information, Authorization).

ABAP Programs are client In-dependent, Database Table, Domain, Function Module



How to create new SAP client

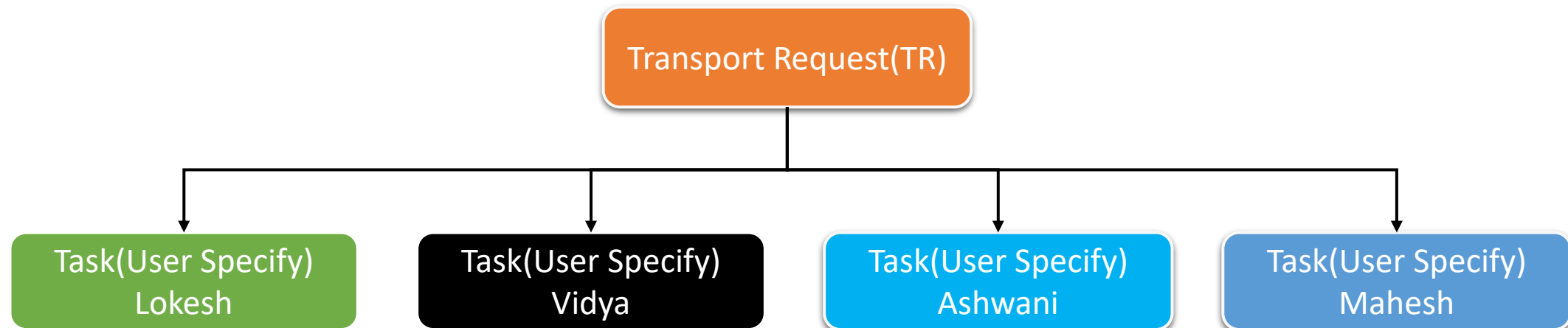
You can create new clients as per the organizational requirements in SAP Basis by using transaction code SCC4.

A screenshot of the 'New Entries: Details of Added Entries' screen. The screen is divided into several sections. The top section contains fields for Client (810), Test Client, City (Bangalore), Logical system (highlighted in yellow), Std currency (INR), Client role (Test), Last Changed By, and Date. Below this is a section for 'Changes and Transports for Client-Specific Objects' with four radio button options: 'Changes without automatic recording', 'Automatic recording of changes' (selected), 'No changes allowed', and 'Changes w/o automatic recording, no transports allowed'. The next section is 'Cross-Client Object Changes' with a dropdown menu set to 'Changes to Repository and cross-client Customizing allowed'. The 'Client Copy and Comparison Tool Protection' section has a dropdown menu set to 'Protection level 0: No restriction'. The 'CATT and eCATT Restrictions' section has a dropdown menu set to 'eCATT and CATT Not Allowed'.

Transport Request

What exactly is a Transport Request?

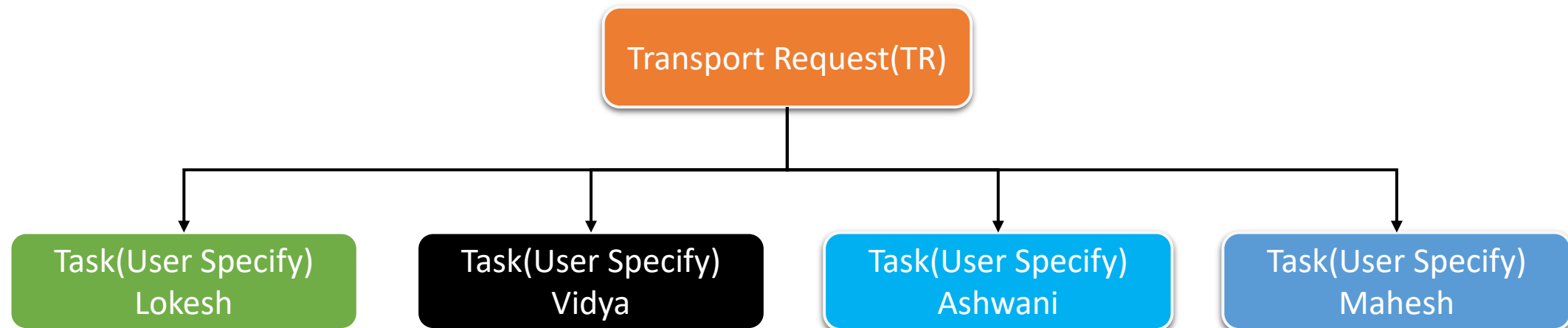
- Transport Requests (TRs) are a kind of '**Container / Collection**' of development system changes. It also keeps track of the type of change, the purpose of transportation, the requested category, and the target system. Change Requests are another name for it.
- Each TR contains one or more change jobs, also referred to as change **Tasks (minimum unit of transportable change)**. Tasks are stored within a TR in the same way that multiple files are stored in a folder. TR can only be released once all tasks contained within it have been completed, released, or deleted.
- A Change Task is a list of objects that have been modified by a specific user. Each task can only be assigned (and released by) one user. However, each Transport Request can have multiple users assigned to it (as it can contain multiple tasks). **Tasks cannot be transported on their own, but only as part of TR.**



Transport Request

What exactly is a Transport Request?

- Transport Requests (TRs) are a kind of '**Container / Collection**' of development system changes. It also keeps track of the type of change, the purpose of transportation, the requested category, and the target system. Change Requests are another name for it.
- Each TR contains one or more change jobs, also referred to as change **Tasks (minimum unit of transportable change)**. Tasks are stored within a TR in the same way that multiple files are stored in a folder. TR can only be released once all tasks contained within it have been completed, released, or deleted.
- A Change Task is a list of objects that have been modified by a specific user. Each task can only be assigned (and released by) one user. However, each Transport Request can have multiple users assigned to it (as it can contain multiple tasks). **Tasks cannot be transported on their own, but only as part of TR.**



Transport Request

Workbench Request - a collection of repository objects as well as 'cross-client' customization objects. These requests are in charge of changing the ABAP workbench objects.

Transport Request
(Workbench/Customizing)

Customizing Request - Objects belonging to 'client-specific' customizing. These requests are automatically recorded based on client settings when users customize settings, and a target system is automatically assigned based on the transport layer (if defined).

Transport Request

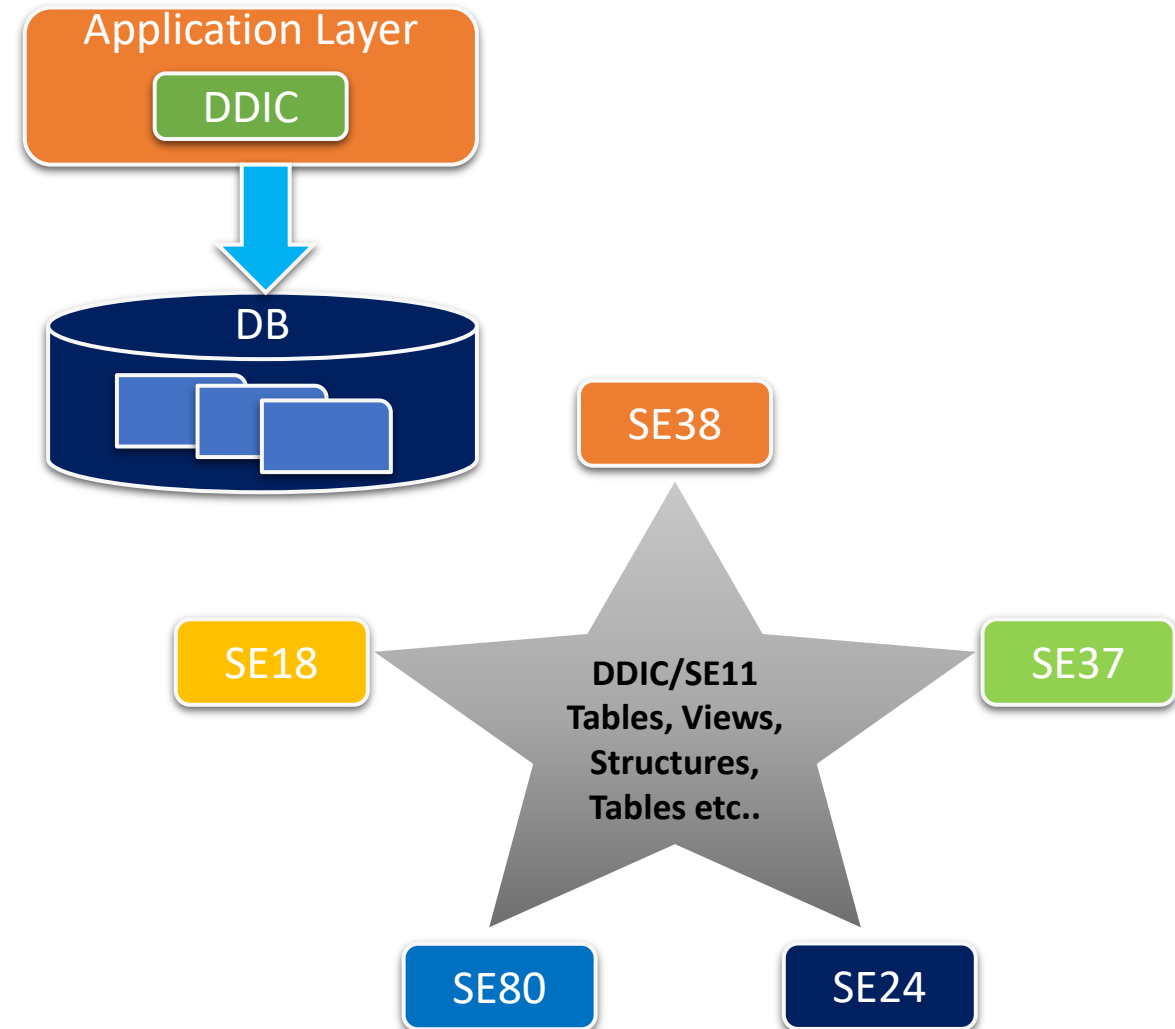
All About Transport Request

- ✓ What is Transport Request?
- ✓ What is Data File Cofile?
- ✓ What are the types of Transport Requests?
- ✓ What is Transport Domain Controller?
- ✓ How to Configure Transport Domain Controller?
- ✓ Transport Route?
- ✓ What are the types of Transport Route Editors?
- ✓ What is R3 Trans?
- ✓ STMS Configuration?
- ✓ How to import a Transport Request?
- ✓ Transport Request Options?
- ✓ What are the Transport Return Codes?
- ✓ What are the Transport Logs?
- ✓ Transport Status?
- ✓ How to change the status of Releases TR?
- ✓ How to Deactivate Fully Loaded (Import all) Truck?
- ✓ Transport Tables?



Data Dictionary/ABAP Dictionary/DDIC → SE11, SE12

- DDIC centrally describes and manages all the data definitions used in a system. Data Dictionary seamlessly integrated with ABAP Work Bench.
- DDIC helps to create objects centrally so that we can avoid creating the same object again and again
- DDIC Define the user defined data type(Domain, Data Element, Structure, Table Type etc..)
- DDIC describes the structure of the database object(Database Table, Views, Index) in the system. When we create a database table and activate the system will also automatically creates the corresponding database object in the database.

**Interview Question?**

What is data dictionary?

What object you will create in Data Dictionary?

Data Dictionary/ABAP Dictionary/DDIC → SE11, SE12**Dictionary Tasks****Object Definition**

The DDIC serves as a central repository for the definition of database objects such as tables, views, structures, and data elements. This includes the definition of these objects' data types, lengths, and other properties.

Data Integrity

To ensure the consistency and accuracy of the data stored in the database, the DDIC enforces referential integrity constraints and data validation rules.

Code Generation

The DDIC provides tools for generating database-independent ABAP code, which simplifies database migrations and improves SAP application portability.

Data Access

The DDIC provides a centralized and consistent way to access and manage data in the database, simplifying the development and maintenance of SAP applications.

Security

The DDIC includes mechanisms for restricting database object access and protecting sensitive data from unauthorized access.

Performance Optimization

The DDIC includes features such as indexing and caching that help to improve the performance of database operations in SAP.

Domain

- Domain Provides **technical attributes** to the data element.
- Domain provides **Technical Attributes**.
 - **Technical Attributes:** Data Type + Data Length, Sign, Lower Case(Case-Sensitive), Value Range, Conversion Routine.

Dictionary: Display Domain

Domain: KAPPL Active

Short Description: Condition application

Properties Definition Value Range

Format

Data Type	CHAR	Character String
No. Characters	2	
Decimal Places	0	

Output Characteristics

Output Length	2
Routine	

☐ Sign

☐ Case-sensitive

Used restrict the input values that's entered into table

It will convert data from internal storage format to external display format and Vice versa.

If you want store the data exactly the same way user enter, then you have t select Case-Sensitive checkbox.

You can select this checkbox to store negative sign in DB table

Data Element

- ✓ Data Element provides semantic + Technical attributes to Tables Field, Screen Fields, Structure or Table Type.
- ✓ **Semantic Attributes:** F1(Documentation), F4(Search Help) and Field Label to Screen Field, Table Field etc.

Field Labels are used to provide the description for table field, screen field. If you don't provide field label and if you have used that data element to create database table and for that if you have created and TMG then instead showing the description for the column header, it will display + sign

Change document will store the information about the old value and new value. When you select change document checkbox in data element, If any change happens to the value of that field then system will create a change document.

Change Document Table: CDHDR/CDPOS

Dictionary: Display Data Element

← 🔍 🔄 ⚙️ 📄 📊 📑 ⓘ 🗑️ 📁 Documentation

Data element KSCHA Active

Short Description Condition Type

Attributes Data Type Further Characteristics Field Label

Search Help

Name

Parameters

Parameter ID VKS

Default Component Name COND_TYPE

☒ Change Document

☐ No Input History

Bi-Directional Options

☐ Basic direction is set to LTR

☐ No BIDI Filtering

If you don't want to store input history of sensitive field then you can select this check box.

Data Element	
--------------	--

This flag can be used to prevent the Unicode formatting characters that specify the writing direction of bidirectional fonts in data transports between GUI and AS ABAP from being filtered out, regardless of the global system settings. The global system setting is made in the program

Dictionary: Display Data Element

← 🔍 🔄 📁 ✨ 🗑️ 📊 📄 ⓘ 🖨️ 📌 📎

Data element: **KSCHA** Active

Short Description: Condition Type

Attributes | Data Type | **Further Characteristics** | Field Label

Search Help

Name:

Parameters:

Parameter ID: VKS

Default Component Name: COND_TYPE

☒ Change Document

☐ No Input History

Bi-Directional Options

☐ Basic direction is set to LTR

☐ No BIDI Filtering

This flag can spe a dynpro field as overwrites the d

The short, medium, and long field labels assigned to a data element, plus a header, can be created where appropriate as descriptions or column headers of lists for fields on the screens of dynpros or Web Dynpros with direct or indirect reference to the data element. A freely definable maximum length with varying upper limits for the different field labels is applied to all languages and must be set accordingly.

This flag can specify the writing direction of a dynpro field as "From left to right", which overwrites the direction "From right to left" specified by logons in a bidirectional font. This flag can be useful for fields with numeric texts.

Database Table

- DB Tables are used to store the data in underlying database in Row and Column format.
- When we activate table definition in SE11 system will create database table with same structure in underlying DB.

Types of Table:**Transparent Table:**

1. In the dictionary, a transparent table has a one-to-one relationship with a table in the database.
2. In R/3 Data Dictionary, its structure corresponds to a single database table.
3. There is one associated table in the database for each transparent table definition in the dictionary.
4. The database table shares the same name, number of fields, and field names as the R/3 table definition.
5. They serve to store application data.
6. An application's master data or transaction data is referred to as application data.

Pool Table:

1. A pool table is another SAP proprietary that has a many-to-one relationship with a database table.
2. There are many tables in the Data Dictionary for each table in the database.
3. The table in the database has a different name than the tables in the DDIC, as well as a different number of fields and names.

A cluster table

1. is a SAP proprietary that has a many-to-one relationship with a database table.
2. Many cluster tables are stored in a single database table known as a table cluster.
3. It has a large number of cluster tables.
4. Tables in a cluster table share a portion of their primary keys.

Database Table

Restrictions on Pooled and Cluster Tables: -

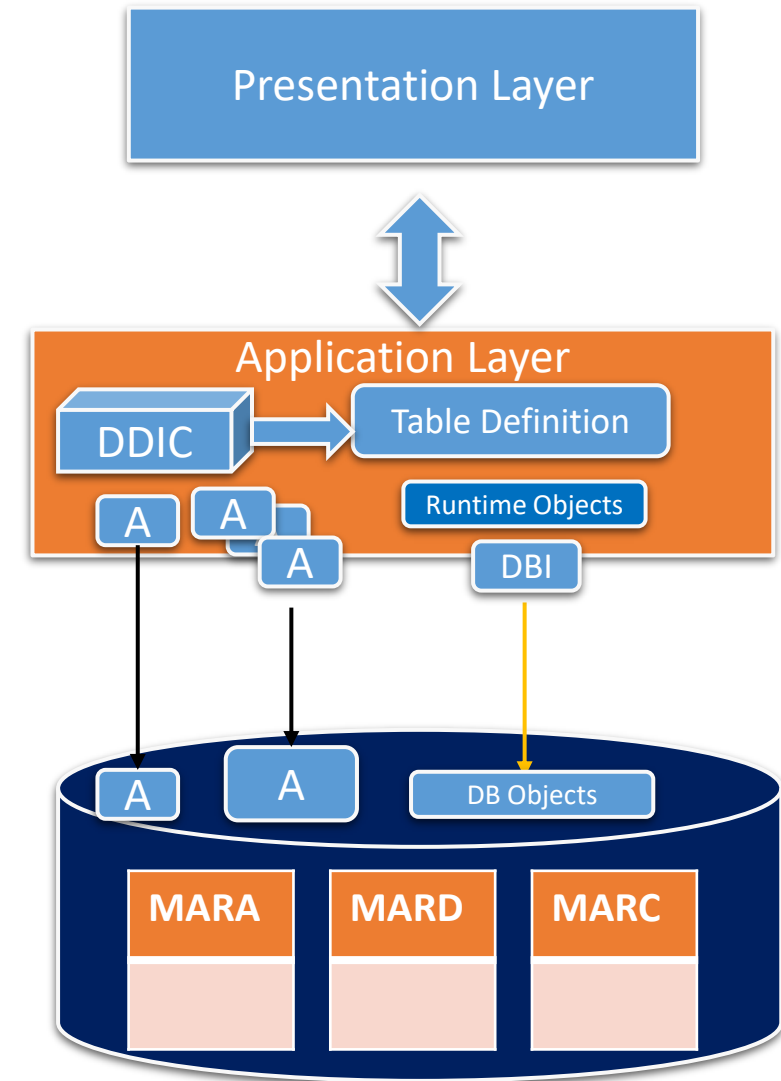
1. Pooled and cluster tables are generally used only by SAP and not used by developers,
due to proprietary format of these tables within the database and technical constraints to use within ABAP/4 programs.
2. Secondary Indexes cannot be created on Pooled and Cluster Tables.
3. Can't be use ABAP/4 SQL construct 'Select Distinct' or 'Group By'.
4. Native SQL cannot be used on Pooled and Cluster Tables.
5. Field names cannot be specify after the 'Order By' clause. 'Order By' with primary key is only allowed.

Note: You can't use a pooled table with a join. In case of Table Pool and Table Cluster, within a technical setting you can only change SIZE.

Field: Field Name can be up to 30 characters, it can contain Number, Underscores and Alphabets

Key field:

- It's is mandatory field of the table, Used to identify the unique record of the table
- When we activate table DDIC will create Primary Index using Key



Database Table

The Delivery Class is used to define the type of data that will be stored in our table.

- The data could be business data or application data (Master and transaction data).
- For our delivery class, we will select the application data.

Display/Maintenance: This option allows us to choose whether we only want to display the data or also want to keep it. In this context, data maintenance refers to the creation, deletion, or updating of data.

For this field, there are three options:

- **Display/maintenance allowed**
- **Display/maintenance not allowed**
- **Display/maintenance allowed with restrictions**

Dictionary: Display Table

Transparent Table: MARA Partly active

Short Description: General Material Data

Attributes | **Delivery and Maintenance** | Fields | Input Help/Check | Currency/Quantity Fields

Delivery Class: A Application table (master and transaction data)

Data Browser/Table View Editing: Display/Maintenance Allowed with Restrictions

Delivery class for controlling data transport of tables for installation, Upgrade and Client Copy and for transporting data between customer system.

This option is used to allow us that whether we want only display the data or also want to maintain the data.

Database Table

Dictionary: Display Technical Settings

Revised<->Active

Name	MARA	Transparent Table
Short Descript.	General Material Data	
Last Changed	SAP	31.01.2023
Status	Actv.	Saved

General Properties DB-Specific Properties

Logical Storage Parameters

Data Class	APPL0	Master Data, Transparent Tables
Size Category	4	Expected Data Records 14.000 to 57.000

Buffering

☒ Buffering Not Allowed
☐ Buffering allowed but switched off
☐ Buffering Activated

Buffering Type

☐ Single records buff.
☐ Generic Area Buffered
☐ Fully Buffered

Number of Key Fields 0

☐ Log Data Changes
☐ Writes only with JAVA

Defines the physical area of the database table (Table Space) in which table space my table should be created.

Used to define/reserve the initial memory of the table.



Database Table

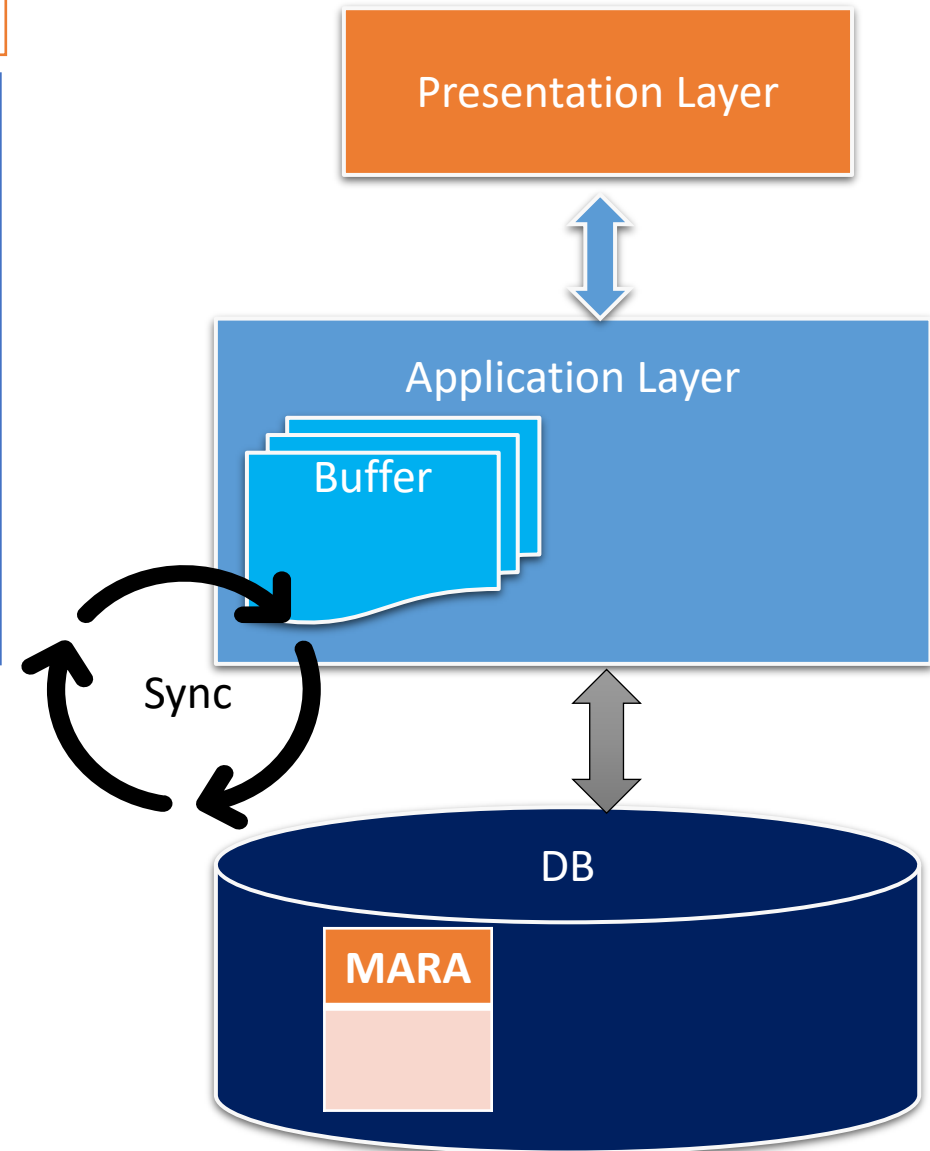
Buffering: Keeping the data in application layer for temporary – AL12

Buffering Types

1. **Single Record Buffered**
2. **Generic Area Buffered**
3. **Fully Buffered**

For what type of table you should activate?

We can enable buffering for table. Which are updated rarely.



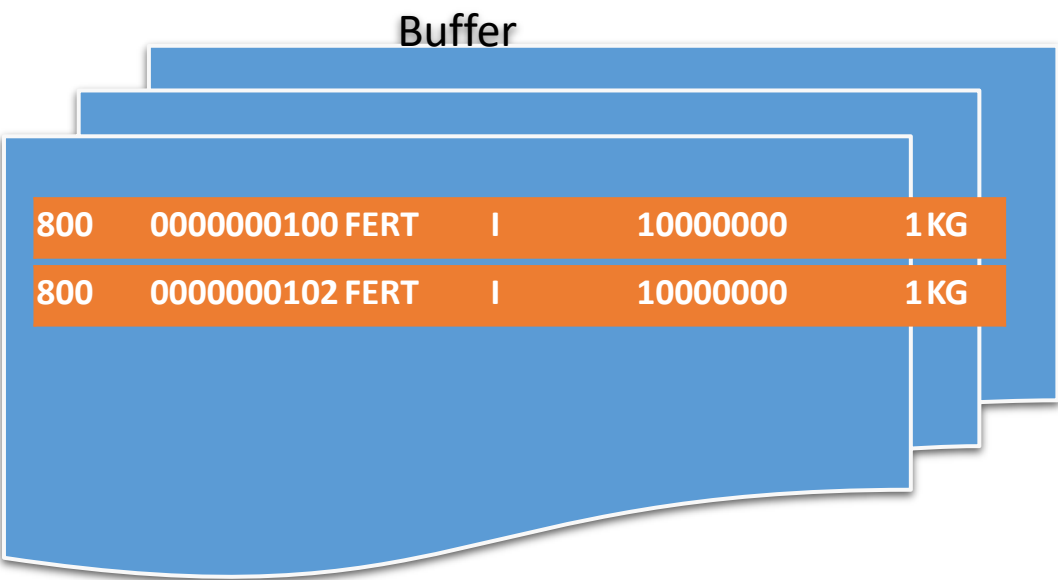
Database Table

Single Record Buffered

```
SELECT SINGLE * FROM MARA
      INTO WA_MARA
      WHERE MATNR = '102'
      AND MTART = 'FERT'.
```

When you select single record buffer only the records which is access are loded into buffer memory. All the key field of table should be include key field select statement

When a table having huge number of entries and business only interested in few entries of an table then we can go with single record buffering



MANDT	MATNO	MAT_TYP	INDSECT	MATKL	GROSWT	UOM
800	0000000100	FERT	I	10000000		1 KG
800	0000000100	HAWA	I	10000000		1 KG
800	0000000101	FERT	I	10000000		1 KG
800	0000000102	FERT	I	10000000		1 KG
800	0000000103	FERT	I	10000000		1 KG
800	0000000104	FERT	I	10000000		1 KG
800	0000000105	FERT	I	10000000		1 KG

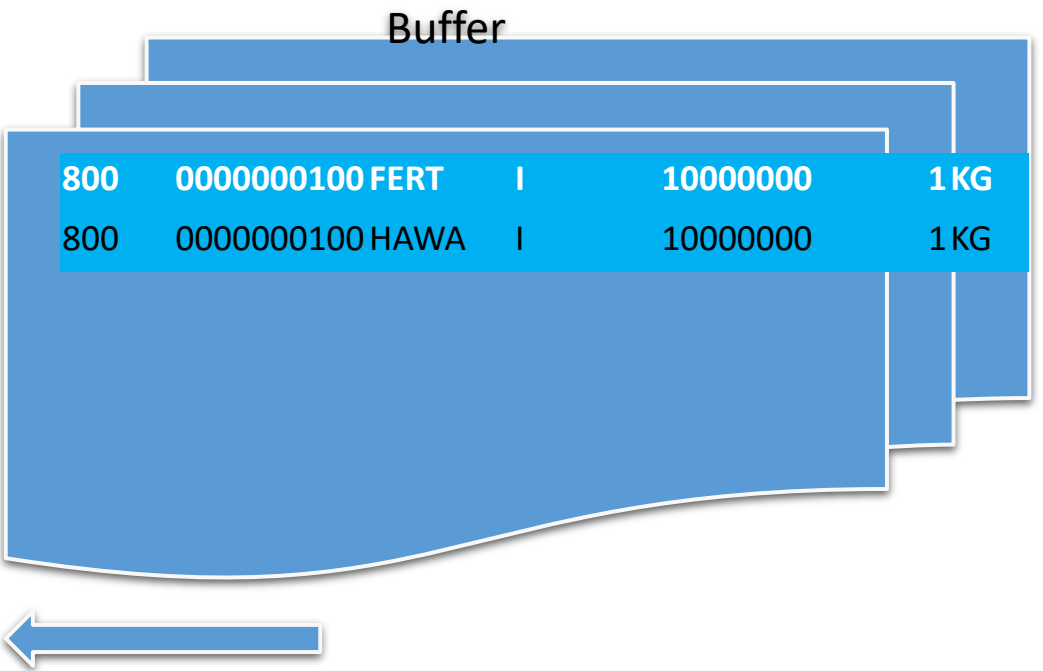
Database Table

Generic Area Buffered

```
SELECT SINGLE * FROM MARA
      INTO WA_MARA
      WHERE MATNR = '100'
      AND MTART = 'FERT'.
```

When we select generic are buffer, when a record of a tables accesses system will load all the records which are part of generic key field

A table should buffered generically if only certain generic areas of table are normally needed for processing



MANDT	MATNO	MAT_TYP	INDSECT	MATKL	GROSWT	UOM
800	0000000100	FERT	I	10000000		1 KG
800	0000000100	HAWA	I	10000000		1 KG
800	0000000101	FERT	I	10000000		1 KG
800	0000000102	FERT	I	10000000		1 KG
800	0000000103	FERT	I	10000000		1 KG
800	0000000104	FERT	I	10000000		1 KG
800	0000000105	FERT	I	10000000		1 KG

Database Table

Fully Buffered

```
SELECT SINGLE * FROM MARA
      INTO WA_MARA
      WHERE MATNR = '100'
      AND MTART = 'FERT'.
```

When we access any row of an table system will load entire table data of table into buffer memory.

Table having small number of entries and all time all the records of table needed.

Buffer						
MANDT	MATNO	MAT_TYP	INDSEC	MATKL	GROSWT	UOM
800	0000000100	FERT	I	10000000	1 KG	
800	0000000100	HAWA	I	10000000	1 KG	
800	0000000101	FERT	I	10000000	1 KG	
800	0000000102	FERT	I	10000000	1 KG	
800	0000000103	FERT	I	10000000	1 KG	
800	0000000104	FERT	I	10000000	1 KG	
800	0000000105	FERT	I	10000000	1 KG	

MANDT	MATNO	MAT_TYP	INDSECT	MATKL	GROSWT	UOM
800	0000000100	FERT	I	10000000	1 KG	
800	0000000100	HAWA	I	10000000	1 KG	
800	0000000101	FERT	I	10000000	1 KG	
800	0000000102	FERT	I	10000000	1 KG	
800	0000000103	FERT	I	10000000	1 KG	
800	0000000104	FERT	I	10000000	1 KG	
800	0000000105	FERT	I	10000000	1 KG	

Database Table

In SAP, you can easily enable change history for database tables, including your own custom z tables.

Simply check the '**Log data changes**' checkbox in the technical settings.

Once this is done and activated, you can simply use transaction **SCU3** to view any changes made to the data in this table. Because logs must be created each time a change is made, there is obviously a slight performance hit involved, so you should probably only enable this for tables for which you want to record history.

Dictionary: Display Technical Settings

Revised<->Active

Name: MARA Transparent Table

Short Descript.: General Material Data

Last Changed: SAP 31.01.2023

Status: Actv. Saved

General Properties DB-Specific Properties

Logical Storage Parameters

Data Class: APPL0 Master Data, Transparent Tables

Size Category: 4 Expected Data Records 14.000 to 57.000

Buffering

☒ Buffering Not Allowed

☐ Buffering allowed but switched off

☐ Buffering Activated

Buffering Type

☐ Single records buff.

☐ Generic Area Buffered

☐ Fully Buffered

☐ Log Data Changes

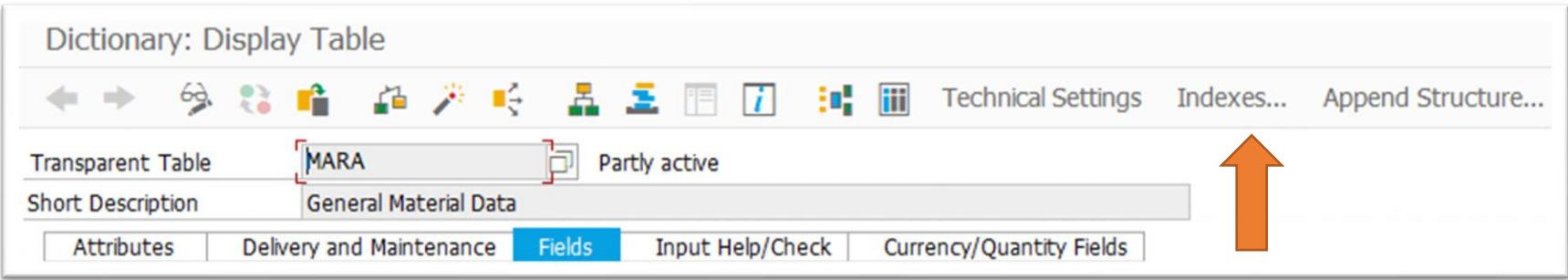
☐ Writes only with JAVA

You can specify that the contents of the table may only be changed from within Java by using the indicator "Write access with Java only."

If you make changes to the table content using ABAP programmers, there is no guarantee that the data will be read correctly from the Java side.

If the ABAP data is ASCII in nature, it can be read from the Java side.

Database Table



Indexes: Used to speed up the searching a table records that satisfy certain search criteria.

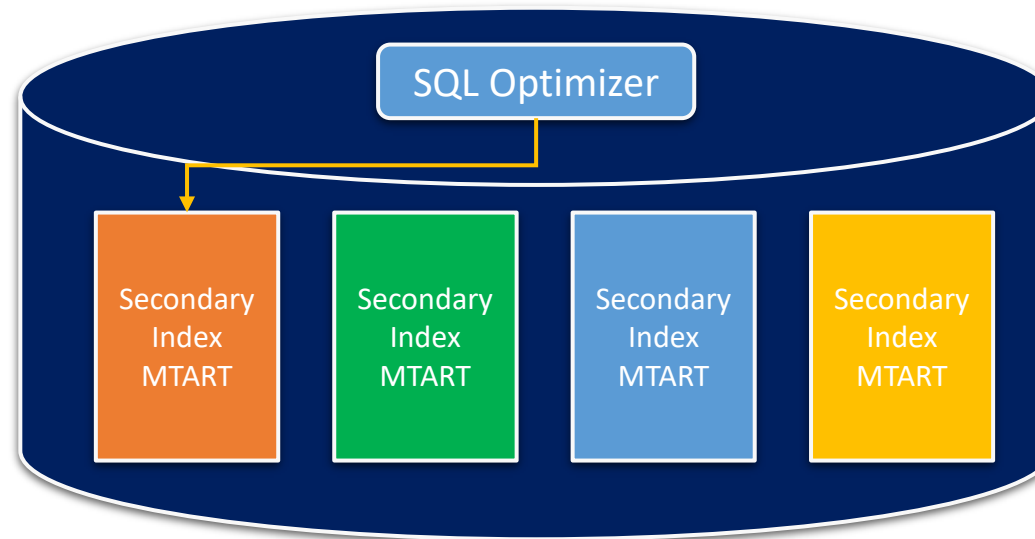
Primary Index: The Primary index create automatically based on the key fields when we activate the table and this will have pointer to the non-key fields of the table.

MATKL	Pointer	MANDT	MATNO	MAT_TYP	Pointer	MANDT	MATNO	MAT_TYP	INDSECT	MATKL
10000000	1	800	0000000100	FERT	1	800	0000000100	FERT	I	10000000
10000000	2	800	0000000100	HAWA	2	800	0000000100	HAWA	I	10000000
10000000	3	800	0000000101	FERT	3	800	0000000101	FERT	I	10000000
10000000	4	800	0000000102	FERT	4	800	0000000102	FERT	I	10000000
10000000	5	800	0000000103	FERT	5	800	0000000103	FERT	I	10000000
10000000	6	800	0000000104	FERT	6	800	0000000104	FERT	I	10000000
10000000	7	800	0000000105	FERT	7	800	0000000105	FERT	I	10000000

Database Table

Secondary Index: We can create Further index to speed up the search process based on Non-Key field called as secondary index.

- By creating index we are duplicating the table data. It consume More memory
- If we create more secondary index and most the index have similar columns then SQL Optimizer may end up choosing the wrong index or it may skip searching values in index table and end up scanning main table



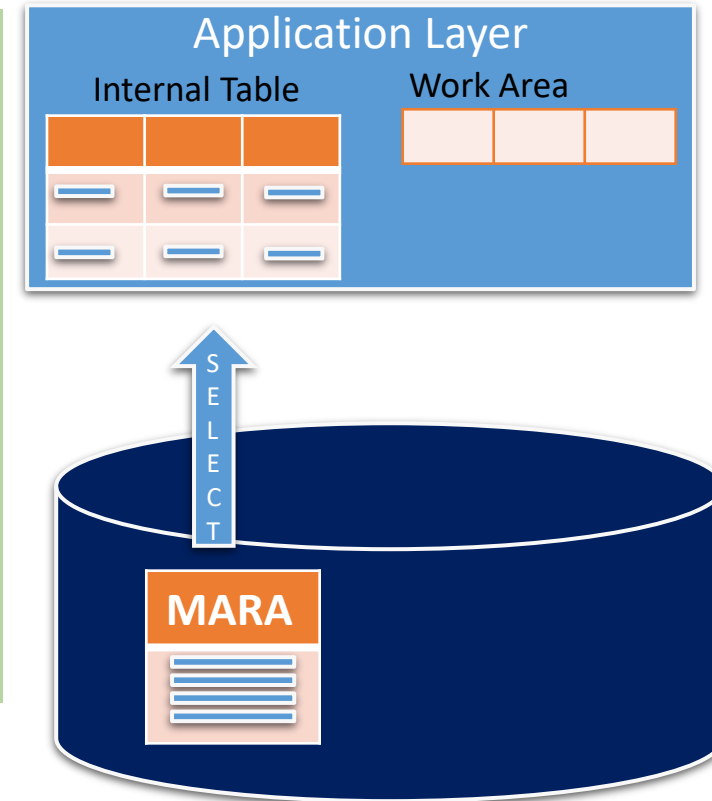
Internal Table & Work Area

Internal Table: It's runtime memory allocated to store **multiple records** in application Layer

Types of Internal Table

1. Standard Internal Table
2. Sorted Internal Table
3. Hashed Internal Table

Work Area: It's runtime memory allocated in application layer to **store single record**



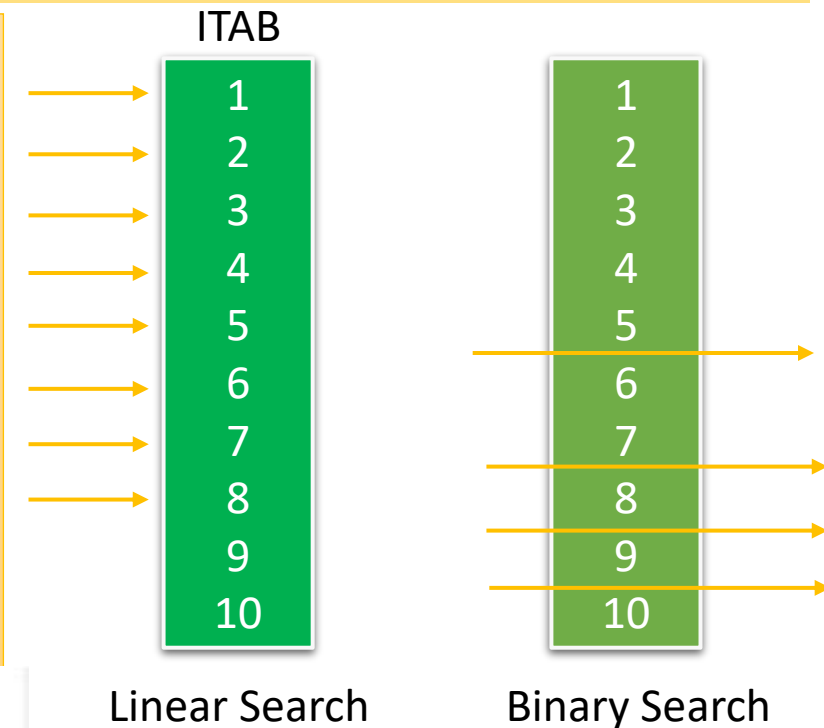
Linear Search & Binary Search

Linear Search: Linear search is sequential Search.

- It will start from top of the internal table and it will compare one by one with searching value if the value matches then it will bring that value to the WA otherwise it return with SY-SUBRC > 0

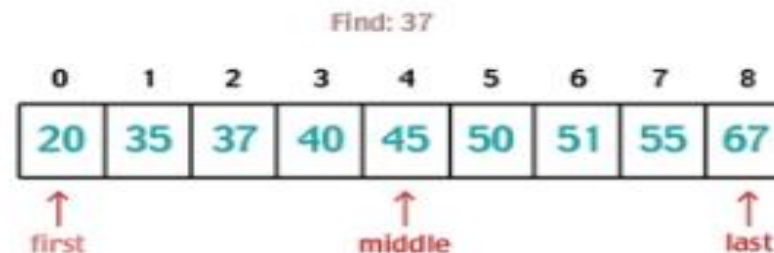
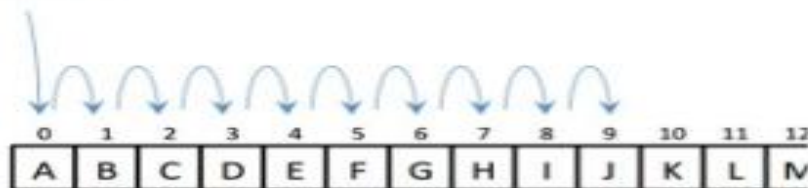
Binary Search: Search the internal table repeatedly by dividing the search intervals into half. If the value of the search key is greater than the middle value then ABAP runtime will ignore the First half and it will divide the second half into two half and it will compare with last value of first it will do same process until unless record found or Search is completed.

For Binary Search the Internal Table must be Sorted in Ascending Order.



Linear search Vs Binary Search

Find "J"



Types of Internal Table

Standard Internal Table:

- **Syntax:** DATA: <ITAB> TYPE STANDARD TABLE OF/ TYPE TABLE OF <STRUC>.
- It will have row index, We can perform Index Operation.
- We can also access records of an Internal table using KEY.
- Standard Internal table by default will have non-unique primary key, It's not mandatory to provide the key field information for standard Internal Table.
- By Default it's support Linear Search. If want to perform **BINARY SEARCH** you specify explicitly.
- The records of an Internal Table are not sorted by default in Ascending or Descending order but you can sort explicitly.

Sorted Internal Table:

- **Syntax:** DATA: <ITAB> TYPE SORTED TABLE OF <STRUC> WITH UNIQUE/NON-UNIQUE KEY <FLD1>.
- It will have row Index, You can perform Index Operation.
- We can also access records of an Internal table using KEY.
- While declaring Sorted Internal Table you must specify Unique/Non-Unique key field of an Internal Table other wise ABAP Editor will throw an error.
- By Default Records of an internal table based on the key field sorted in ascending order and you can't write any further SORT statement.
- When you access any record of an sorted internal table using key field it will do Binary Search.

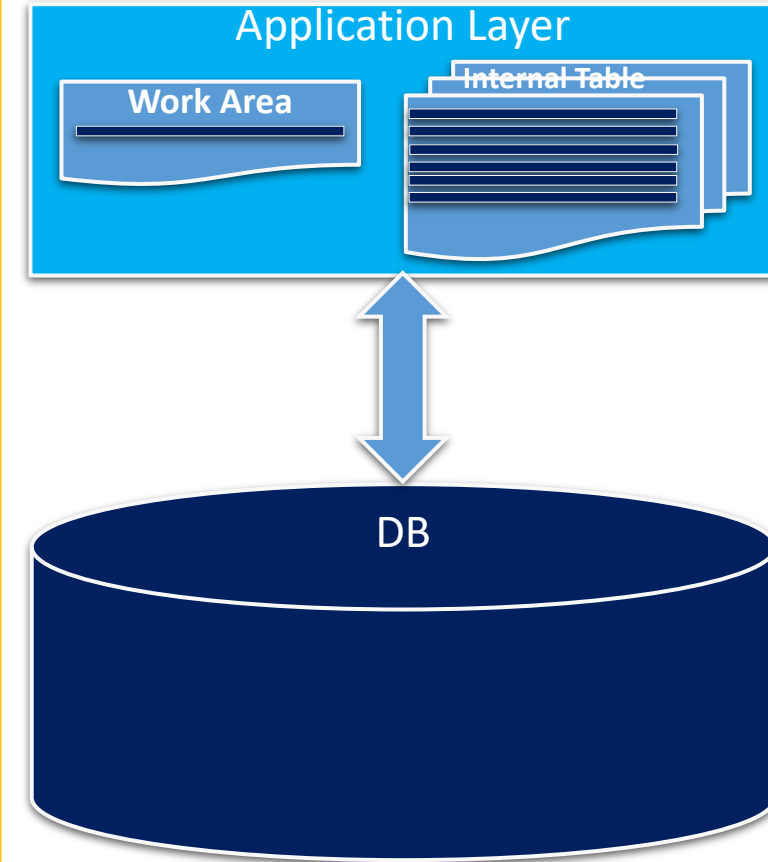
Types of Internal Table

Hashed Internal Table:

- **Syntax:** DATA: <ITAB> TYPE HASHSED TABLE OF <STRUC> WITH UNIQUE KEY <FLD1>.
- Hashed Internal Table will not have any row Index, That means you can't perform any Index Operation.
- You can access record of an Internal table using Key Field
- Hashed Internal Table only support Unique Key
- The record of the internal table will not be their in a sorted order, You can SORT this internal Table Data Explicitly.
- When you access any record of an Hashed Internal table It will use Hash algorithm because of this reason it will ABAP runtime will take same time if you access the first or the last records of an internal Table

Internal Table Operation

1. **APPEND:** Used to append the rows at the end of an Internal table
 - **APPEND LINES OF:** It's used to add multiple rows from one internal table to another internal table.
 - **APPEND INITIAL LINE:** Used to add an empty row with default value to Internal table.
2. **INSERT:** Used to insert record to an internal table at specified location/index
 - **INSERT LINES OF:** It's used to insert multiple lines from an internal to another internal table at the specified index position.
 - **INSERT INITIAL LINE:** Used insert an empty line to an internal table at specified index
3. **COLLECT:** Collect behaves bit differently from APPEND Statement. When you try to add record to the internal table using COLLECT Statement, ABAP Runtime will check if the row already exists in an Internal Table with same key field value. If row already exist in an Internal Table then ABAP Runtime will add numeric column values to the existing row rather inserting a new row. **If the row doesn't exist the it add a row at the end of Internal Table.**
4. **DELETE:** Used to delete any record from an Internal Table



Interview Question?

What is difference between APPEND and INSERT?
What is Difference between APPEND and COLLECT?

Internal Table Operation

DELETE ADJUCENET DUPLICATE: Used for deleting adjacent duplicate records from an Internal Table. **Sorting Internal In Ascending or Descending order is Mandatory**

MODIFY: Used to change the single or multiple rows and column value of an internal table.

TRANSPORTING Used to specify the list of columns data to be modified.

SORT: Used to sort he rows of an Internal Table in Ascending or Descending Order

MOVE: Used to copy values from one data object to another data objects.

MOVE-CORRESPONDING: Used to move data from one data object to another when the structure is not same.

DESCRIBE TABLE: Used to count number of record in an Internal Table

CLEAR: Used clear the content of an Work Area

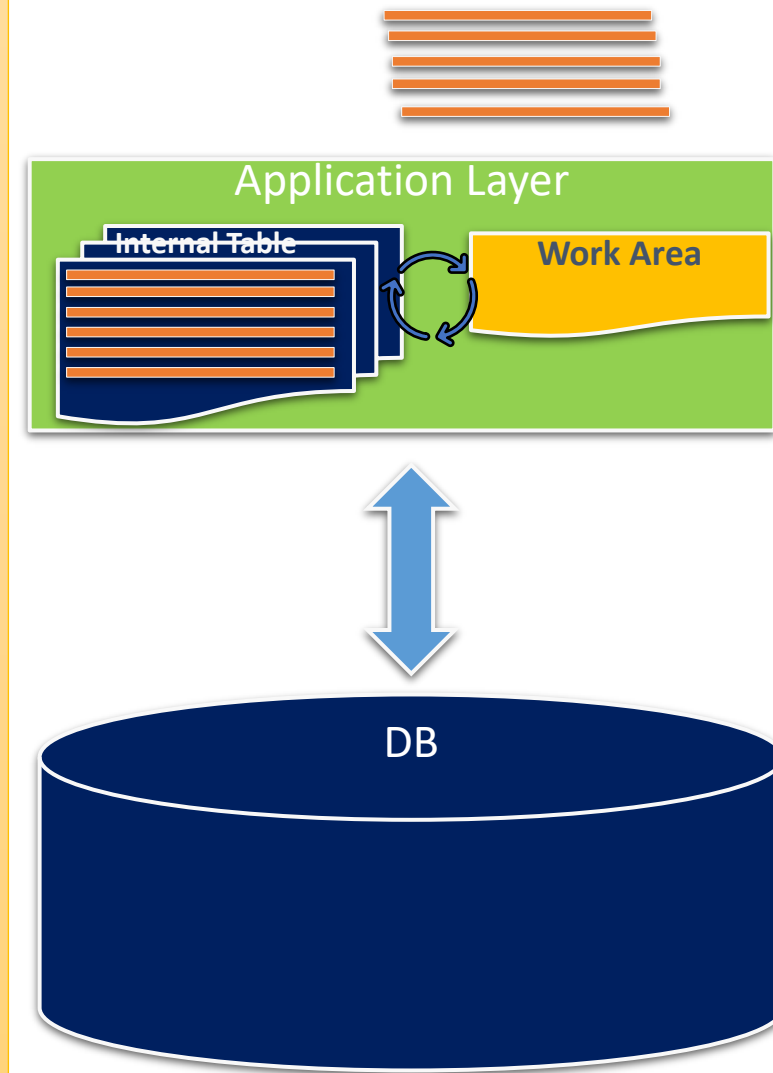
REFRESH: Used to clear the content of an Internal Table

FREE: Used to **Removed allocated memory** of an Internal Table.

READ TABLE: Used to copy single record from an Internal Table to work area. **If there are multiple rows matching the same key value then it will return the first matching rows.**

TRANSPORTING: Used to specify the which fields data should be copied form Internal Table to work Area.

TRANSPORTING NO FIELDS: If you use this keyword no column data will be copied.



Internal Table Operation

LOOP AT: Used to process multiple rows of an internal table. System will loop between LOOP and ENDLOOP by reading one row at a time to work area. It keeps reading the data from an internal table to work area until all matching rows or all records of an internal table are processed.

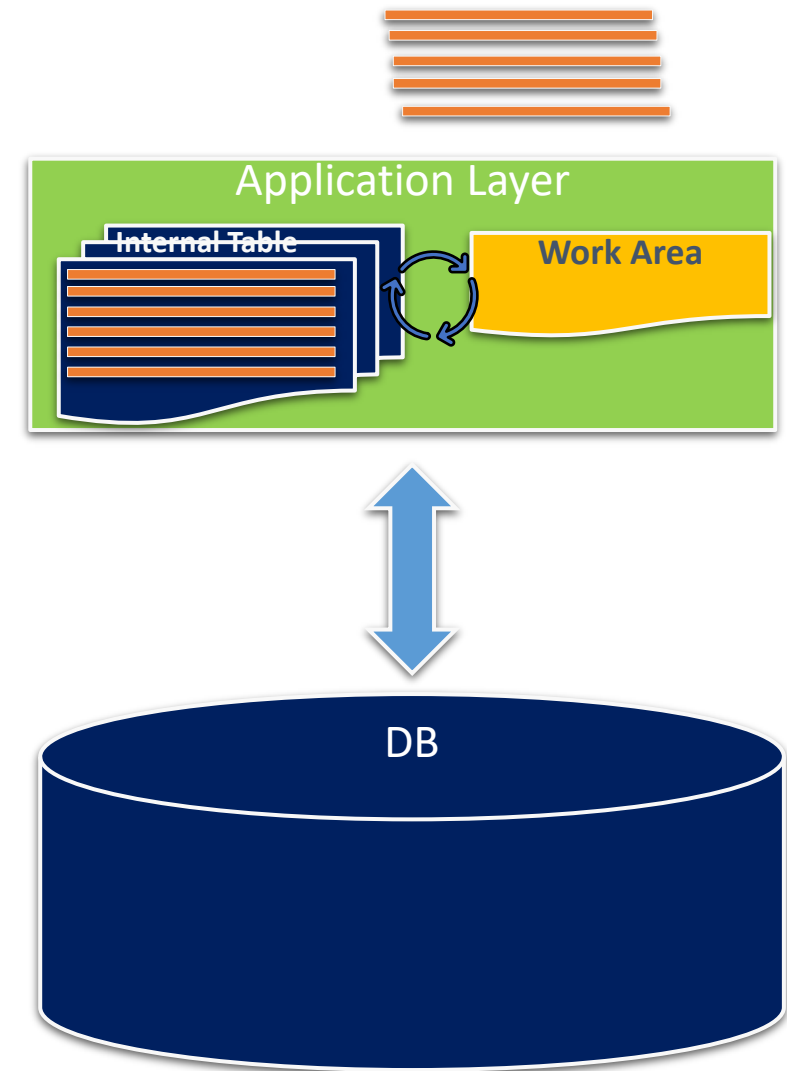
DO: **It's is an unconditional Loop.** **TIMES:** Used to specify how many times the loop should run. If we don't specify and if we don't terminate loop then it will go for infinite loop.

WHILE...ENDWHILE: Conditional Loop, Loop statement will continue execution only if condition is true. This loop also can run in infinite mode.

EXIT: When exit keyword is used inside a loop it will come out of loop. When Exit statement is used inside a procedure then it will terminate only that procedure execution.

When Exit is used in a program directly it will terminate the Program execution.

CONTINUE: It can only be used inside a loop. When this key is used then the system will skip the current loop pass and it will go to next loop pass.



SQL Joins

SQL Joins are used to join two or more tables based on common field between them.

1. **Inner Join:** Always return the matching row from both tables.
2. **Outer Join:** Always return all the records from one table irrespective of other table entry.
 - **Left Outer Join:** Always return all the entries from left table and matching rows from right Table.
 - **Right Outer Join:** Always return all the entries from right table and matching rows from left table.

Inner Join				Left Join				Right Join			
MATNR	MTART	MATNR	MAKTX	MATNR	MTART	MATNR	MAKTX	MATNR	MTART	MATNR	MAKTX
1000	FERT	1000	Laptop	1000	FERT	1000	Laptop	1000	FERT	1000	Laptop
2000	HAWA	1000	Laptop	2000	HAWA	1000	Laptop	2000	HAWA	1000	Laptop
4000	HALB	2000	Mobile	4000	HALB	2000	Mobile	4000	HALB	2000	Mobile
		3000	Mouse			3000	Mouse			3000	Mouse

MATNR	MTART	MAKTX
1000	FERT	Laptop
1000	FERT	Laptop
2000	HAWA	Mobile

MATNR	MTART	MAKTX
1000	FERT	Laptop
1000	FERT	Laptop
2000	FERT	Mobile
4000	HALB	

MATNR	MTART	MAKTX
1000	FERT	Laptop
1000	FERT	Laptop
2000	FERT	Mobile
3000		Mouse

Views(In SE11)

Story:**Have you ever taken a look out your window?**

Assume a beautiful girl and her parents leave right in front of your house. Assume you have a large window in your room from which you can see both the girl's and her parents' windows. If you only want to see the girl, use curtains to hide one side of the window; if you want to see her parents, move the curtains again. It's called Views in ABAP.

Views in ABAP helps us to see what we want to see from one table or many.



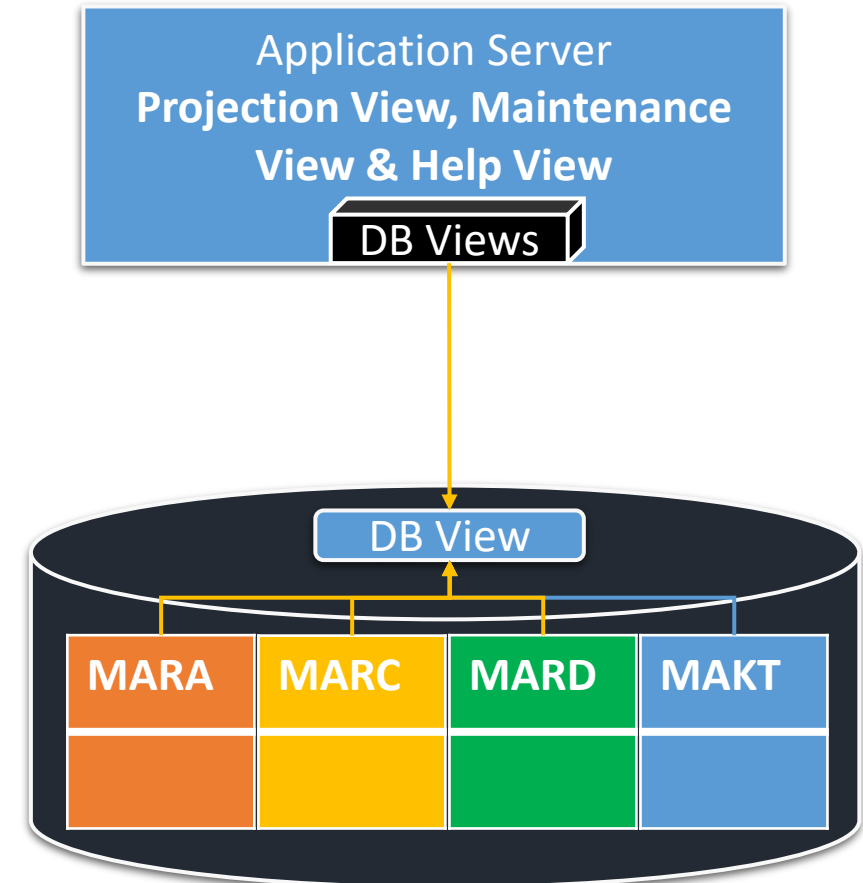
Views(In SE11)

Views: Views are used to join the data which segregated across multiple Table.

Views are also called as virtual Table. They will not store any data in Database they will bring the data which is stored in the Database.

❖ Inner Join

1. **Database View:** When we activate the database view it generates native SQL statement to create view in the Database. Since this view will created in the DB we can also use this in SELECT statements. You can only read database views with more than one table
- ❖ **No SQL Joins**
2. **Projection View:** are created in single table, Projection views are used hide the unwanted field of table. Using Projection view you can also modify the data of an table.
- ❖ **Left Outer Join**
3. **Maintenance View:** Are mainly used to maintenance the data of tables. For maintenance view TMG is mandatory
4. **Help Views:** Used in search help as a selection methods. You can't Display, Modify, delete the data of tables



Views(In SE11)

Process to create Views in SE11: [CLICK HERE](#)

Assignment

1. Create DB view on top of VBUK, VBAP, VBAK
2. Create Projection Views for MARA Table
3. Create Maintenance View for EKKO, EKPO
4. Create Hel View with MARA and MAKT

Interview Question?

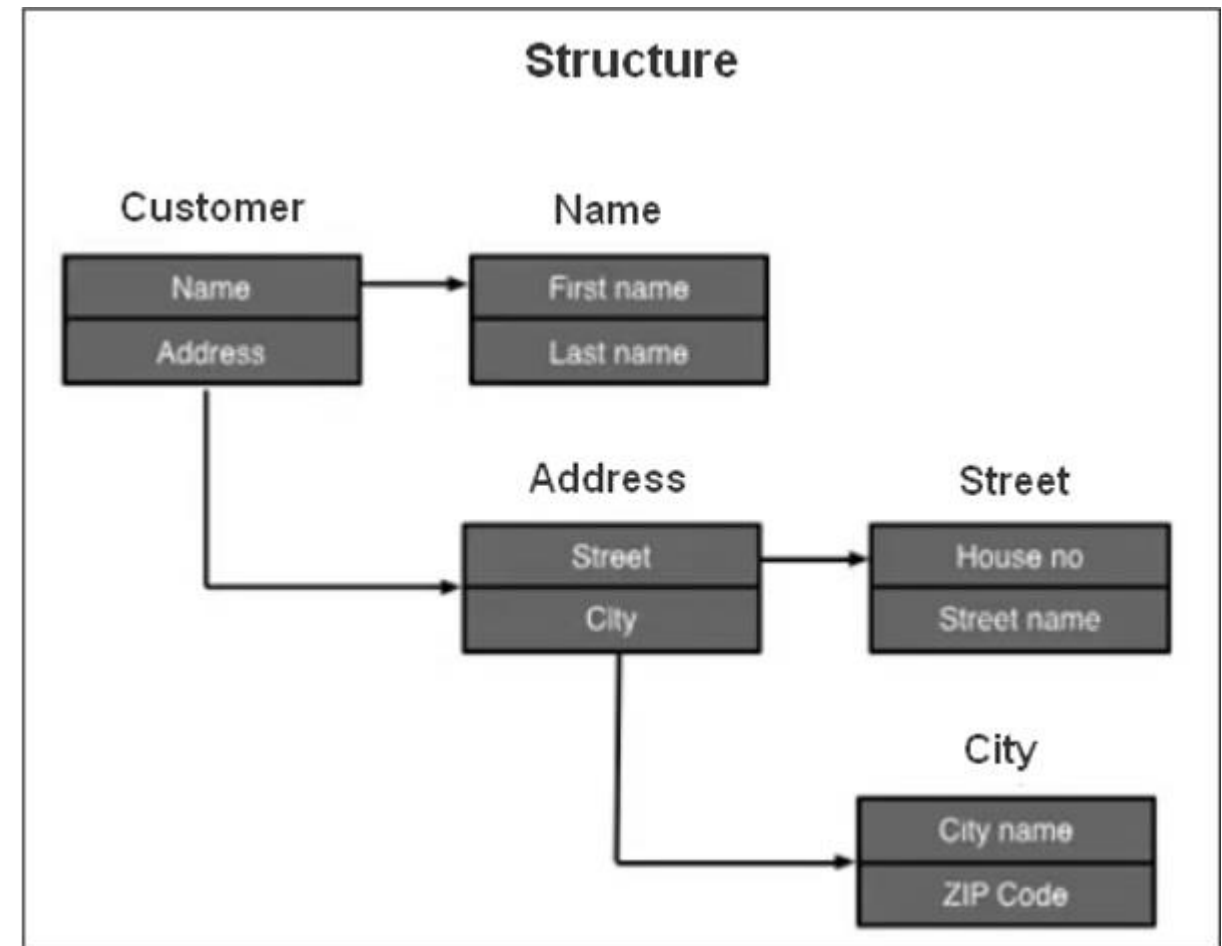
What is different types of View?

What is difference between the View?

Structure(in SE11)

A **structure** is a data type that consists of multiple components, each of which can be of a different data type. These components are stored in contiguous memory locations. This means that when you define a structure, the components are stored one after the other in the memory, making it easier to access them.

For example, you can define a structure that contains a person's name, age, and address, with each of these components having a different data type, such as character, integer, and string. By storing these components in a structure, you can easily access all of the information about the person in one place.



Structure(in SE11)

Structure is templet, It will not store any data in the database. Structure used to define the data objects and used In Parameter Interface for Function Modules, Smartforms, Methods and Subroutines.

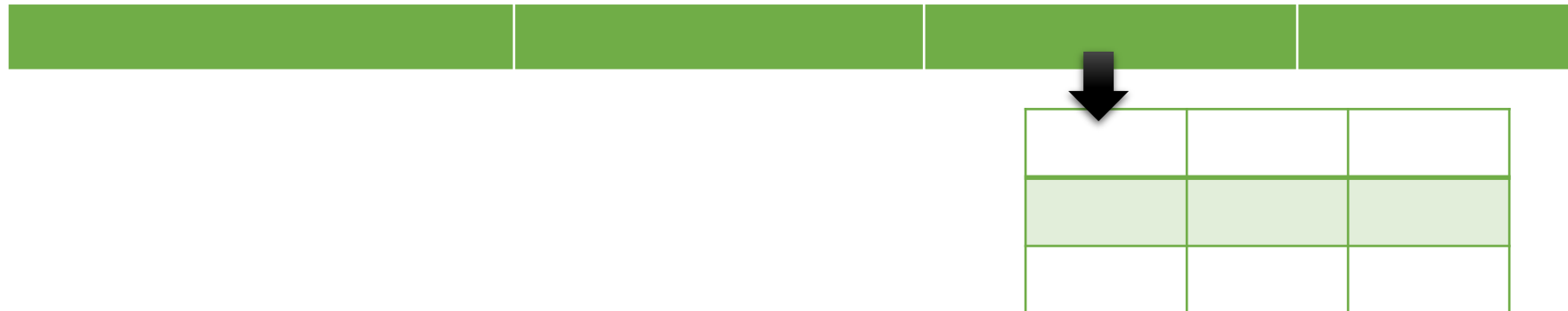
1. **Flat Structure**: This Structure use the data type reference as data element or Pre-defined Data Type.



2. **Nested Structure**: Nested structure use the data type reference as another structure.



3. **Deep Structure**: Deep structure will have data type reference a table type.



Structure(in SE11)

ABAP Dictionary: Initial Screen

1

☐ Database table

☐ View

☒ Data type

☐ Type Group

☐ Domain

☐ Search help

☐ Lock object

ZSTR_00_SELLFORE_RPT

Display

Change

Create

Create Type ZSTR_00_SELLFORE_RPT

☒ Data element

☐ Structure

☐ Table type

Dictionary: Change Structure

3

Structure

ZSTR_00_SELLFORE_RPT

New(Revised)

Short Description

Attributes

Components

Input Help/Check

Currency/quantity fields

Predefined Type

1 / 0

Component	Typing Method	Component Type	Data Type	Length	Decim...	Short Description

Structure(in SE11)

Here are some interview questions on Structure in SE11 ABAP:

- 1.What is a structure in SE11 ABAP?
- 2.What are the components of a structure?
- 3.How is a structure different from a table in ABAP?
- 4.How can you create a structure in SE11?
- 5.What are the different data types that can be used to define the components of a structure?
- 6.Can a structure have a table type component?
- 7.How can you include a structure as a component of another structure?
- 8.How can you check the fields of a structure in SE11?
- 9.What is the difference between a transparent table and a structure?
- 10.How can you use a structure in a program to store data?

Table Type(in SE11)

In SAP ABAP, a **Table Type** is a globally reusable object that defines the structure and functional features of an

internal table in ABAP.

A table type is defined as follows:

Line Format: It establishes the structure and data type.

Access Mode: the type of internal table, for example. Sorted, hashed, and indexed

Keys: The table's primary and secondary keys

Reference type creates a pointer to data object

Dictionary: Change Table Type

Table Type: ZTT_00_SELLFORE_RPT New(Revised)

Short text: Table Type for Sell forecasting

Attributes Line Type Initialization and Access Primary Key Secondary Key

☒ Line Type **ZSTR_00_SELLFORE_RPT**

☐ Predefined Type

Data Type: []

No. of Characters: 0 Decimal Places: 0

☐ Reference type

☐ Referenced Type []

☐ Reference to Predefined Type

Data Type: []

Length: 0 Decimal Places: 0

Table Type will create a structure similar to this line type.

Length Type can be any predefined data types with any length of field and decimal places. It is similar to the creation of Data Elements.

Table Type(in SE11)

Dictionary: Change Table Type

Table Type: ZTT_00_SELLFORE_RPT New(Revised)

Short text: Table Type For Fore Casting

Attributes | Line Type | Initialization and Access | Primary Key | Secondary Key

Initial Number of Rows:

Access

- ☒ Standard Table
- ☐ Sorted Table
- ☐ Hashed Table
- ☐ Index Table
- ☐ Not Specified

It is sorted internally on basis of key. It uses binary search to access key.

It means table can be any among Standard, Sorted or Hashed. The access of Index is not allowed here.

It uses linear search to access key.

It is sorted internally on basis of hashed key. The hash key is generated for each key and this key is used to access data irrespective of index. Hashed keys are generated using Hash Algorithm internally.

In this case the table can be Standard or Sorted. It allows Index access for such table. Index tables are used to define generic parameters of a Subroutine Form & Function Module.

Just Like **OCCURS** statement in ABAP **Internal Table**, here too we can specify the number of Initial lines occupied by the internal table.

Table Type(in SE11)

Dictionary: Change Table Type

Table Type: ZTT_00_SELLFORE_RPT New(Revised)

Short text: Table Type

Attributes | Line Type | Initialization and Access | **Primary Key** | Secondary Key

Key definition	Key category
☒ Standard Key	☐ Unique
☐ Line Type	☒ Non-Unique
☐ Key Components	☐ Not specified
☐ Not specified	

Choose components

Key component

Alias

It defines the key of Table Type on basis of Line type. The Key contains all the character fields of the table line in case of structure, table or view and It is the entire table line in case of Data Element or Reference type.

We can select Primary key from the structure specified by us in Line Type tab earlier. In this case, we need to select the Primary keys

It defines the entire fields of line type as Primary key.

In this case key is Generic. And the table type is called Generic Table Types.

Table Type(in SE11)

Dictionary: Change Table Type

Table Type
Short text

Attributes Line Type Initialization and Key Secondary Key

Key definition

- ☒ Standard Key
- ☐ Line Type
- ☐ Key Components
- ☐ Not specified

Key category

- ☐ Unique
- ☒ Non-Unique
- ☐ Not specified

Choose components

Alias

It tells whether our table type will make an internal table that can store only Unique keys or can have duplicates also.

It means internal table can have only unique records.

It means internal table can have duplicate records too.

It can be any Unique or Non-Unique. It defines Generic Table Types.

Table Type(in SE11)

Combination of Access Mode and Key category that is allowed:

Access Mode	Key category	Description
Not Specified	Not specified	It means that if the access mode is the one as mention here, than the key categories must be chosen from this table only.
Index Table	Not specified	
Standard Table	Non-Unique	
Sorted Table	Unique, Non-Unique or Not specified	
Hashed Table	Unique	

Here are some interview questions on table types in SE11 in SAP ABAP:

- 1.What is a table type in SAP ABAP?
- 2.How is a table type defined?
- 3.What is a line type in a table type?
- 4.What are the different access modes available for internal tables?
- 5.How do you create a new table type?
- 6.How do you assign a table type to an internal table?
- 7.What is the difference between a table type and a structure in SAP ABAP?
- 8.Can you have multiple line types in a table type?
- 9.What are primary and secondary keys in a table type?
- 10.How do you define a sorted table type?

Search help(in SE11)

Search help provides input help to screen field and Table Field.

1. **Elementary Search Help:** Elementary Search help Provides single Search Path
2. **Collective Search Help:** Collective search help provide multiple search path. It's a collection of elementary and Collective Search Helps or Collective search help is collection of two or more search help.

Hot Key: Used to search Elementary search help in collective search help.

Search Help Exit: It's used to enhance the standard futures of Search help, **For Ex:** Delete Adjacent Duplicate Records, Sor the Records in Ascending or Descending Order, Filter Records etc.

Ex: Delete Duplicate Records, Sor The Search Help

FM: F4IF_SHLP_EXIT_EXAMPLE

Restriction Window

Hit List

The screenshot displays the SAP Search Help interface. At the top, a header bar shows 'Restrict Value Range (2)' and '25 Entries found'. Below this, a 'Restrictions' section contains a 'Material:' field with a search icon and the value '10*'. To the right of the field are three buttons: a green checkmark, a blue double arrow, and a blue 'X' in a square. Below the restrictions section is a 'Hit List' table. The table has a header row with the title 'Material' and a list of results: '10' and '1000000000'. At the bottom of the table, it says '25 Entries found'. A toolbar with various icons (checkmark, close, search, magnifying glass, star, question mark, print, dropdown, and refresh) is located above the table.

Search help(in SE11)

ABAP Dictionary: Initial Screen

☐ Database table

☐ View

☐ Data type

☐ Type Group

☐ Domain

☒ Search help

☐ Lock object

Choose the search help

Create Search Help ZSRCH_00_EMPL

☒ Elementary search help

☐ Collective search help

Search help(in SE11)

The possible entries for a field displayed in the hit list are determined at runtime by selection from the database. The selection method describes the database object from which the data is read. A database table or view can be defined as selection method.

Dictionary: Change Search Help

Elementary Help: ZSRCH_00_EMPL New(Revised)

Short description:

Attributes Definition

Data Collection

Selection method:

Text table:

Dialog Behavior

Dialog type: Display values immediately

Hot key:

Enhanced Options

☐ Proposal Search for Input Fields Autosuggestion for input field.

☐ Multi-Column Full Text Search (Database-Specific)

Accuracy Value for Error-Tolerant Full Text Search: 0,8

Search help exit:

FUZZY Search %

Parameter

Search help parameter	IMP	EXP	LPos	SPos	SDis	Data element	M...	Default value
	☐	☐			☐		☐	
	☐	☐			☐		☐	

The hot key permits the user to select an elementary search help from the collective search help directly in the input field with the short notation. The restrictions can also be entered directly in the dialog box for restricting values. Letters and digits are allowed as hot key.

Search help(in SE11)

This is a checkbox that indicates whether or not a Search help parameter is an import parameter. The export or import is with a reference to the search help.

This field is a checkbox for indicating whether a Search help parameter is an export parameter. The export will be transfer of field values from the selection list to screen fields.

This value determines where the Search help parameter or field appears in the selection list. If you enter the value 1, the field will be at the top of the selection list, and so on.

It determines the physical location of the Search Help parameter or field within the restricted dialogue box. If you enter a value of 1, the field will appear at the top of the restricted dialogue box, and so on.

If this flag is set, the search help parameter appears in the selection popup window for restricting values as a pure display field.

By default, every Search Help parameter or field is allocated a data element that was assigned to it in the data source (Table or View). In display mode, this data element's name appears.

Dictionary: Change Search Help

Attributes Definition

Data Collection

Selection method

Text table

Enhanced Options

☐ Proposal Search for Input Fields

☐ Multi-Column Full Text Search (Database-Specific)

Accuracy Value for Error-Tolerant Full Text Search

Search help exit

Parameter

Search help parameter	IMP	EXP	LPos	SPos	SDis	Data element
	☐	☐			☐	
	☐	☐			☐	

Search help(in SE11)

The dialogue type of an elementary search help is used to specify how the hit list is shown when the input help is invoked.

For an elementary search help, you can select one of the following dialogue types:

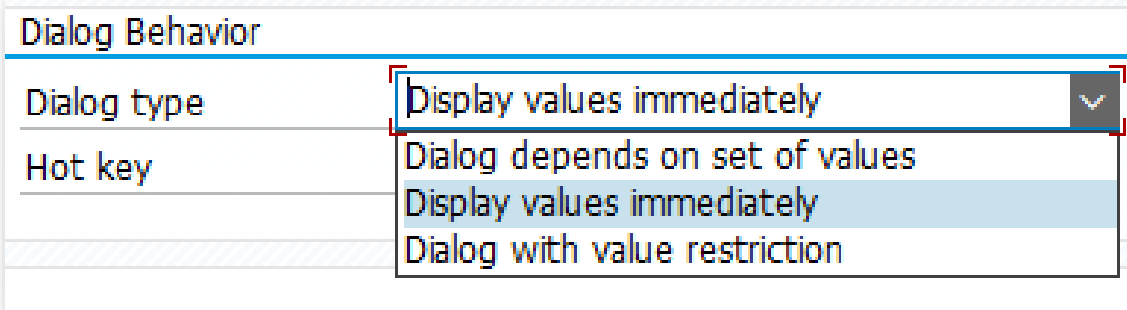
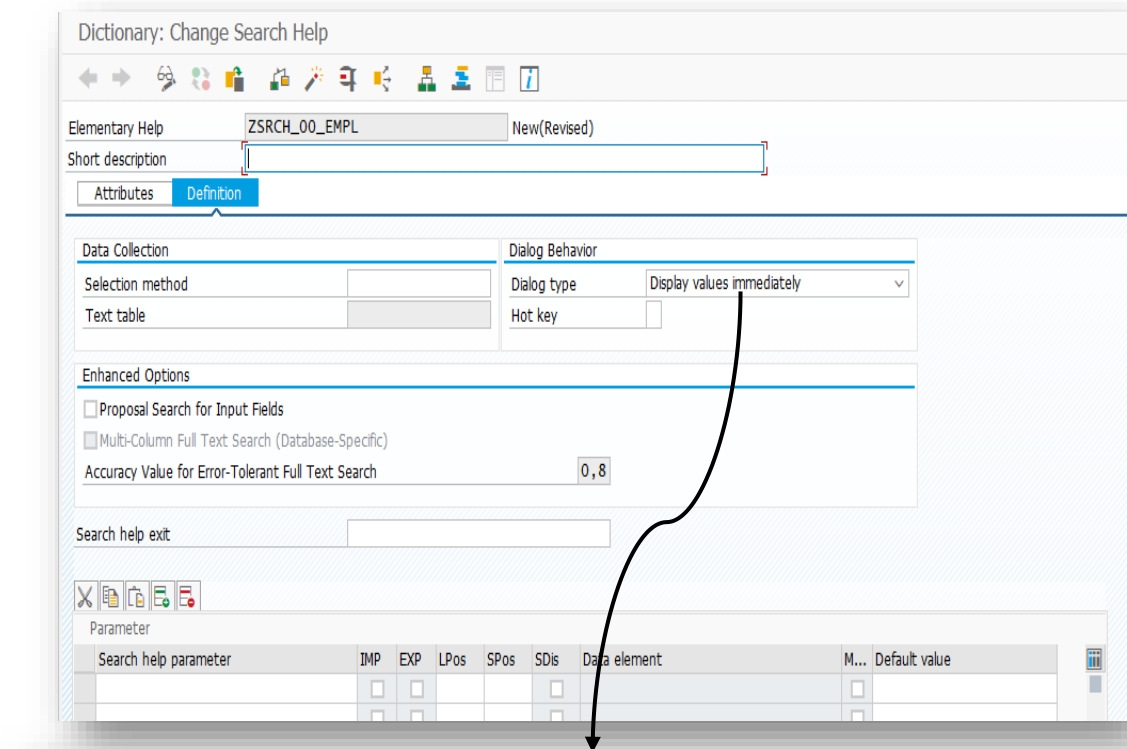
Display values immediately:

When you call input help, the hit list is immediately presented. This makes sense only if the hit list has a few entries.

Dialog with value restriction: Immediately available is the dialogue window for limiting values. If the list of possible entries is lengthy, select this option. The hit list becomes more understandable and the system burden during value selection is lowered when the user limits the amount of data to be processed.

Dialog depends on set of values:

If the hit list has fewer than 100 elements, it is displayed instantly. If the hit list comprises more than 100 entries, the system shows the value restriction dialogue box.



Search help(in SE11)**Interview Question?**

1. What is Search Help Exit?
2. What is Difference between Elementary & Collective Search Help?
3. What is Selection Methods?
4. What is Hot Key?
5. What is the use of matchcode object in SAP ABAP?
6. How do you assign a search help to a selection screen field in SAP ABAP?
7. What is the use of search help exit in SAP ABAP?
8. How do you maintain search help in SE11?
9. How do you create a search help based on a database view in SAP ABAP?
10. What is the difference between basic search help and enhanced search help in SAP ABAP?
11. How do you activate and test a search help in SAP ABAP?

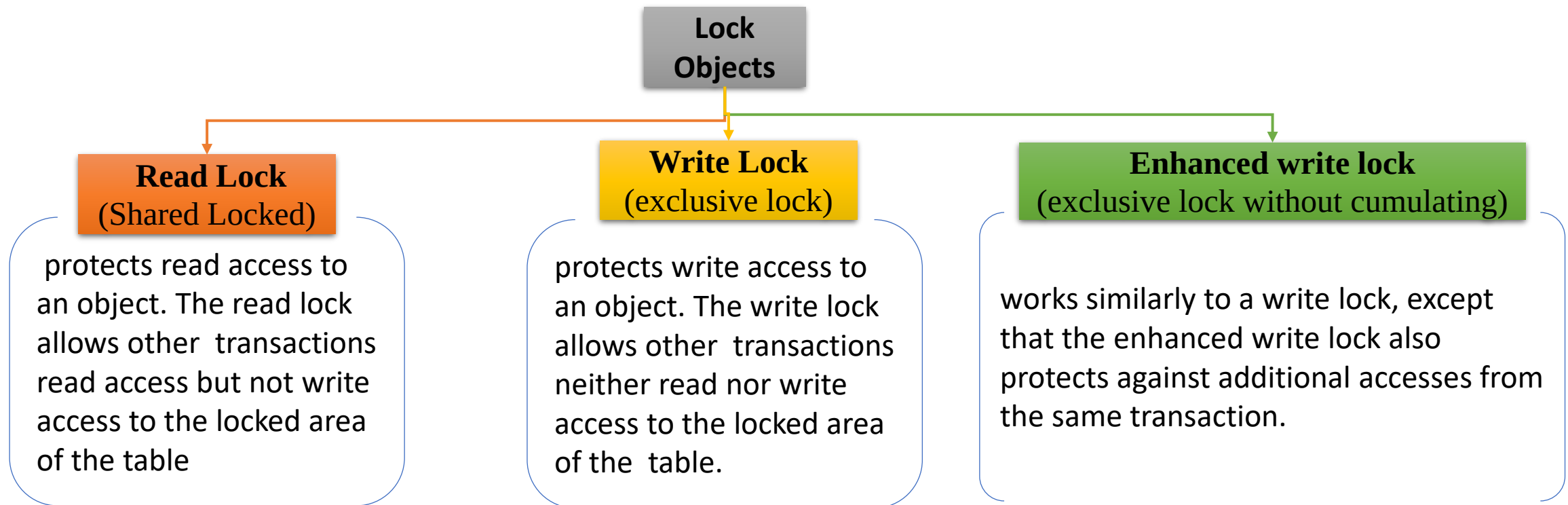


Lock Objects(in SE11)

What is Lock Objects?

Lock objects are used in SAP to prevent inconsistencies when inserting or changing data in the database.

How many types of lock objects?



Lock Objects(in SE11)

How to create Lock objects?

With transaction SE11, you can build a lock on a SAP object with any sensible name beginning with **EZ**, for example EZTEST_LOCK.

ABAP Dictionary: Initial Screen

Database table
View
Data type
Type Group
Domain
Search help
☒ Lock object

EZTEST_LOCK

Display Change Create

Dictionary: Change Lock Object

Lock object EZTEST_LOCK New(Revised)
Short Description Test Lock

Attributes Tables Lock parameter

Primary Table

Name ☒
Lock Mode

- Write Lock
- Shared Lock
- Exclusive, not cumulative
- Set Optimistic Lock
- Promote optimistic lock; transform from 'O' to 'E'
- Only conflict check extended exclusive lock, as with 'X'
- Only conflict check exclusive lock, as with 'E'
- Conflict check for shared lock only, as in 'S'
- Only promotion check optimized lock, as with 'R'
- Reserved
- Reserved

Second Name

Lock Objects(in SE11)

Use:

You can notice this in almost all transactions when you open an object in Modify mode. SAP could not enable another user to open the same object in change mode.

In HR, for example, when we enter a personal number in the master data maintenance page, SAP cannot enable any other user to use the same personal number for updates.

Technically:

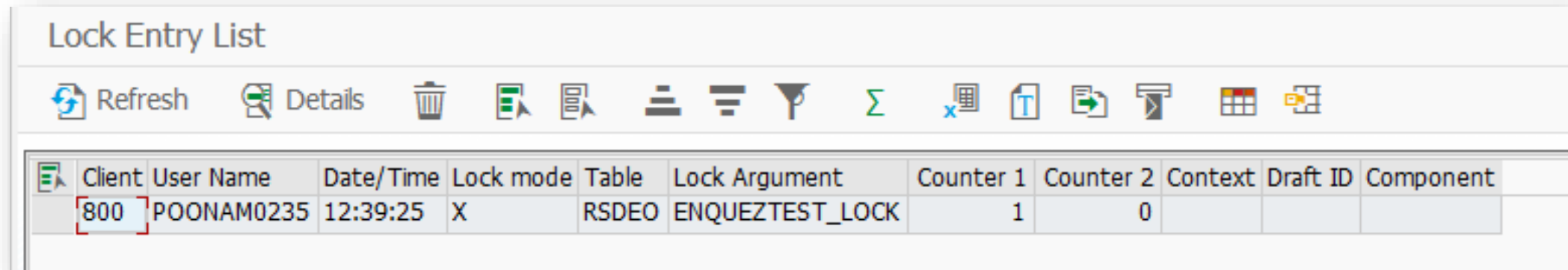
When a lock object is created, the system automatically creates two function modules.

1. **ENQUEUE_<Lockobject name>** to add the object to a queue.
2. **DEQUEUE_< Lockobject name>** to remove the object from the queue.

These function modules must be included in your program.



Lock Objects(in SE11)



Client	User Name	Date/Time	Lock mode	Table	Lock Argument	Counter 1	Counter 2	Context	Draft ID	Component
800	POONAM0235	12:39:25	X	RSDEO	ENQUEZTEST_LOCK	1	0			

For instance:

In order to change a sales order, the user goes to VA02, where a lock has already been set. Such locks can be seen in transaction **SM12**. If a lock is set and another user attempts to access the same sales order, they will see a notice indicating that the sales order is locked. The **ENQUEUE** function module is used to set the lock, whereas the **DEQUEUE** function module is used to unlock it.

Here are some interview questions on lock object in SAP ABAP:

- 1.How do you implement lock objects in SAP ABAP programs?
- 2.How do you release a lock in SAP ABAP?
- 3.How do you check if a lock exists on a particular record in SAP ABAP?
- 4.What is the significance of the "check only" option in the ENQUEUE statement?

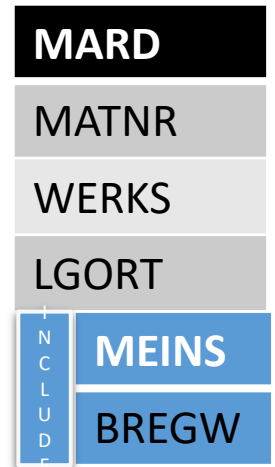
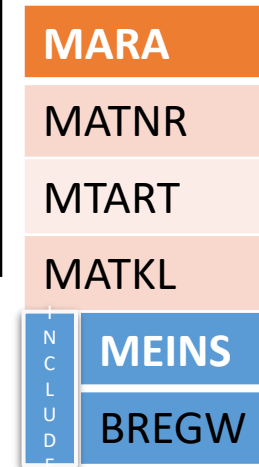
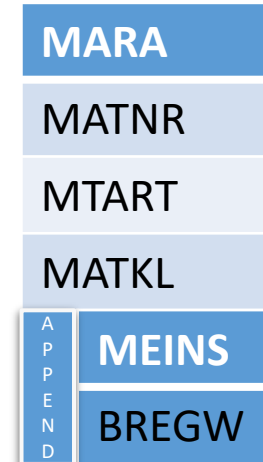
Append & Include Structure

Append Structure:

- Append structure used to enhance the standard table structure or Structure.
- Append Structure can't be reused it's exactly assigned to one table
- It will always add the fields the end of Table
- This will start with .APPEND

Include Structure:

- It's mainly used reusing the same column in multiple table
- We can create an structure with reusable fields and use them in Multiple Tables or Structures
- Using Include you can insert structure at any place
- This will start with .INCLUDE



Interview Question?

Difference between Include and Append?

Modularization

Breaking down and organizing program into modular structure

Advantages of Modularization:

- ✓ Improves the readability of the code.
- ✓ Helps to develop reusable software blocks.
- ✓ Save the development effort.
- ✓ Helps to design applications with future scope for enhancement.
- ✓ Helps to break down the complex logic/Requirement into smaller and easy maintenance software blocks. Later we can combine this block of code to get the final result.
- ✓ Splits the development effort across the team.
- ✓ Easy to find the bug and fix it.



Modularization

Modularization in SAP ABAP

Include Programs

Subroutines

Events Block

Dialogue
Modules

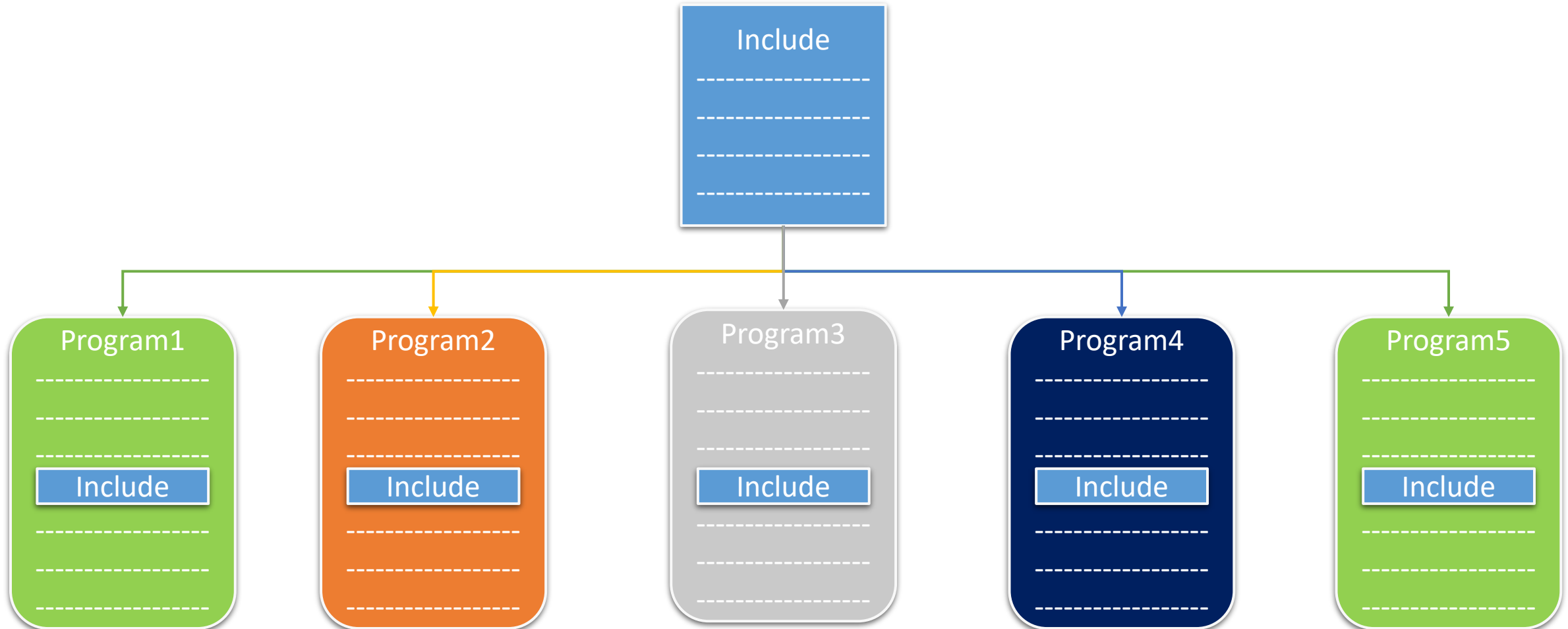
Functional
Modules

Class's
Methods

Macros

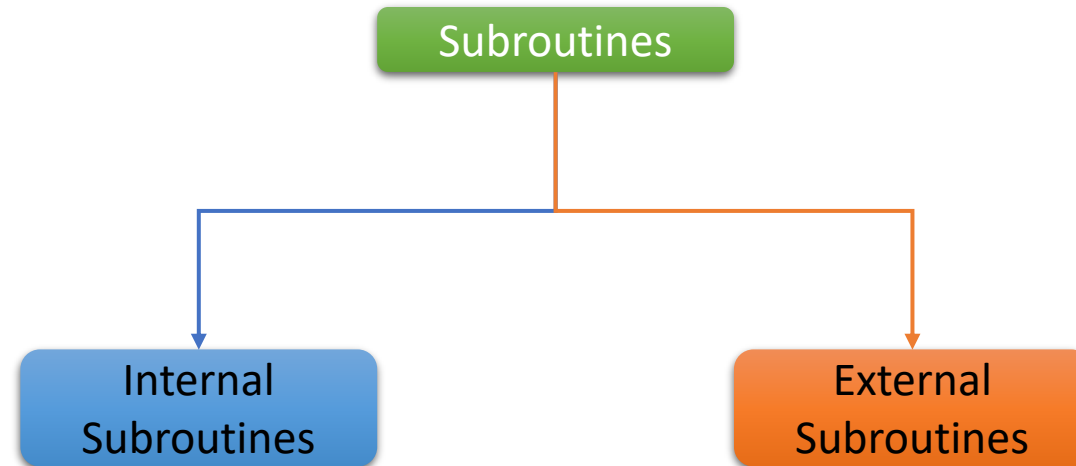
Include Programs

Include Programs are non-executable programs, But this program can be called from another program. This helps us to develop reusable code and also modularize.



Subroutines

Subroutines helps to develop reusable block of code within the program. Subroutines blocks are enclosed with FORM...ENDFORM. And this block of code can be called using PERFORM statement. After ENFORM you can't have any executable code you can have another FORM...ENDFORM.



Subroutines

Internal Subroutines

Internal Subroutine defined in the same programme that is being called.
All data items declared in the main ABAP/4 programme can be accessed.

Calling Part: `PERFORM <subroutine> [<pass>].`

Subroutines

External Subroutines

External Subroutine is one that is defined outside of the programme being called.
It is necessary to utilize the <pass> option or define data objects in shared memory.

Calling Part: `PERFORM <subroutine> (<Program>) [<pass>].`

`PERFORM <subroutine> (<Program>) [<pass>] [IF FOUND].`

`PERFORM (<subroutine>) IN PROGRAM (<Program>) [<pass>] [IF FOUND].`

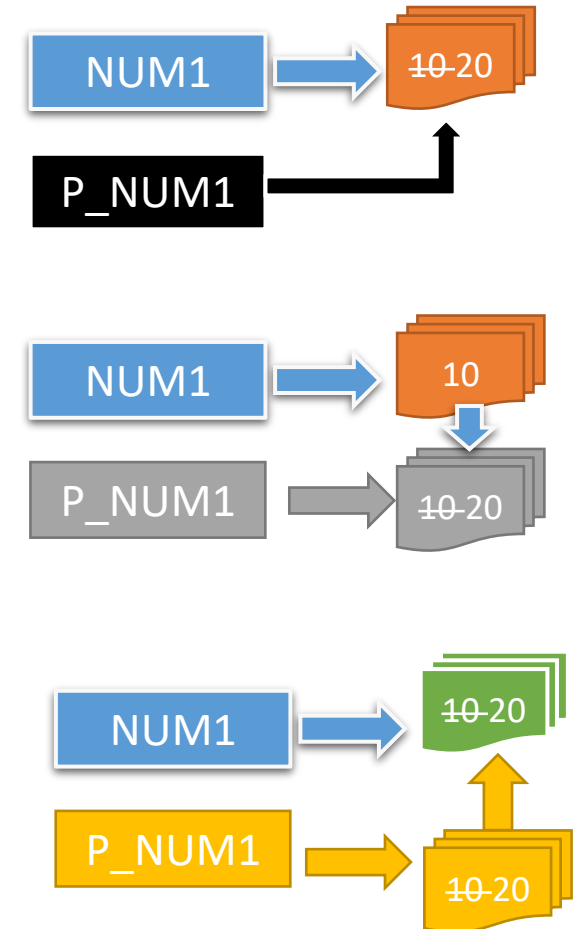
`PERFORM <index> OF <subroutine1> <subroutine2> <subroutine3> [<pass>].`

Parameter Interface

1. **Pass By Reference:** The formal parameter has no memory of its own. When calling all the subroutines, the memory address of the actual parameter value is passed to the formal parameter. If the value of the formal parameter changes it will also change the value of the actual Parameter.
2. **Pass By Value:** The formal parameter & Actual Parameter will have their memory. When you call the subroutine, the actual parameter value is passed to the formal parameter. If the value of the formal parameter changes it does not have any effect on the actual Parameter value.
3. **Pass By Value and Return:** The formal parameter & Actual Parameter will have their own memory. When you call the subroutine, the actual parameter value is passed to the formal parameter. If the value of the formal parameter changes upon successful execution or if the subroutine is terminated by EXIT, CHECK or RETURN then it will change the value of the actual parameter.

Actual Parameter: It will have the Original Value.

Formal Parameter: It will store the copy of the actual value or Memory address of the actual value



Parameter Interface

Points to Note

1. Nested calls are allowed in subroutines (i.e. PERFORM within a FORM ... ENDFORM).
2. Recursive calls are also possible.
3. To define local data, use the DATA statement after FORM . Each time you enter the subroutine, the data is recreated (with an initial value) and released at the end (from the stack).
4. To define global data used within a subroutine, use the LOCAL statement after FORM . The values are saved when you enter the subroutine and then released at the end (from the stack).

Interview Questions

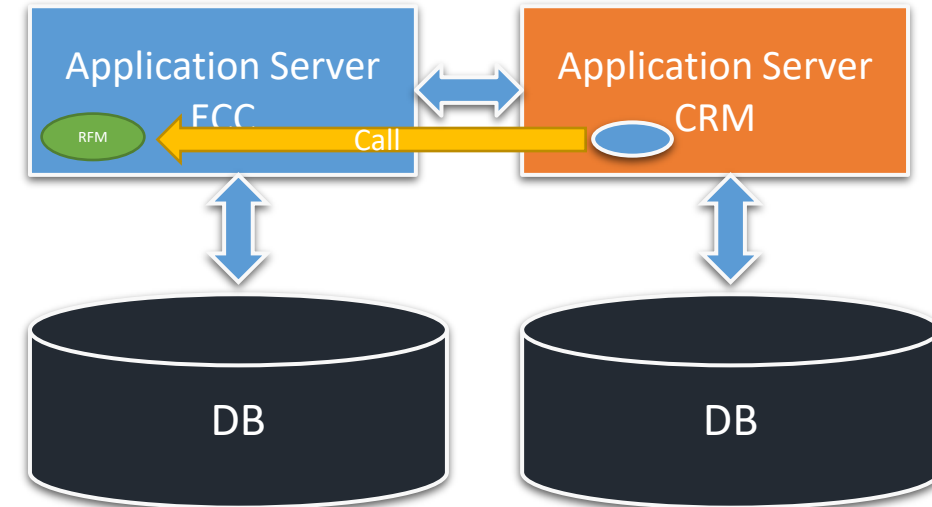
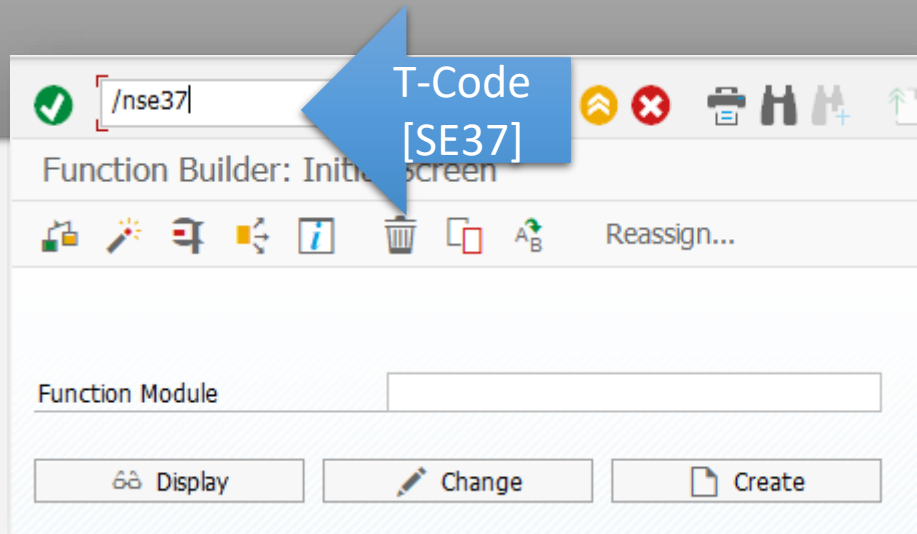
- 1.Can you pass parameters to a subroutine in SAP ABAP? If yes, how?
- 2.What is the maximum number of parameters that can be passed to a subroutine in SAP ABAP?
- 3.What is a parameter interface in SAP ABAP subroutines?
- 4.What is the difference between a function module and a subroutine in SAP ABAP?
- 5.Can you debug a subroutine in SAP ABAP? If yes, how?
- 6.What are the best practices to follow while using subroutines in SAP ABAP?
- 7.How do you pass tables to a subroutine in SAP ABAP?
- 8.Can you have nested subroutines in SAP ABAP? If yes, how do you call them?

Function Modules

Function Modules are a collection of code that can be reused by other programmes. Creating a function module is far more useful and maintainable than copying identical statements within programmes. Function Modules can be developed using **importing, exporting, changing parameters, and exceptions.**

Function Modules can be run independently and from any programme.

Function Group: It's container of function modules. You can keep similar function modules inside a function.



Function Modules

Function Builder: Display ZFM004SORTING

Function module: ZFM004SORTING Active

Attributes Import Export Changing Tables Exceptions Source code

```
1 FUNCTION ZFM004SORTING.  
2  *-----  
3  *""Local Interface:  
4  *  IMPORTING  
5  *    VALUE (TAB4) TYPE  ZTAB001  
6  *    VALUE (I) TYPE  I  
7  *    VALUE (J) TYPE  I  
8  *    VALUE (S) TYPE  I  
9  *  EXPORTING  
10 *    REFERENCE (TAB4_4) TYPE  ZTAB001  
11 *-----  
12  
13 "Sorting logic without using sort statement.  
14 data: emp type zstr01 ,  
15       empl type zstr01.  
16 WHILE I <= J .  
17   s = I + 1 .  
18   WHILE s <= J .  
19     READ TABLE TAB4 INTO emp INDEX i .  
20     READ TABLE TAB4 INTO empl INDEX s .  
21     IF emp-empid > empl-empid .  
22       MODIFY TAB4 FROM EMP1 INDEX i .  
23       MODIFY TAB4 FROM EMP INDEX s .  
24     ENDIF.  
25     S = S + 1 .  
26   ENDWHILE.
```

ABAP

Import Parameters are inputs to the Function Module that are sent from the main programme

The source code is where actual reasoning is implemented.

Exception parameters: These are parameters that cause our code to throw particular exceptions.

Changing Parameters: These are input parameters whose values are altered at the conclusion of the procedure and which then become output parameters for the function module.

Export parameters are Function Module outputs. The parameters are sent from the main programme empty, and they are returned to the main programme with the output value.

Types Of FM's

Function Builder: Display ZFM004SORTING

Function module: ZFM004SORTING Active

Attributes Import Export Changing Tables Exceptions Source code

Classification

Function Group: ZIT01 internal table

Short Text: sorting

Processing Type

☒ Regular Function Module

☐ Remote-Enabled Module ☐ BasXML supported

☐ Update Module

☒ Start immed.

☐ Immediate start (not updateable)

☐ Start Delayed

☐ Coll.run

General Data

Person Responsible: AISHA0295

Last Changed By: AISHA0295

Changed on: 11.07.2022

Package: ZAISHA0295

Program Name: SAPLZIT01

Include Name: LZIT01U06

Original Language: EN

Not released

☐ Edit Loc

☐ Global

FM Can be called within the system.

This Function support Logical Unit of work. Inside this function module we will write DML. Update FM will run in separate session.

Update FM will trigger when **COMMIT WORK** statement is used.

RFC(Remote Function Call) These FM can be called within the system as well as from an external system.

Types Of FM's

Here are some interview questions on **Function Modules (FM) in SAP**:

1. What is a Function Module in SAP and what is its purpose?
2. What are the different types of Function Modules in SAP?
3. How do you create a Function Module in SAP?
4. How can you test a Function Module in SAP?
5. What is the difference between an RFC Function Module and a normal Function Module?
6. How can you transport a Function Module from one system to another?
7. How do you debug a Function Module in SAP?
8. What is the difference between a Function Module and a subroutine in SAP?
9. How can you find out which Function Module is used in a program in SAP?
10. What is the use of export and import parameters in a Function Module?
11. How can you pass table parameters to a Function Module in SAP?
12. What is the use of exceptions in a Function Module?
13. What are the advantages of using a Function Module in SAP?
14. How do you call a Function Module from another Function Module?
15. How can you optimize a Function Module for better performance in SAP?

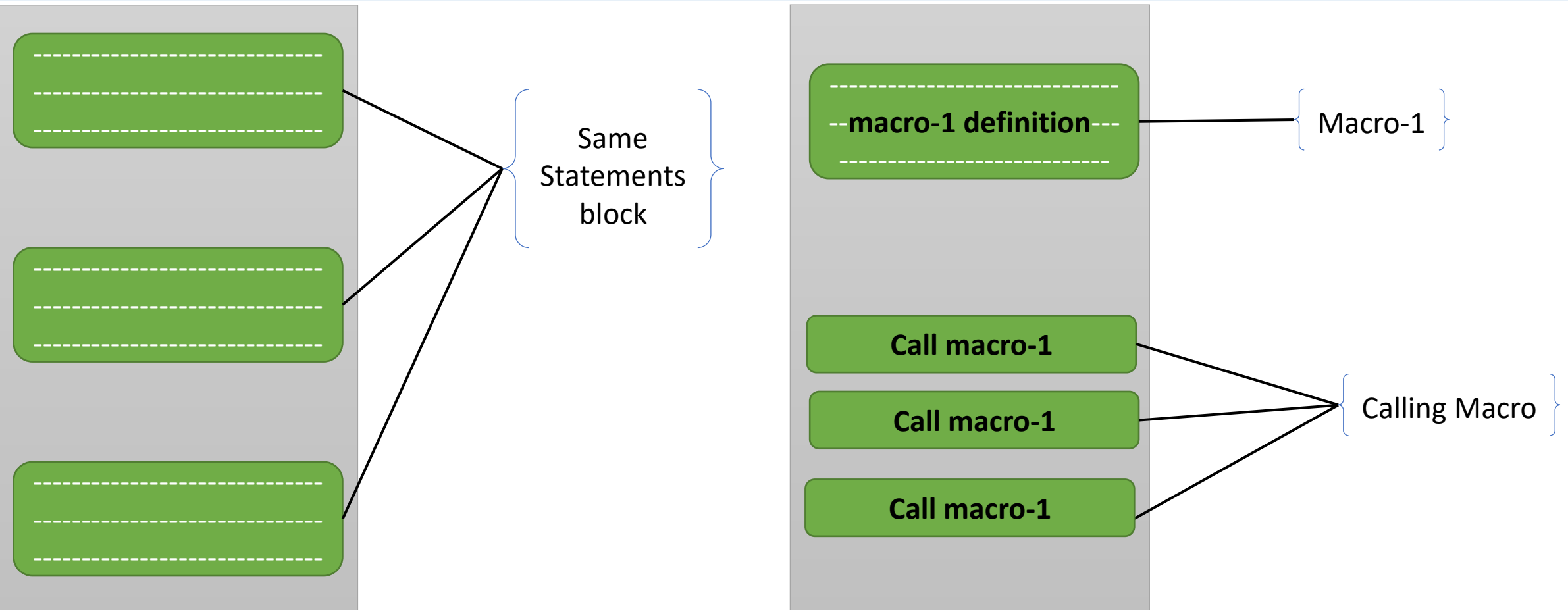
Macros

Macros are obsolete now!

If we wish to reuse the same collection of statements in a programme more than once, we can incorporate them in a macro.

A macro can only be used within the programme in which it is defined.

The macro should be defined before it is used in the programme.



Program Y without macro

Program Y with macro

Macros

We can use the following syntax to define a macro.

DEFINE <macro name>.

.....

END_OF_DEFINITION.

here are some interview questions on macros in SAP:

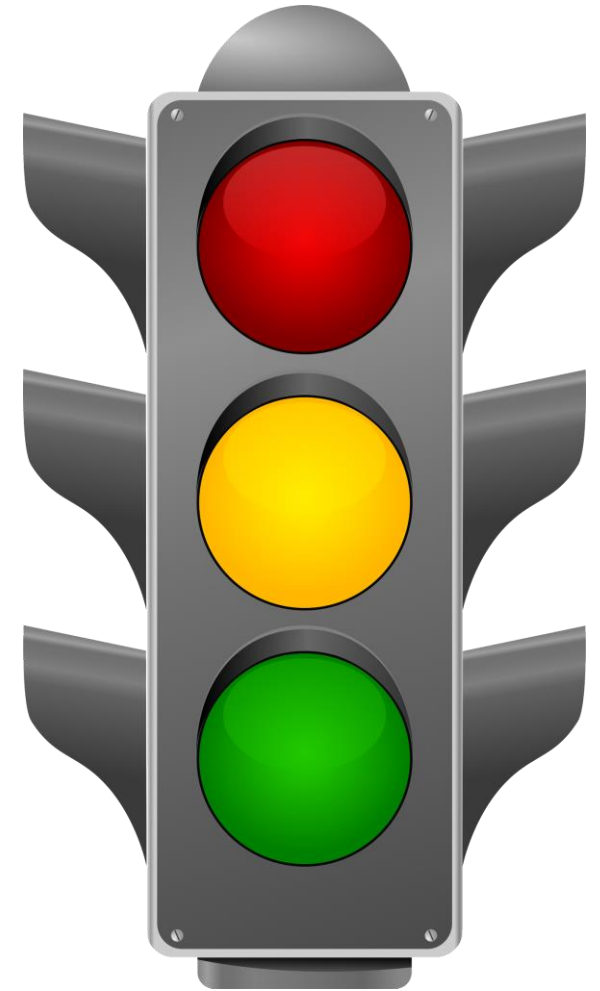
- 1.What are macros in SAP?
- 2.How are macros different from subroutines in SAP?
- 3.Can you explain the syntax of a macro in SAP?
- 4.What is the use of '&' symbol in a macro in SAP?
- 5.How can you pass parameters to a macro in SAP?
- 6.What is the scope of a macro in SAP?
- 7.How can you call a macro from an ABAP program in SAP?
- 8.What is the significance of '&' in a parameter list while calling a macro in SAP?
- 9.Can you use macros to call other macros in SAP?
- 10.Can you use macros to call a function module in SAP? If yes, how?

Debugging

As a SAP coder, you know the frustration that comes when a program you've worked hard on throws a runtime error or gives a dump. But fear not, for SAP debugger is here to save the day! This powerful tool allows you to examine your code line by line and diagnose any errors that may be occurring during runtime. With debugging in SAP ABAP, you have the ability to gain insight into the inner workings of your program and check its flow in real-time.

Ever since its release in SAP 7.0, ABAP debugger has been the go-to tool for developers looking to troubleshoot their code and ensure its seamless execution. So the next time your program hits a snag, don't panic - just fire up the debugger and get to work!

Imagine you're driving a car, and suddenly you see a red light at the intersection. You have to stop the car and wait for the green light to proceed. Similarly, in programming, a break-point acts as a red light that stops the execution of the program and gives control back to the programmer. The programmer can then inspect variables, step through the code, and even change the program's logic to fix any issues. With break-points, you're in the driver's seat, able to control the program's execution and make sure it's running as intended.



Debugging

Types of Break Points

1. **Soft Break-Point/Session Break Point:** Session Breakpoints are only valid for particular session
2. **Debugger Break Point:** These breakpoints are only valid for particular debugger session
3. **External Break Point:** If the program/Function Module called from an external system(SAP system or Non-SAP System) and you want debug the code which is written in SAP system to understand or identify the logical errors then you can keep external break-point.
4. **Hard Break Point:** **Not Recommended.** When you use keyword **BREAK-POINT** in the source called as hardcoded break point. We can't deactivate/Remove the break-point in the debugger. Their a chance that we might forgot to remove the breakpoint before moving to production system.

Single Step: F5: Used to execute the program by line by line

Execute: F6: It's used to skip the block of code if user don't want to see execution of the particular block

Return: F7: If you're e already inside a procedure block and you want to come outside of procedure block then you can press F7

Continue: F8: Used to execute the program. If there are any further breakpoints it will stop the program execution at the breakpoint otherwise it complete the program execution.

Watch Point: Watchpoints are used to track the changes to the value of data objects. ABAP debugger will stop the program execution when the condition of the watcher data objects meets(Conditional Break Points)

Debugging

Some interview questions on Debugger in SAP ABAP:

1. What is Debugger in ABAP? How is it useful in debugging programs?
2. What are the different types of breakpoints that can be set in Debugger?
3. What is Watchpoint in Debugger? How is it useful in debugging?
4. Can you explain the difference between a Static and Dynamic Breakpoint?
5. How do you use the Evaluate Expression feature in Debugger?
6. Can you explain the difference between Single Step and Into statement in Debugger?
7. How do you analyze runtime errors using Debugger?
8. Can you explain the difference between a Local and Global breakpoint in Debugger?
9. How do you set Breakpoints in ABAP Objects?
10. What are the different debugging techniques you can use in ABAP?



OOABAP(Object Oriented ABAP)

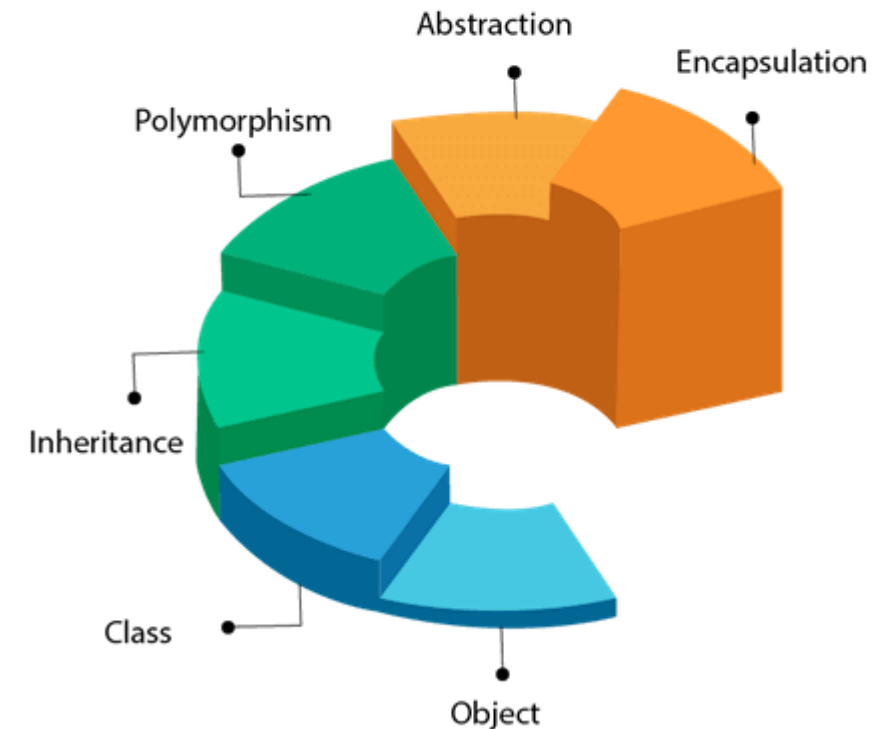
INTRODUCTION:

Object-Oriented ABAP, or OOABAP for short, is an extension of ABAP that uses an object-oriented approach to programming. It emphasizes the creation and manipulation of objects, which are like real-life entities that have properties and behaviors. Think of it as giving your code a more human touch.

To put it simply, just like a person with knowledge and abilities can perform different tasks, an object in OOABAP has fields to store data and can perform various functions based on its properties and methods.

The classes and objects are the building blocks of OOABAP, and they allow you to create reusable, modular code that can be easily maintained and modified. So, instead of just focusing on the code flow, OOABAP helps you think about the bigger picture of how objects interact with each other to accomplish tasks in a more efficient and intuitive way.

OOPs (Object-Oriented Programming System)



OOABAP(Object Oriented ABAP)

OBJECT:

In the world of programming, objects are like real-life entities that have their own unique identity and set of characteristics. Just like a person, an object can have different attributes, such as their height, weight, or hair color. These attributes define the state of the object. However, just like how a person's state can change over time, an object's state can also change based on its behavior, which refers to the changes that occur in its attributes over a period of time.

In the world of ABAP, an object can be associated with a class by declaring it with the same data type as the class within which it was created. This allows programmers to think of their code in terms of real-life entities and create programs that are more intuitive and easier to maintain.

Imagine an object as a living creature, like a pet. Just like every pet has its own unique traits, characteristics, and behaviors, an object also has its own distinct features. For example, a pet dog may have a specific breed, fur color, and temperament, which are its state or attributes. Its name and microchip number make it uniquely identifiable among other dogs. Finally, its behavior, like whether it likes to play fetch or not, is another characteristic that distinguishes it from other dogs.

Similarly, an object in programming also has

1. **a state (its attributes)**
2. **a unique identity**
3. **may or may not exhibit certain behaviors depending on how it was designed.**



OOABAP(Object Oriented ABAP)

CLASS:

Imagine you want to build a house. You can't just start building without a plan, right? Similarly, in programming, you need a plan for your objects, and that's where classes come in. ***A class is like a blueprint for an object, which defines the attributes and behavior of that object.***

For example, let's say you want to create a class for a car. You would define the attributes of the car, such as its color, make, model, and year. You would also define the behavior of the car, such as how it starts, how it accelerates, and how it brakes.

We'll break down the components of a class in later post.



The three pillars of OOABAP

Encapsulation:

Imagine you have a top secret formula for a new soft drink that you want to keep confidential. **Encapsulation is like a protective shield around that formula that keeps it safe from unauthorized access.** It groups similar types of data together and hides them from other parts of the program that don't need to access them.

There are three levels of visibility in encapsulation.

Private means that only the class that defines it can access it.

Protected means that the class and any classes that inherit from it can access it.

Public means that the component is available to any part of the program.

By using encapsulation, you can control which parts of your program have access to certain data, reducing the risk of errors or security breaches.



The three pillars of OOABAP

Polymorphism:

Imagine you have a special move in a video game that allows you to defeat your enemy, but you can only use it once. ***Polymorphism is like a superpower that allows you to reuse a piece of code in different contexts without having to write it all over again.*** With polymorphism, you can reuse that same move in different levels or against different enemies, with some modifications to make it even more powerful.

In object-oriented programming, we use interfaces to achieve polymorphism.

An interface is like a blueprint for a function that can be implemented differently in different classes. This means that a single method or function can have multiple implementations that are specific to different objects or scenarios.

It's like having a recipe for a cake that can be customized with different flavors or toppings to create different variations of the same cake.

The great advantage of polymorphism is that it saves us a lot of time and effort. Instead of writing new code every time we need to perform a similar task, we can reuse the same code with some tweaks to make it work in a different context. This not only saves us time but also makes our code more organized and easier to maintain.



The three pillars of OOABAP

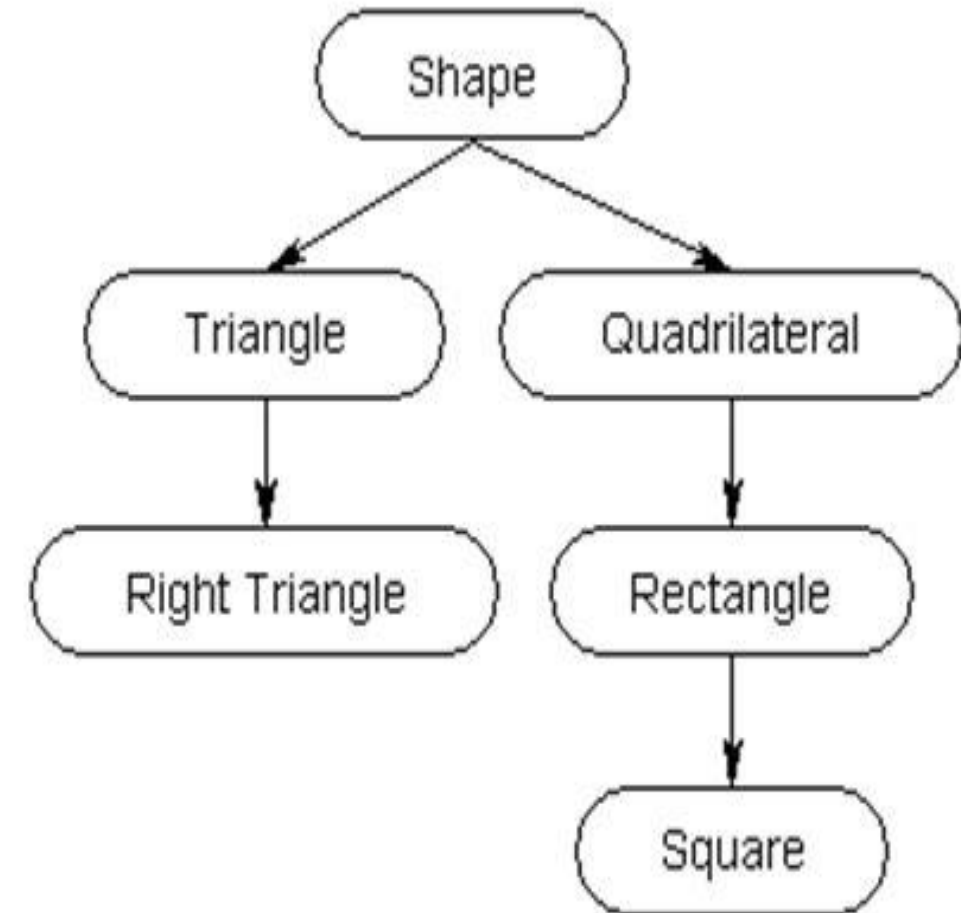
Inheritance :

Inheritance is like a family tree where children inherit traits from their parents. Similarly, in object-oriented programming, ***a child class can inherit properties and methods from its parent class.***

For example, let's say we have a parent class called "Vehicle" with properties such as "make", "model", and "year". We also have a child class called "Car" which inherits these properties from the parent class. However, the child class can also have its own unique properties and methods, such as "number of doors" or "engine size".

Inheritance is useful because it allows us to reuse code and avoid redundancy.

For instance, if we have another child class called "Truck" which also needs the properties of "make", "model", and "year", we can simply create a new class called "Truck" that inherits from the "Vehicle" class instead of rewriting all the code for those properties. This saves time and makes our code more efficient.



The three pillars of OOABAP

Advantages of Object-Oriented ABAP (OOABAP):

- 1.Improved Data Management:*** OOABAP programs are divided into objects, which leads to better and more powerful data management.
- 2.Encapsulation:*** OOABAP provides properties like data hiding (encapsulation) and code reusability (inheritance) with more data security, making it easier to maintain and debug code.
- 3.Better Performance:*** OOABAP is faster and consumes less time due to the concept of polymorphism, inheritance, and other features.
- 4.Future Orientation:*** OOABAP is relatively flexible and adaptable to changing business needs, making it easier to modify or extend applications as needed.
- 5.Improved Maintenance:*** OOABAP is simple and easier to maintain as compared to procedural ABAP programming. Also, the code can be reused with some modifications.
- 6.Power and Flexibility:*** Object-oriented programming languages include features such as "Class," "Instance," "Inheritance," and "Polymorphism" that increase the power and flexibility of an object, leading to better and more efficient programming.

The three pillars of OOABAP

Some interview questions related to the advantages of OOPS in ABAP:

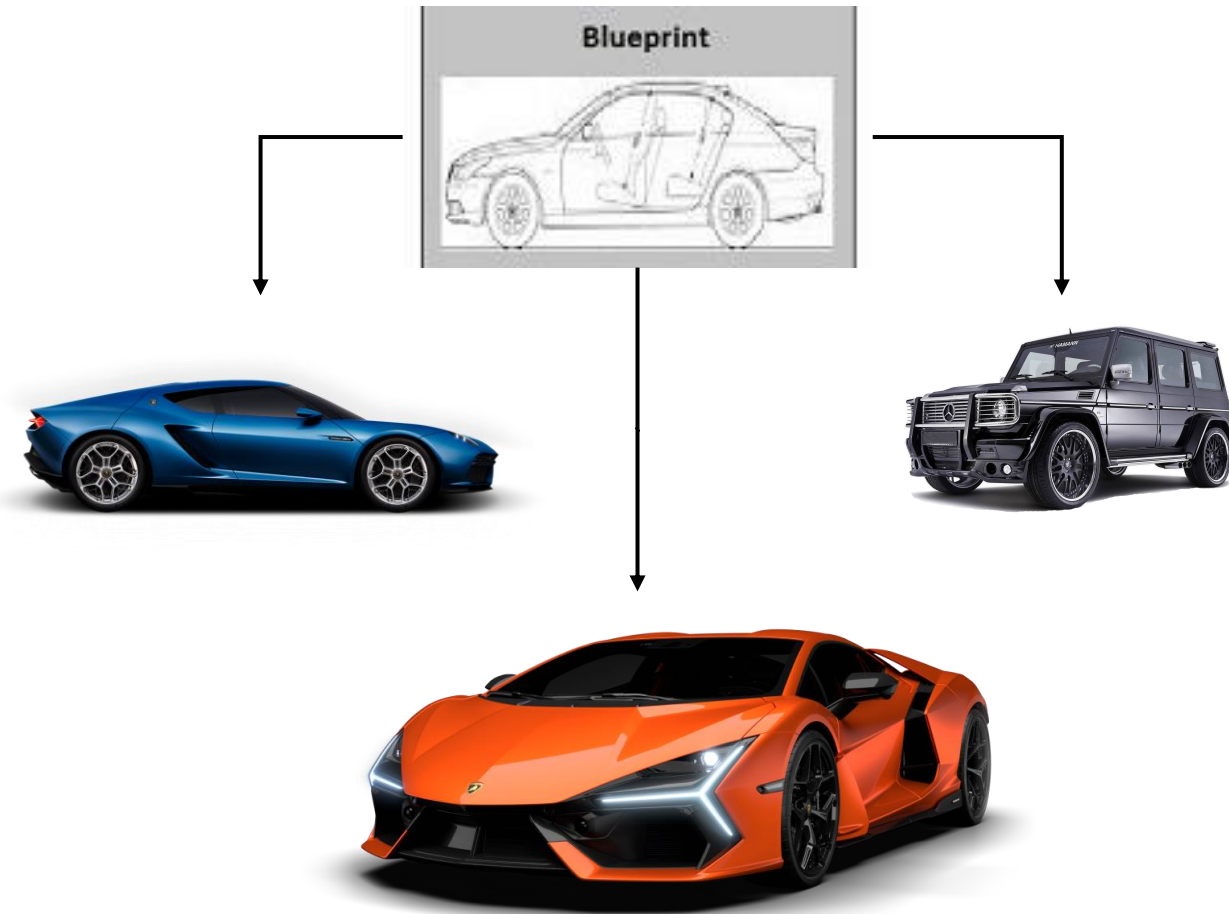
1. *What is the difference between data hiding and encapsulation?*
2. *How does inheritance help in code reusability? Can you give an example?*
3. *Can you explain how polymorphism works in ABAP?*
4. *How does OOPS make programming more maintainable compared to procedural programming?*
5. *What is the benefit of using objects in ABAP for data management?*
6. *Can you give an example of how OOPS helped in making a program more flexible and adaptable to changing business needs?*
7. *What are the features of OOPS that make it more powerful and flexible than procedural programming?*
8. *How does OOPS improve performance and reduce consumption of time?*
9. *What are some best practices to follow when developing object-oriented programs in ABAP?*
10. *Can you explain the concept of abstraction and how it is used in OOPS?*

SAP ABAP Classes

In our previous post, we explored the essential concepts and benefits of Object-Oriented ABAP. Now, let's take a deeper dive into one of its major components - **SAP ABAP Classes**.

In OOABAP, everything revolves around classes and their objects. Classes define the object they are holding, and each class can have an infinite number of objects with a unique combination of state and behavior. The classes contain properties, methods, and other attributes that help developers define what the class represents.

However, a class on its own is useless unless it has some objects. An object, also known as the instance of a class, is essential to give meaning to the class. We create an instance of an object that points to a class to make the class useful.



SAP ABAP Classes

Imagine ABAP Classes as different types of ice cream flavors that come in different sizes.

One flavor of ABAP Class is the "**Local Class**," which is like a small scoop of ice cream that can only be found and enjoyed within a specific program. This means that the class can only be accessed within that program and cannot be used outside of it.

To create a Local Class, developers need to declare it within the program by using the "**CLASS DEFINITION**" coding block. This is where they can define all the components of the class, such as its attributes, methods, and events.

After defining the class, developers can then implement the methods of the class in the "**CLASS IMPLEMENTATION**" block. *It's important to note that if methods are declared in the **CLASS DEFINITION**, then they must be implemented in the **CLASS IMPLEMENTATION**.*

In summary, ABAP Classes come in different "flavors," with Local Classes being a small, program-specific scoop of ice cream that can only be accessed within that program.



SAP ABAP Classes

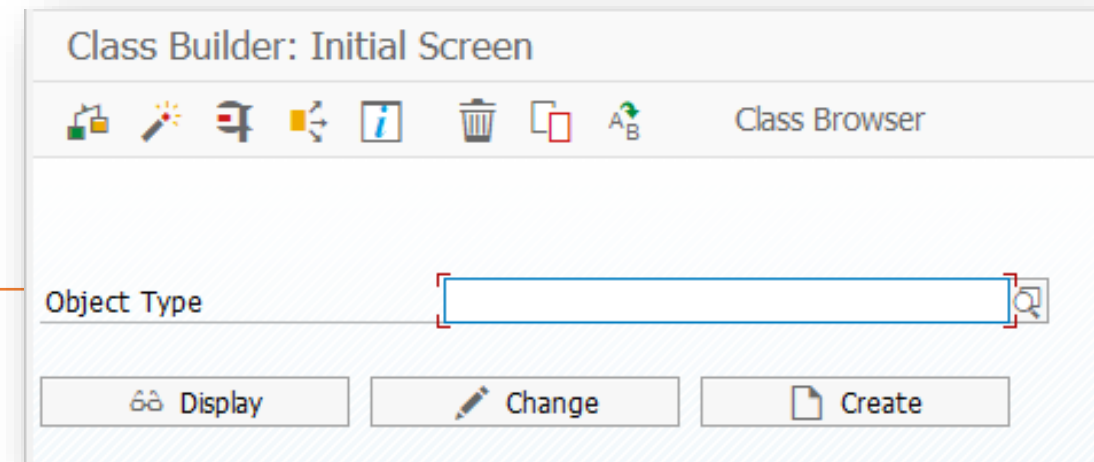
Do you know that just like how we humans have a structure for our body, SAP ABAP classes also have a structure!

And this structure is divided into 3 parts –

1. ***Class Components***
2. ***Visibility section of Components***
3. ***Implemented methods of class.***

When it comes to creating a global class, you can use the T-code **SE24 (Class Builder)** in ABAP workbench. Global classes are accessible to all ABAP programs by instantiating an object of that class. This means that the same class can be used in multiple programs without having to define it again and again. Cool, isn't it?

Let's discuss all the structure of class in detail in future posts!



SAP ABAP Classes

So, in SAP OOABAP, we have two types of components: instance and static.

Instance components are like personal belongings that belong to each object, while **static components** are like shared belongings that belong to the whole class.

To give you a fun analogy, let's compare instance components to a backpack. Imagine you're going on a hike, and you bring your trusty backpack with you. Inside the backpack, you have your water bottle, snacks, sunscreen, and a map. These items belong to you and only you, and they are specific to your backpack. Just like instance components in a class, they belong to each object and are unique to that object.



On the other hand, **static components** are like a shared community garden. Imagine you and your neighbors have a community garden where you all grow vegetables together. You have a shared tool shed that contains tools like shovels, rakes, and watering cans. These tools are not specific to one person, but rather they are shared among all the gardeners. Just like static components in a class, they belong to the whole class and are shared among all the objects.

I hope this analogy helps you understand the difference between instance and static components in SAP OOABAP in a more fun and interesting way!



SAP ABAP Classes

In SAP ABAP programming, a class is a blueprint that defines the attributes and behaviors of an object. There are three types of components that can be included in a class: **attributes, methods, and events**.

Attributes are variables or constants that are declared within a class and can be accessed by all the methods of that class. There are two types of attributes: instance attributes and static attributes. Instance attributes are specific to each object in a class, while static attributes are defined for the whole class.

Methods determine the behavior of an object and provide functionality. Similar to attributes, there are two types of methods: instance methods and static methods. Instance methods can be accessed by all the attributes of a class, while static methods can only access static attributes.

Events are a mechanism through which one method can raise the method of another class. There are two types of events: instance events and static events. Instance events can only be triggered in an instance method, while static events can be triggered by both static and instance methods.

Remember that each component name should be unique within a class to avoid conflicts. By using these components in your classes, you can create more efficient and powerful programs in SAP ABAP.



SAP ABAP Classes

Imagine you have a secret diary where you write down all your thoughts and feelings. The diary is like a class in OOABAP, and the entries you make are like the components of the class.

Now, you don't want anyone else to read your diary, so you keep it private. This means that the entries you make in your diary are only visible to you, just like the components with a ***"private" visibility section in a class are only visible to the class where they are defined.***

However, you might want to share some of your thoughts and feelings with your best friend. In this case, you would let them read your diary, but you still wouldn't want anyone else to see it. This is like the components with a ***"protected" visibility section in a class, which are visible not only to the class where they are defined but also to any class that inherits from it.***

Finally, there might be some entries in your diary that you're happy to share with anyone who wants to read them. These are like the components with a ***"public" visibility section in a class, which are available to all classes.***

In summary, the visibility section of a class determines who can see and use the components of the class. Just like you might want to keep some thoughts and feelings private, some components of a class might need to be hidden away, while others can be shared with the wider world.

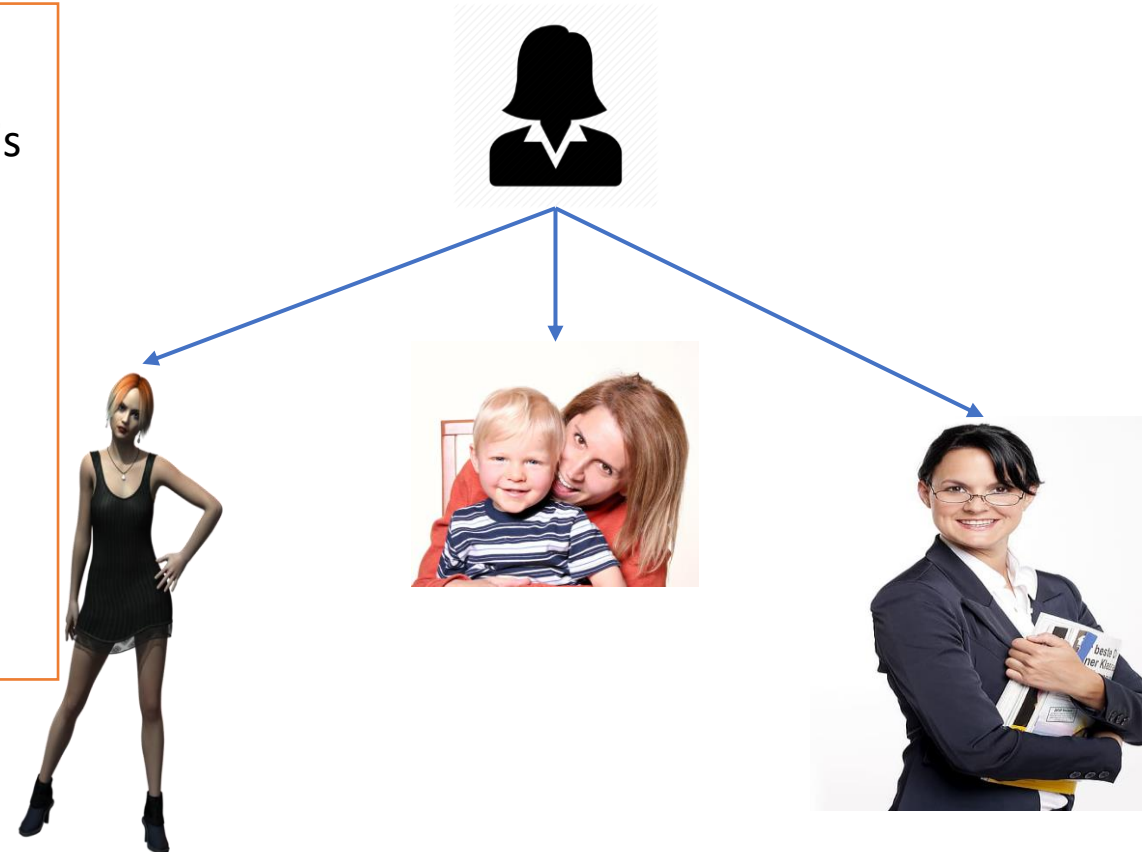


Interfaces

Object-Oriented Programming (OOP) has four important principles: Encapsulation, Abstraction, Inheritance, and Polymorphism.

Polymorphism means that one message can have many forms, and it's achieved through interfaces. **Interfaces allow for multiple implementations of the same concept or idea, which extends the functionality of classes.**

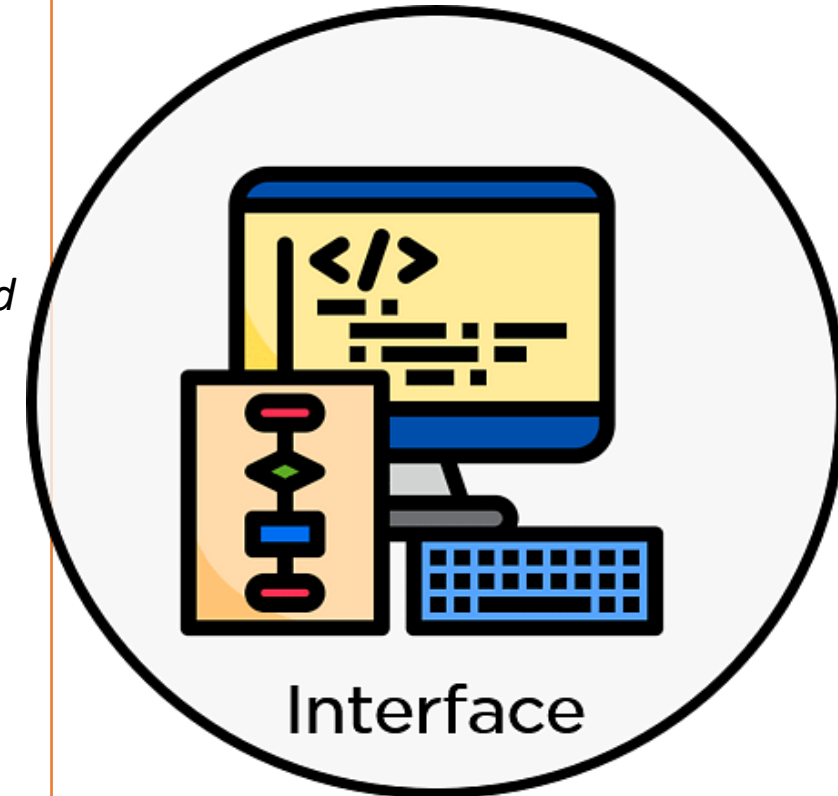
Think of a person who can have multiple roles or features, such as being a mother, wife, daughter, or employee, and performs different actions based on the situation. Similarly, an interface is a self-contained entity that can have many implementations, which enhances the capabilities of classes.



Interfaces

Interfaces are like a set of rules that define what methods should be available to any class that implements them. Here are some key properties of interfaces:

- ✓ *Interfaces only declare method signatures, not their implementation. The implementation is done in the classes that implement the interface.*
- ✓ *Any number of classes can implement the same interface, making it easy to extend the functionality of multiple classes.*
- ✓ *Each implementing class can have its own implementation of the interface methods, allowing for customization based on the class's specific needs.*
- ✓ *Unlike classes, interfaces don't have a visibility section in their declaration. However, when a class implements an interface, it must list the interface in its public section.*
- ✓ *You can create interfaces using the SE24 transaction in ABAP. Simply provide a name for the interface, list the methods it should contain, and then implement the interface in the classes that need it.*



Interface's Example

Imagine you have different types of animals, each with their own unique behaviors. For example, a dog can bark, a cat can meow, and a bird can chirp. Now imagine you want to create a program that allows you to interact with these animals and their behaviors in a standardized way, without having to worry about the specific details of each animal.

This is where interfaces come in handy. An interface is a set of methods that define a certain behavior. In the animal example, you could create an interface called "**AnimalBehavior**" that includes methods such as "**makeSound**" or "**move**". Then, you could create classes for each animal that implement this interface, which means they must provide their own implementation for each method in the interface.

By using interfaces, you can allow for the option of having multiple classes that implement the same behavior in different ways, but still use the same method names. This allows for greater flexibility and extensibility in your code.

To implement an interface in a class, you simply list the interface name in the class definition using the "**INTERFACES**" statement. Then, you must provide an implementation for each method in the interface in the class's implementation part. The components defined in the interface can then be addressed in the implementation part of the class.

Interface

Interview Questions

1. *What is an interface in ABAP Object-Oriented Programming?*
2. *How do you create an interface in ABAP?*
3. *What are the properties of an interface in ABAP OOP?*
4. *How are interfaces different from abstract classes in ABAP?*
5. *What is the purpose of interfaces in ABAP OOP?*
6. *Can an interface inherit from another interface in ABAP OOP?*
7. *Can a class implement multiple interfaces in ABAP OOP?*
8. *What is the use of interface methods in ABAP OOP?*
9. *How do you implement an interface in ABAP OOP?*
10. *How do you ensure that all methods defined in an interface are implemented in class that implements that interface?*



TASK

Here's a task for a fresher to practice implementing interfaces in OOABAP:

Task: *Create a program that uses an interface to calculate the area and perimeter of different shapes.*

Create an interface named "**Shape_Interface**" with the following methods:

getArea () : returns the area of the shape

getPerimeter () : returns the perimeter of the shape

Create three classes that implement the **Shape_Interface** interface: Circle, Rectangle, and Triangle. Each class should have its own implementation of the **getArea()** and **getPerimeter()** methods, depending on the type of shape.

In your program, create an object of each of the three classes (Circle, Rectangle, and Triangle).

Use the objects to calculate and print the area and perimeter of each shape.

Add comments to your code explaining the purpose of each method and any other relevant information.

Test your program with different values to ensure that it works correctly.

Optional: Add a new shape class (e.g., Square) that implements the **Shape_Interface** and test your program with this new shape.

Good luck with your task!

Abstract Class and Interface

*Imagine you're building a superhero team. You have a blueprint for a superhero with specific powers and abilities, but you don't know what their specific superhero name and costume will be. **That blueprint is like an Abstract Class! It's a special kind of class that can't be used to create an object directly, but it provides a framework for subclasses that can.***

Abstract Classes must have at least one abstract method, which is like a placeholder method without any implementation - it's just a declaration. This means that the subclasses of the Abstract Class must implement these abstract methods with their own specific implementation.

Just like how you can't hire a generic superhero without a specific name and costume, you can't create an object of an Abstract Class. However, you can define variables that reference the Abstract Class and then assign them at runtime to specific subclasses that have been implemented. This allows for a lot of flexibility and customization in your program.



Abstract Class and Interface



let me explain Interface in simpler terms:

An interface is a special thing in programming that only has empty method declarations and components. It doesn't actually have any implementation code, which means it doesn't do anything on its own. But, it allows other classes to use its method declarations and components to build their own functionality.

Think of it like a blueprint or a template that other classes can use to build something. The interface sets the guidelines for what methods and components should be included, but it's up to the implementing classes to actually put in the code to make those methods work.

Abstract Class and Interface

Abstract classes and interfaces are two distinct entities in ABAP OOP and they differ in a few ways. Here are some interesting differences between them:

Multiple Inheritance: Interfaces provide a way to achieve multiple inheritance in ABAP, while abstract classes cannot. This means that a class can implement multiple interfaces to inherit functionality from multiple sources, but can only inherit from one abstract class.

New Functionality: When a new method is added to an interface, all implementing classes are required to implement it, otherwise, it will result in a run-time error. However, in the case of an abstract class, if a non-abstract method is added, it is not required to redefine it in each subclass.

Default Behavior: Abstract classes can provide a default behavior for non-abstract methods, but interfaces only contain empty stubs and cannot have any implementation. This means that classes implementing an interface must provide their own implementation for all methods.

Visibility: All components of an interface are public by default, but for abstract classes, the visibility of each component can be set according to the requirements of the implementation.

Understanding these differences is important when designing a class hierarchy and choosing whether to use an abstract class or an interface to achieve the desired functionality.

Abstract Class and Interface

Let's take a look at some exciting recommendations based on the differences between interfaces and abstract classes:

1. **Firstly**, if you want to achieve multiple inheritance, then interfaces are the way to go. You can have multiple interfaces in a class definition, and it allows you to achieve multiple inheritance.
2. **Secondly**, if you wish to create various types or flavors of objects with some default operations, it's better to use abstract classes. With abstract classes, you can add default operations to the base class, and all subclasses inherit the new default operations.
3. **Thirdly**, if you want to provide common, implemented functionality among all implementations of your component, then abstract classes are the way to go. Abstract classes allow you to partially implement your class, whereas interfaces don't contain any implementation for members.
4. **Fourthly**, if you are working with a wide range of objects, you should use interfaces. Interfaces only contain the stub and enforce no default behavior. On the other hand, abstract classes should be used for classes that are closely related to each other.
5. **Fifthly**, interfaces can act as a common point to hierarchies that can't sit next to each other, and they gain interface functionality regardless of where they lie in the hierarchy tree. If they require common implementation, a helper class can be utilized.

So, these are some exciting recommendations that one should keep in mind while working with interfaces and abstract classes.

Abstract Class and Interface**Interview Questions:**

1. *What is an abstract class and how is it different from a regular class?*
2. *Can an abstract class be instantiated? Why or why not?*
3. *Can an abstract class have non-abstract methods? What is their purpose?*
4. *What is an interface and how is it different from an abstract class?*
5. *Can an interface have non-public methods or components? Why or why not?*
6. *Can a class implement multiple interfaces? Why or why not?*
7. *What is the purpose of using an interface in a program?*
8. *When would you choose to use an abstract class instead of an interface?*
9. *Can an abstract class inherit from another class or abstract class? Why or why not?*
10. *How do you define an interface in ABAP? How do you define its components and methods?*
11. *Can you give an example of a scenario where you would use an interface to achieve a specific functionality in a program?*
12. *How do you implement an interface in ABAP? Can you give an example?*
13. *What is the purpose of an abstract method in an abstract class or interface?*
14. *How do you access the components of an interface or abstract class within a class that implements them?*
15. *Can an interface have implementation code for its methods? Why or why not?*

Class Constructors

Class Constructors are like the chefs in a restaurant kitchen who prepare the ingredients and assemble them to make a delicious dish. In the same way, Class Constructors prepare and initialize the data members of a class so that they can be used later by other parts of the program.

Think of a Class Constructor as the mastermind behind a perfect heist, where they carefully plan and execute the operation to achieve their goal. Similarly, the Class Constructor plans and executes the initialization of data members to ensure smooth operation of the program.

But just like a mastermind needs tools to carry out the heist, Class Constructors also need tools to carry out their tasks. These tools are called parameters, which are like the different gadgets and tools that a mastermind uses to execute the heist. Parameters help the Class Constructor to initialize the data members with specific values, making the program more flexible and efficient.

To summarize, ***Class Constructors are like the chefs and masterminds of a program, carefully preparing and initializing data members, and using parameters as their tools to achieve their goals.***



Class Constructors

In SAP ABAP, a constructor is a special method that is automatically called when an object of a class is created. The purpose of a constructor is to initialize the state of an object. The state of an object refers to its instance variables or attributes, which are defined in the class.

A constructor method has the same name as the class and does not have a return type. In other words, it always returns the instance of the object that it has created. In addition to the instance variables, a constructor can also take parameters to initialize the object's state.

In ABAP, there are two types of constructors:

Instance Constructors: These constructors are used to create objects of a class. They are defined using the `CONSTRUCTOR` keyword followed by the class name. Instance constructors can be overloaded, meaning there can be multiple constructors with different parameters in the same class.

Static Constructors: These constructors are used to initialize static attributes of a class. They are defined using the `CLASS_CONSTRUCTOR` keyword followed by the class name. Static constructors are executed only once, when the class is loaded into memory.

Class Constructors

Constructors can also be used to perform certain operations when an object is created. For example, a constructor can be used to connect to a database or to initialize a log file.

One important point to note is that if a class does not have a constructor defined explicitly, then the system automatically creates a default constructor that does not take any parameters and does nothing. However, it is always recommended to define at least one constructor in a class to ensure that the object's state is initialized properly.

In summary, *constructors in SAP ABAP are methods that are automatically called when an object is created. Their main purpose is to initialize the object's state, and they can take parameters to do so. There are two types of constructors - instance and static - and they can be used to perform certain operations when an object is created.*

Interview Questions:

1. What is a constructor in ABAP?
2. What is the purpose of a constructor in ABAP?
3. How many types of constructors are available in ABAP?
4. What is the difference between a static constructor and an instance constructor?
5. What is the difference between a public constructor and a private constructor?
6. How do you define a constructor in ABAP?
7. Can you have multiple constructors in a single ABAP class?
8. What is the order of execution of constructors and other methods in a class?
9. What happens if a constructor is not defined explicitly in a class?
10. How do you pass parameters to a constructor in ABAP?

Friend Class

In object-oriented programming, a "friend class" is a special relationship between two classes that grants one class access to the private and protected members of the other class.

In ABAP, a "friend class" is created using the FRIENDS statement, which is used to declare a class as a friend of another class. The friend class can access private and protected components of the class it is friends with, just as if they were public components.

The friend class relationship is useful in situations where you want to give certain classes access to the internals of another class, without exposing those internals to the public. This can be especially helpful when you have related classes that need to work closely together, but you want to keep certain details private to maintain encapsulation.

It's important to note that the friend class relationship is one-way. Only the friend class can access the private and protected components of the class it is friends with - the reverse is not true.



F.R.I.E.N.D Class

Some advantages of using friend classes in OOABAP :

- 1. **Controlled access:** By using a friend class, you can allow access to certain private or protected components of a class to only a limited number of other classes. This can help you control access to critical data and methods, ensuring the integrity of your program.*
- 2. **Avoid excessive getter and setter methods:** With a friend class, you can give another class access to private and protected components directly, eliminating the need for excessive getter and setter methods. This can help simplify your code and improve performance.*
- 3. **Encapsulation:** Friend classes can help maintain encapsulation by ensuring that only authorized classes can access certain components of a class. This can help prevent unintended changes to the behavior of your code.*
- 4. **Flexibility:** Using friend classes can give you greater flexibility in designing and implementing your code. You can use them to create more complex relationships between classes, which can help improve the overall structure of your program.*
- 5. **Efficiency:** Friend classes can help improve the efficiency of your code by reducing the amount of code you need to write and maintain. By eliminating the need for getter and setter methods, you can make your code simpler and more streamlined.*

F.R.I.E.N.D Class

Task: Implement a banking application that allows bank tellers to create new customer accounts and perform transactions on behalf of customers. The application should have two main classes - Customer and Transaction. **GOOD LUCK!**

STEPS →

1. The Customer class should have the following attributes: name, account number, account balance, and contact information. It should also have methods to deposit, withdraw, and check balance.
2. The Transaction class should have the following attributes: transaction ID, transaction type (deposit or withdrawal), transaction amount, and transaction date. It should also have a method to display transaction details.
3. The bank teller should have access to create new customer accounts and perform
4. transactions on behalf of customers. However, only bank managers should be able to
5. view the transaction details.
6. To implement this functionality, create three classes - Customer, Transaction, and BankManager. The Customer and Transaction classes should be public. The BankManager class should be a friend class to the Transaction class, which will allow it to access the transaction details.
7. Create a program that allows bank tellers to create new customer accounts and perform transactions on behalf of customers. The program should also allow bank managers to view the transaction details.

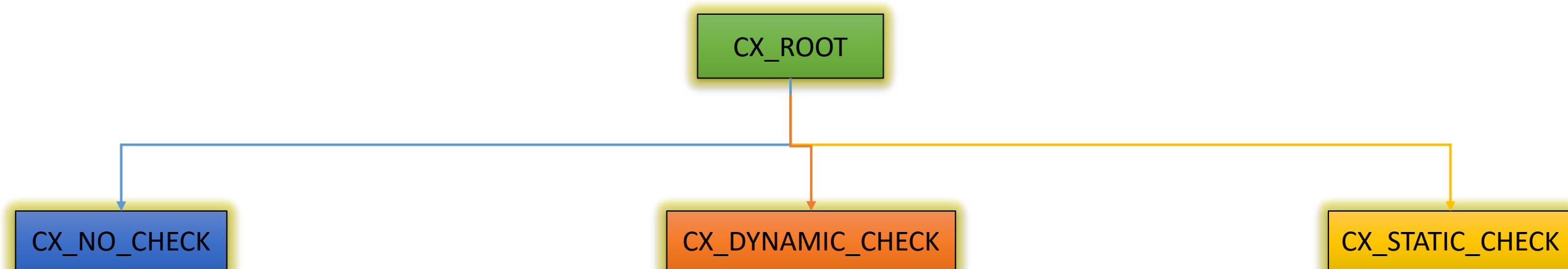


Exception Class

*When we execute any program, we expect it to behave as intended. However, sometimes discrepancies or errors occur which can interrupt the normal flow of execution. These errors are like events that occur due to some reason and are known as **'Exceptions'**.*

*In order to handle such exceptions, SAP ABAP has a special class known as an **'Exception Class'**. An exception class is a blueprint for defining custom exceptions that can be thrown during the runtime of a program.*

The purpose of an exception class is to prevent the execution flow of any program from being interrupted by handling the exceptions. Exception classes can be defined in both global and local classes.



Exception Class

Think of yourself as a superhero trying to save the world of programming from chaos and confusion. In this world, there are different types of villains (exceptions) lurking around, waiting to disrupt the smooth flow of programs. To fight these villains, you need different tools (exception classes) at your disposal.

***Finally, we have CX_STATIC_CHECK** - this is like a hammer that can smash through any obstacle. This type of exception class is checked both by the compiler and runtime system. It is used to check for syntax errors and helps to prevent errors before runtime.*



***Next, we have CX_DYNAMIC_CHECK** - this is like a laser beam that can accurately target and eliminate the villains. This type of exception class is checked only at runtime and includes all system exceptions. It provides a more detailed error message, allowing developers to quickly identify and resolve issues.*



***First, we have CX_NO_CHECK** - this is like a ninja star that can quickly and efficiently take down the villains. This type of exception class is chosen when exceptions occur frequently and there is no need for a detailed error message. With CX_NO_CHECK, the system can continue the program flow without any further interruption.*



Exception Class

Imagine you are a chef and you are making a delicious cake. You follow the recipe step by step, but suddenly, you realize that you ran out of sugar! Now, this is a discrepancy or an exception, and it needs to be handled.

In ABAP, an exception class is like a superhero that comes to your rescue when such discrepancies occur. When an exception happens during the execution of a program, an object (or superhero) is created with information about the error. This superhero can be caught by using **TRY....CATCH.....ENDTRY block**.

Think of TRY....ENDTRY block as a shield that protects your program from unexpected errors. Inside the shield, you can catch the superhero using the CATCH statement. One TRY....ENDTRY block can catch more than one superhero, but you need to catch them in the sequence they appear.



Now, when it comes to the types of exception handling cases, it's like two different ways of dealing with a situation. One is like when you accidentally spill some flour on the floor while making the cake, but you quickly clean it up before it causes any harm. The other is like when you don't have enough eggs for the recipe, but you figure out a way to substitute them with something else and still make the cake.

In conclusion, exception handling in ABAP is like having a team of superheroes that come to save the day when something unexpected happens during program execution. And just like superheroes, they can be caught and handled in different ways depending on the situation.

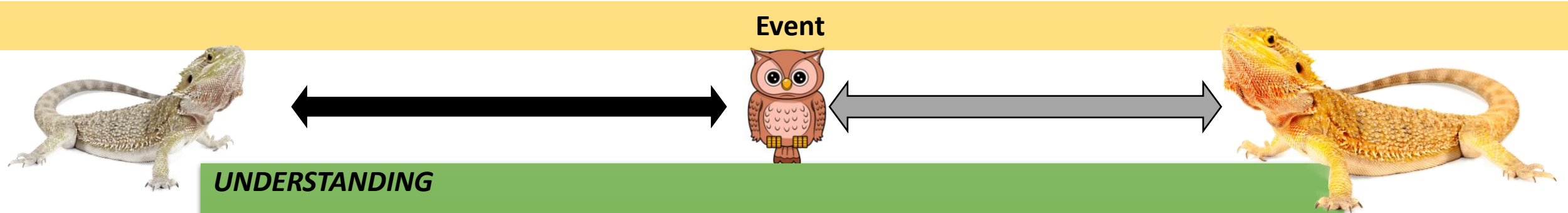
Exception Class

Advantages:

- ✓ **Robustness:** Exception handling allows the program to handle unexpected errors gracefully, making it more robust and reliable.
- ✓ **Readability:** Exception handling code is easy to read and understand, making it easier to maintain and debug.
- ✓ **Modularity:** Exception handling code is modular and can be separated from the main code, making it easier to manage and reuse.
- ✓ **Scalability:** Exception handling code can be reused across multiple programs, making it easier to scale the application.
- ✓ **Error Reporting:** Exception handling allows for detailed error reporting, which makes it easier to diagnose and fix problems.
- ✓ **Security:** Exception handling can help prevent security breaches by catching and handling errors before they can cause harm.

Interview Questions:

1. How do you handle multiple exceptions in a single TRY...CATCH...ENDTRY block in ABAP?
2. What is the difference between CX_SY_ZERODIVIDE and CX_STATIC_CHECK in ABAP?
3. What is the use of RAISING clause in ABAP?
4. How do you debug an exception in ABAP?
5. What is the role of the MESSAGE statement in exception handling in ABAP?
6. How do you handle exceptions in function modules in ABAP?



UNDERSTANDING

“Imagine you are a wizard who wants to communicate with other wizards in the magical world. However, you don't want to bother each wizard by constantly checking whether they have any new messages for you. So, you decide to use an owl post service that delivers messages to you whenever a wizard sends you one. This way, you can focus on your work and get notified only when there is a new message waiting for you.

Similarly, in OOABAP, events are like the owls that deliver messages to the interested parties. An object can send an event message to other objects that are subscribed to receive it. The interested objects are notified when the event is raised, and they can then act upon the message or data received. This way, objects can communicate with each other in a loosely coupled way without interfering with each other's activities.

Events can be either instance events or static events, just like how you can either send an owl to a specific wizard or send a general message to all the wizards in the wizarding community. With instance events, an object sends a message to other objects that are interested in receiving it, while with static events, any object can send a message without the need for an instance.

Events are a powerful way to manage communication between objects in ABAP, just like how owl post is a magical way to manage communication between wizards in the wizarding world.”

Event

“

What are the events in OOABAP?

Events in OO ABAP allow objects to notify other objects of a change in their status or provide information about themselves. Other objects can then respond to that change or information using these events.

In simpler terms, events serve as a mechanism for objects of different types to interact with each other based on specific occasions or actions. It enables a loosely coupled communication between objects, allowing for flexibility and reusability in object-oriented programming.

”

Event

Imagine you have a party, and you want to invite your friends. There are two types of invitations you can send:

Open invitation: This invitation can be sent out to anyone, and they don't need any specific person to invite them. You can post about the party on social media, and everyone who sees it can decide if they want to attend.

Personal invitation: This invitation is specific to a certain person, and you need to send it to them directly. You can't just post it on social media and hope that person sees it. You have to send them a message or give them a call to invite them.

Similarly, in OOABAP, there are two types of events: static and instance-dependent.



Static events can be triggered without the need for an object instance. Just like an open invitation, they can be triggered from a static method, and anyone who has subscribed to the event can respond to it.

Instance-dependent events/ Instance events, on the other hand, are specific to an object instance. Just like a personal invitation, you need a specific object instance to trigger the event, and only those who have subscribed to that particular instance can respond to it.

Event

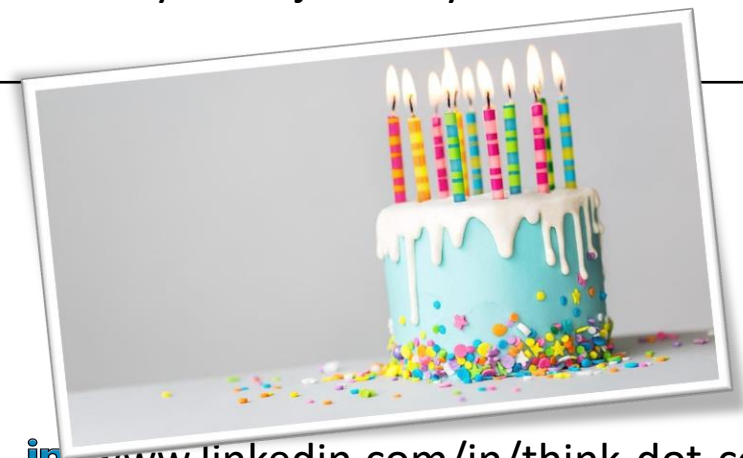
How to trigger Events?

Imagine you are the star of a cooking show, and you just whipped up a delicious cake. Your audience is eagerly waiting to try it, but you don't want to serve it just yet. Instead, you want to let everyone know that the cake is ready and they can come and get a piece.

In the world of SAP, this is similar to how events work. An object, like the cake, can publish an event to let other objects know that something has happened. This event is like the announcement that the cake is ready. Other objects, like hungry people, can subscribe to this event to be notified when it happens.

For example, imagine the SAP class CL_GUI_ALV_GRID has published events like USER_COMMAND, DOUBLE_CLICK, and ONDRAG. Other objects, like programs, can subscribe to these events and use them as needed.

Just like how you can't eat the cake until it's ready, an event can't be triggered until it's raised by an object. Any method of a class can raise its events, like how you can finally serve the cake once it's fully baked.



Event

Here are the ***steps followed in the event mechanism in OO ABAP:***

1. Define Event: The first step is to define an event. This is done at the time of class declaration using the keyword "EVENTS". Standard SAP classes usually have pre-defined events, but custom classes can define their own events. Instance events are declared in the public section of a class and can be triggered by instances of that class. Static events, on the other hand, are declared in the class definition and can be triggered without the need for an instance.

2. Raise Event: Events can be raised by any method of the class. This is done using the ABAP statement "RAISE EVENT". The event name should match the name defined in the class declaration.

For Example: METHOD event_raising_method.

RAISE event_name.

ENDMETHOD.

3. Define Handler Class: A handler class is responsible for taking action on the events subscribed by other objects. It can include events of different object types (classes). The handler class can group handler methods of different events from one or many classes. It is defined separately from the original class.

For example: CLASS handler_class DEFINITION.

PUBLIC SECTION.

METHODS: handler_method

FOR EVENT event_name

OF class_name.

PRIVATE SECTION.

DATA: some_data TYPE string.

ENDCLASS.

Event

4. Define Handler Method: Handler methods are the methods that describe the action to be performed when the event is raised. They are defined within the handler class and must have the same name as the event they are handling. They can take any number of parameters and perform any action.

For example: CLASS handler_class IMPLEMENTATION.

METHOD handler_method.

WRITE: / 'Event', event_name, 'was raised.'.

"...some logic here

ENDMETHOD.

ENDCLASS.

5. Register Event Handler: This step involves subscribing to the event of a particular class. It is done using the ABAP statement **"SET HANDLER"**. The handler instance is an instance of the handler class that will be called when the event is raised. The handled instance is an instance of the class that is being monitored for the event.

For example: SET HANDLER handler_instance-handler_method

FOR ALL INSTANCES.

SET HANDLER handler_instance-handler_method

FOR handled_instance.

Event

Conclusion:

Events in SAP follow a "broadcast" pattern. Imagine a news station publishing a newsletter, where anyone who wants to read it needs to subscribe for a copy. Similarly, events are published in the class definition and different objects then subscribe to those events depending on their needs.

When an event is triggered, the handlers that have registered themselves with the SET HANDLER statement are executed by the runtime. This allows objects to communicate with each other in a loosely coupled way, without needing to know the specifics of each other's implementation.

*The steps involved in implementing events in SAP are relatively uniform across different usages of object-oriented programming in SAP. For example, the class CL_GUI_ALV_GRID for ALV Grid has published events like **USER_COMMAND**, **DOUBLE_CLICK**, and **ONDRAG**, which can be subscribed to and used by other objects (programs).*



Singleton class



Singleton class

UNDERSTANDING:

In India, Aadhaar card is a unique identification card issued to each citizen. Just like Aadhaar card, a singleton class is also unique and there can be only one instance of it.

Think of a singleton class as an Aadhaar card issuing authority. There is only one such authority in the country, and it issues unique Aadhaar cards to each citizen. Similarly, a singleton class can only be instantiated once, and it ensures that there is only one instance of the class at any given time.

Let's say we have a program that needs to generate a unique ID number for each citizen, just like the Aadhaar card number. We can create a singleton class called "AadhaarSingleton" that generates the unique ID numbers.

Now, when we need to generate an Aadhaar card number for a new citizen, we can simply call a method in the AadhaarSingleton class to generate the number. The class will ensure that the number is unique and no two citizens have the same number.

For example, let's say we have a new citizen named Ram Lakhan. We can call the AadhaarSingleton class to generate a unique ID number for him. The class will generate a new number and return it to us. Now, every time we need to generate a new Aadhaar card number, we can simply call this singleton class and get a new unique number.

This is just one example of how a singleton class can be used in OOABAP to ensure that only one instance of the class exists and to provide a unique functionality.

Singleton class

A singleton class is a special type of class in OOABAP that restricts the instantiation of the class to a single object. This means that there can be only one instance of the class at any given time, and this instance can be accessed by other objects through a global method call. The main idea behind using a singleton class is to ensure that there is only one object that manages a particular resource or functionality, thus avoiding duplication and inconsistencies in the system.

To create a singleton class in SAP ABAP, the class constructor is defined as private and a static method is created to get the instance of the class. This method is responsible for creating the instance of the class if it doesn't exist, or returning the already created instance if it does. This ensures that there is always only one instance of the class in the system.

ADVANTAGES :

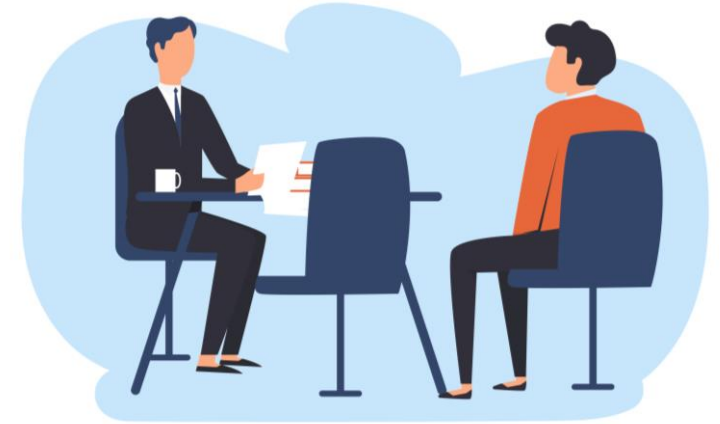
- ✓ **Global access:** Singleton class allows global access to its unique instance from anywhere in the program. This means that the instance can be accessed and used by any object in the program.
- ✓ **Memory management:** Singleton class helps in efficient memory management by creating only one instance of the class which can be reused multiple times, reducing the memory usage in the program.
- ✓ **Consistency:** Since there is only one instance of the Singleton class, it ensures consistency throughout the program. Any changes made to the instance will reflect in all objects using that instance.
- ✓ **Security:** Singleton class can also be used to implement security measures, such as restricting access to certain resources or limiting the number of instances of a particular class.
- ✓ **Performance:** The use of Singleton class can also improve the performance of the program as it reduces the need for creating and destroying objects frequently.

Singleton class

For Code Example: Click [HERE](#)

Interview Questions:

1. What is a Singleton Class in OOABAP SAP?
2. What is the purpose of a Singleton Class in OOABAP SAP?
3. How do you implement a Singleton Class in OOABAP SAP?
4. Can you create multiple instances of a Singleton Class in OOABAP SAP?
5. What is the difference between a regular class and a Singleton Class in OOABAP SAP?
6. What is the use of a private constructor in a Singleton Class in OOABAP SAP?
7. Can you explain the Singleton pattern in OOABAP SAP?
8. How do you ensure thread-safety in a Singleton Class in OOABAP SAP?
9. How do you access the instance of a Singleton Class in OOABAP SAP?
10. Can you give an example of a scenario where a Singleton Class can be useful in OOABAP SAP?



Design Patterns in SAP ABAP

INTRODUCTION:

Imagine you're a software wizard and you're about to embark on an epic journey to explore the mystical land of design patterns in SAP ABAP. Along the way, you'll encounter all sorts of challenges and obstacles as you attempt to uncover the secrets of this mysterious world.

As you set out on your adventure, you'll quickly realize that many projects share similar processes and flows, much like how many adventurers must cross treacherous rivers and climb steep mountains. But fear not, for there is a magical tool at your disposal that can help you overcome these challenges: the design pattern.

And just like how wizards have their own unique spells and incantations, each design pattern has its own unique recipe for success. With this knowledge in hand, you'll be able to tackle any challenge that comes your way, whether it's designing a login system or creating a powerful spell to defeat a fearsome dragon.

So grab your wand (or keyboard) and get ready to explore the fascinating world of design patterns in SAP ABAP!



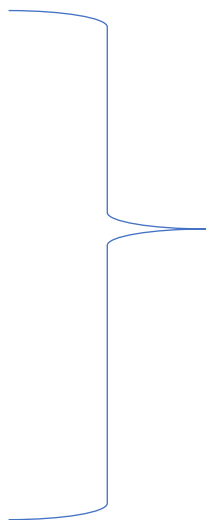
Design Patterns in SAP ABAP

Design Patterns in SAP ABAP:

In SAP ABAP, design patterns are general solutions to common problems that can be reused. They are templates for how to solve specific problems that can be used in different scenarios. Design patterns in SAP ABAP are considered best practices for solving common problems while designing a product.

Some of the popular design patterns in SAP ABAP are:

- MVC (model, view, controller) pattern
- Singleton pattern
- Factory pattern
- Builder pattern
- Observer pattern
- Visitor pattern
- Lazy initialization pattern
- Template method
- Strategy pattern
- Decorator pattern



[Click HERE](#)

Design patterns are sometimes considered as Syntax. Well, it is not and can be amended as per the project requirement

Design patterns are helpful for improving code quality, reducing development time and effort, and promoting code reuse. On the other hand, anti-patterns are common mistakes that lead to bad design and inefficient code. It is important to avoid them while designing a product.

Design Patterns in SAP ABAP

Advantages :

1. **Reusability:** Design patterns provide a reusable solution to a common problem, which saves time and effort in coding.
2. **Best Practices:** Design patterns are considered as the best practices, which means using them ensures a high-quality design and fewer errors.
3. **Flexibility:** Design patterns are flexible and adaptable to various scenarios, which makes them useful in different applications.
4. **Standardization:** Design patterns help to standardize the code, making it easier to maintain and update in the future.
5. **Collaboration:** Design patterns are commonly used across various development teams, which allows for better collaboration and communication between team members.
6. **Scalability:** Design patterns make it easier to scale applications and add new features without disrupting the existing codebase.
7. **Improved Performance:** Design patterns can optimize the code, leading to better performance and faster execution.
8. **Cost-Effective:** Using design patterns can reduce development time and cost, as well as improve the overall quality of the product.

Design Patterns in SAP ABAP

Limitations :

1. **Increased complexity:** Design patterns can add additional complexity to a project, which can make it more difficult to understand and maintain.
2. **Over-engineering:** Using design patterns when they are not necessary can lead to over-engineering, where the code becomes too complex for the requirements of the project.
3. **Learning curve:** Developers may need to spend time learning and understanding the various design patterns, which can increase the learning curve and delay the development process.
4. **Performance:** Some design patterns may have a negative impact on performance, as they can introduce additional layers of abstraction and overhead.
5. **Non-standard approach:** Some design patterns may not be commonly used in SAP ABAP, which can make it difficult for other developers to understand the code.
6. **Limited applicability:** Not all design patterns are applicable to every situation, and some may be better suited for specific types of projects or scenarios.



Task on OOAAP(For Freshers)***Some tasks on OOABAP for fresher:***

1. Create a class called "Car" with the following attributes: "make", "model", "year", and "color". Implement a constructor method that takes these attributes as parameters and initializes the object. Implement getter and setter methods for each attribute.
2. Create a class called "Calculator" that has four methods: "add", "subtract", "multiply", and "divide". Implement each method to perform the corresponding arithmetic operation on two numbers that are passed as parameters.
3. Create a class called "Person" with the following attributes: "name", "age", "gender", and "address". Implement a constructor method that takes these attributes as parameters and initializes the object. Implement a method called "printInfo" that prints all the attributes of the person object.
4. Create a class called "BankAccount" with the following attributes: "accountNumber", "balance", and "ownerName". Implement a constructor method that takes these attributes as parameters and initializes the object. Implement a method called "deposit" that takes an amount as parameter and adds it to the balance. Implement a method called "withdraw" that takes an amount as parameter and subtracts it from the balance.
5. Create a class called "Shape" with a method called "area". Implement two subclasses of "Shape" called "Rectangle" and "Circle" with their own implementations of the "area" method. The "Rectangle" class should have attributes "length" and "width", and the "Circle" class should have an attribute "radius". Implement a constructor method for each subclass that takes the necessary attributes as parameters.

Task on OOAAP(For Freshers)***Some tasks on OOABAP for fresher:***

6. Create a class called "Employee" with the following attributes: "name", "id", "designation", and "salary". Implement a constructor method that takes these attributes as parameters and initializes the object. Implement a method called "calculateBonus" that calculates the bonus for the employee based on their salary and designation. Implement a method called "printSalarySlip" that prints the employee's name, id, designation, salary, and bonus.
7. Create a class called "Animal" with the following attributes: "name", "species", "age", and "sound". Implement a constructor method that takes these attributes as parameters and initializes the object. Implement a method called "makeSound" that prints the sound of the animal. Implement two subclasses of "Animal" called "Dog" and "Cat" with their own implementations of the "makeSound" method.
8. Create a class called "Student" with the following attributes: "name", "rollNumber", "marks", and "grade". Implement a constructor method that takes these attributes as parameters and initializes the object. Implement a method called "calculateGrade" that calculates the grade for the student based on their marks. Implement a method called "printResult" that prints the student's name, roll number, marks, and grade.

Task on OOAAP(For Freshers)***Some tasks on OOABAP for fresher:***

9. Create a class called "Address" with the following attributes: "street", "city", "state", and "zipCode". Implement a constructor method that takes these attributes as parameters and initializes the object. Implement a method called "printAddress" that prints the full address of the object.
10. Create a class called "Book" with the following attributes: "title", "author", "publisher", and "price". Implement a constructor method that takes these attributes as parameters and initializes the object. Implement a method called "discountPrice" that calculates the discounted price of the book based on a discount percentage that is passed as a parameter. Implement a method called "printDetails" that prints the book's title, author, publisher, price, and discounted price.

Good Luck!
😊

Type Casting

UNDERSTANDING:

Imagine you are at a party and you meet a person who introduces themselves as "a person". You can't really tell much about them just from that description. They could be anyone - tall, short, blonde, brunette, etc.

Now imagine you ask them "Are you a dancer?" and they respond "Yes, I am a ballet dancer".

Suddenly, you know much more about them! They have a specific skill and area of expertise that sets them apart from just any random person.

In the same way, type casting in OOABAP SAP allows us to take a variable of a general data type and "narrow" it down to a more specific type. This can be useful when we want to access certain properties or methods that are only available to that specific type.

For example, let's say we have a variable "my_obj" that is declared as type "OBJECT". This means that it could hold any object of any class. However, if we know that "my_obj" is actually an instance of a specific class "MY_CLASS", we can "cast" it to that class using the syntax "my_obj->cast()".

Once we have cast "my_obj" to "MY_CLASS", we can access all the properties and methods that are specific to that class. It's like suddenly discovering that the person you met at the party is a ballet dancer - now you can ask them specific questions about ballet and they will have the knowledge and expertise to answer.

Overall, type casting can be a powerful tool in OOABAP SAP programming, allowing us to make use of the specific features and capabilities of different classes in our code.



Type Casting

Type casting in OOABAP SAP is a technique that allows you to convert an instance of one class into an instance of another class. This is achieved by explicitly stating the desired type of the object reference.

Type casting is necessary when you want to use an object of a derived class as if it were an object of its base class. This is useful when you want to call methods or access properties that are defined in the base class, but not in the derived class.

The syntax for type casting in OOABAP SAP is as follows:

```
DATA: obj TYPE REF TO class1,  
      ref TYPE REF TO class2.
```

```
ref ?= obj.
```

In this example, obj is an instance of class1 and ref is an instance of class2. The ?= operator is used to cast the obj reference into a ref reference.

Type casting can also be used to convert an instance of a class into an interface. This allows you to call methods defined in the interface on the object.

Type Casting

There are two types of type casting in OOABAP SAP: explicit and implicit.

1. **Explicit type casting** is when you specify the desired type of the object reference in the code.
2. **Implicit type casting** is when the system automatically converts the object reference to the desired type based on the context in which it is used.

One important thing to note is that type casting can lead to runtime errors if the object reference cannot be cast into the desired type.

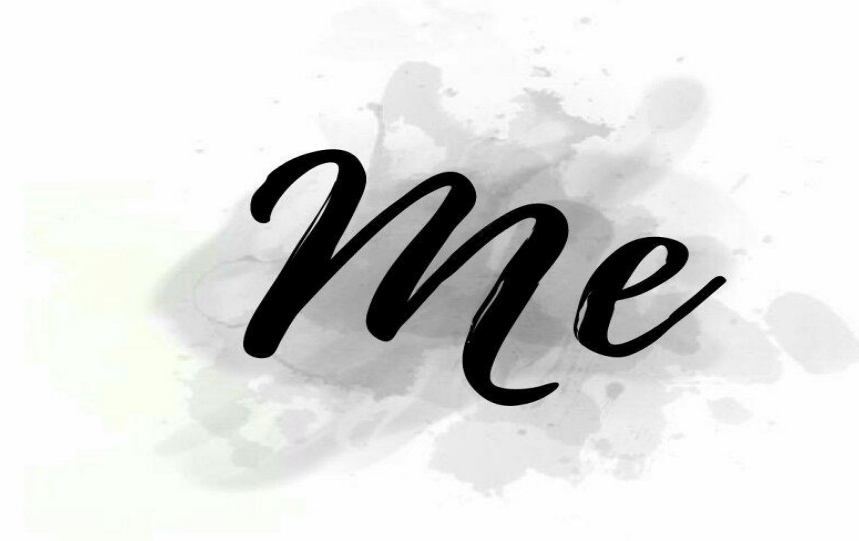
Therefore, it is important to always check the type of the object reference before casting it to avoid runtime errors.

Overall, type casting is a powerful technique in OOABAP SAP that allows you to work with objects of different classes and interfaces in a flexible way.

ME Keyword

Picture this: you're a programmer, working hard to write some ABAP code. Suddenly, you realize that you need to refer to the object that you're currently working with, but you don't want to use the object's name directly (maybe it's too long, or you're worried about typos). What do you do?

Enter the **"me"** keyword! This little gem of a keyword allows you to refer to the current object in your code without actually having to type out its name. It's like having a personal assistant that knows exactly what you're working on, and is always ready to help you out.

***Here's an example:***

let's say you're working on a class called "Car", and you need to write a method that returns the car's color. Instead of typing "Car->color", you can use the "me" keyword like this: "me->color". It's shorter, sweeter, and a lot less prone to errors. But wait, there's more! The "me" keyword isn't just for objects - it can also be used in interface methods to refer to the implementing class. So if you have a class that implements an interface, and you need to call a method from that interface within the class, you can use "me" to refer to the class itself. It's like having a secret code that only the interface and the implementing class can understand.

So there you have it - the "me" keyword in all its glory. It may be small, but it's definitely mighty. And who knows - maybe one day, you'll find yourself in a situation where you'll need to use it too!

ME Keyword

In SAP ABAP, the keyword "**ME**" is used to refer to the current instance of a class. It can be used to access the methods and attributes of the current object.

```
REPORT ZPRG_FSDEOOI.  
CLASS lcl_parent DEFINITION.  
  PUBLIC SECTION.  
    METHODS: display_message.  
ENDCLASS.
```

```
CLASS lcl_parent IMPLEMENTATION.  
  METHOD display_message.  
    WRITE: 'Hello from Parent class.'.  
  ENDMETHOD.  
ENDCLASS.
```

```
CLASS lcl_child DEFINITION INHERITING FROM parent.  
  PUBLIC SECTION.  
    METHODS: display_message.  
ENDCLASS.
```

```
CLASS lcl_child IMPLEMENTATION.  
  METHOD display_message.  
    WRITE: 'Hello from Child class.'.  
    ME->parent~display_message( ).  
  ENDMETHOD.  
ENDCLASS.
```

```
START-OF-SELECTION.  
  DATA: object_ref TYPE REF TO child.  
  CREATE OBJECT object_ref.  
  object_ref->display_message( ).
```



Private & Final Class

SS

Tuesday 4:40 PM

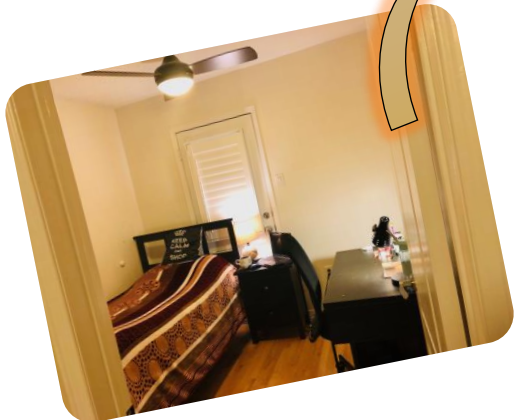
Good Evening sir..I have a request.. Could you please explain the difference between private and final class..I have theoretical knowledge but want to know when private and final will be used?

I was intrigued by this question, so I've decided to include it this series.

UNDERSTANDING

Think of a private class as a private room in a house, where only certain people are allowed to enter. In the same way, a private class can only be accessed by the code within the same class and not from any other class.

Now, let's think of a final class as a sealed box, where nothing can be added or changed once it's sealed. Similarly, a final class cannot be extended by any other class, and no new methods or variables can be added to it.



Private & Final Class

A private class is a class that can only be accessed from within the same enclosing class, and it cannot be accessed from outside the class. This means that any methods or variables in the private class are only visible and accessible within the same class. Private classes are typically used for encapsulation, which is a programming practice that involves hiding the implementation details of a class from the outside world. This is useful for maintaining the integrity of the class and for preventing outside code from modifying its internal state.

So, when would you use a private class? You would use a private class when you have some functionality or data that should only be accessed by the code within the same class, and not by any other classes in the program.

On the other hand, a final class is a class that cannot be subclassed or extended. This means that once a class is declared final, no other class can inherit from it or override its methods. Final classes are typically used to prevent inheritance or to ensure that a particular implementation cannot be changed. Final classes are useful when you want to create an immutable class, which is a class whose state cannot be changed once it has been created. This is useful for creating objects that are guaranteed to be thread-safe and that cannot be modified accidentally.

When would you use a final class? You would use a final class when you want to prevent any other classes from extending it, perhaps because you want to maintain the functionality or data exactly as it is without any changes.

In summary, the main difference between a private and a final class is their scope and their intended use. A private class is used for encapsulation and restricting access to a class's implementation details, while a final class is used to prevent inheritance and ensure that a class's implementation cannot be changed.

BAPI(Business Application Programming Interface)***UNDERSTANDING:***

Imagine you're the captain of a ship, sailing across the vast ocean of SAP products. You have a map and a compass, but you need a way to communicate with the different ports you encounter along the way. That's where SAP BAPIs come in.

BAPIs are like little boats that ferry information between your ship and the ports. They have a set path and know exactly how to navigate the waters to reach their destination. And just like how you need to follow certain rules and regulations when entering a new port, BAPIs ensure that all the business rules and checks are properly executed when interacting with SAP's business object models.

Think of BAPIs as a team of superheroes, each with their unique powers. They are implemented as function modules that call upon the internal code of SAP products. These function modules have the power to access and modify business objects, depending on the permissions granted to them.

So whether you're a captain of a ship, a superhero in need of backup, or just a developer looking for a way to interact with SAP products, BAPIs are here to help you navigate the waters and get the job done.



BAPI(Business Application Programming Interface)

INTRODUCTION:

SAP BAPI (Business Application Programming Interface) is a standardized interface that provides access to business object models in SAP products. BAPIs serve as the primary method for customer code and third-party applications to interact with SAP products. These interfaces encapsulate the internal layers of SAP's business object model and ensure that all business logic, validations, and authorization checks are executed properly when accessing or changing business objects.

BAPIs are implemented as function modules that call SAP internal code, and depending on which set of BAPIs is being used, they may call business object models defined using the Business Objects Processing Framework (BOPF) or legacy models defined using programs, tables, and function modules. The architecture of BAPIs allows for the creation of a standardized interface, which helps to reduce development time and costs by providing a common framework for accessing SAP business objects.

There are two main types of BAPIs in SAP:

- 1. *Standard BAPIs:*** These are pre-defined BAPIs that are delivered with the SAP system. They cover a wide range of business processes and are usually used as-is, although they can also be modified to meet specific business requirements.
- 2. *Custom BAPIs:*** These are BAPIs that are created by developers to meet specific business requirements. Custom BAPIs are created using the same development tools and techniques as standard BAPIs, and can be integrated into SAP systems in the same way.

Additionally, there are also ***transactional BAPIs*** and ***non-transactional BAPIs***. Transactional BAPIs are used to update SAP data and are executed within a transaction, while non-transactional BAPIs are used to retrieve data and can be executed without being in a transaction.

BAPI(Business Application Programming Interface)***Some additional details about BAPIs:***

- 1. BAPIs are usually provided by SAP as pre-defined function modules, but they can also be created by customers or third-party developers.*
- 2. BAPIs can be used to read data from SAP, create or modify data in SAP, and perform other operations such as executing business processes or running reports.*
- 3. BAPIs are typically used in integration scenarios where data needs to be exchanged between SAP and other systems, such as an e-commerce website or a third-party logistics provider.*
- 4. BAPIs can be called from a variety of programming languages, including ABAP, Java, and .NET.*
- 5. BAPIs provide a layer of abstraction that shields the calling application from the complexities of the underlying SAP system. For example, the calling application doesn't need to know the specifics of the SAP data model or business logic.*
- 6. BAPIs are designed to be stable and backward-compatible, meaning that they should continue to work even as SAP products evolve over time.*
- 7. BAPIs can be used in conjunction with other SAP technologies such as IDocs, RFCs (Remote Function Calls), and ALE (Application Link Enabling) to build complex integration scenarios.*
- 8. BAPIs can be used to access a wide range of SAP modules and data types, including materials management, financials, human resources, and more.*

Overall, BAPIs provide a powerful and flexible mechanism for integrating SAP with other systems and applications, and they are widely used in the SAP ecosystem.

BAPI(Business Application Programming Interface)

For hand-on example

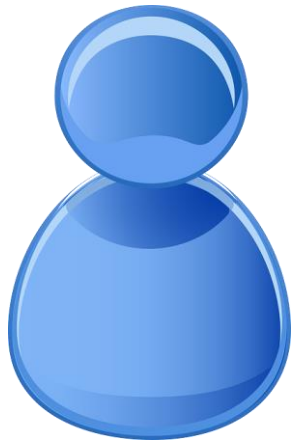
Please visit: <https://sapcodes.com/2015/11/19/bapi-an-introduction/>

Some interview questions on BAPI in SAP:

1. What is a BAPI?
2. What are the advantages of using BAPIs in SAP?
3. How is a BAPI different from an RFC?
4. Can you give an example of a standard BAPI in SAP?
5. How do you create a custom BAPI in SAP?
6. How can you test a BAPI in SAP?
7. What is the difference between a committed and uncommitted BAPI?
8. What is the difference between a synchronous and asynchronous BAPI call?
9. Can a BAPI return multiple values?
10. What is a BOR (Business Object Repository), and how does it relate to BAPIs?
11. How can you monitor and troubleshoot BAPI calls in SAP?
12. What are some best practices for using BAPIs in SAP?
13. Can you use BAPIs in non-SAP systems? If so, how?
14. What is the difference between a BAPI and a Web service?
15. Can a BAPI be called from ABAP code? If so, how?



RFC(Remote Function Call)

**UNDERSTANDING:**

*Have you ever needed to call your friend who lives in another city to get some information? You pick up your phone and dial their number, and voila! You are able to communicate with them and get the information you need. Similarly, **RFC or Remote Function Call** is like a **phone call between business applications to exchange information (in pre-defined formats) with other systems, located either within or outside the organization.***



RFC(Remote Function Call)

INTRODUCTION:

RFC stands for "Remote Function Call". It is a mechanism that allows different software applications to communicate and exchange information in pre-defined formats with each other, regardless of the technology or platform they are running on. RFC is a standard SAP protocol that enables the execution of a function module in a remote system. This allows applications running on different systems to interact with each other seamlessly.

It has two interfaces: one for ABAP programs and one for Non-SAP programs. When an ABAP program wants to use a function in a remote system, it uses the **CALL FUNCTION...DESTINATION** statement. The **DESTINATION** parameter is used to specify which system should run the called function.

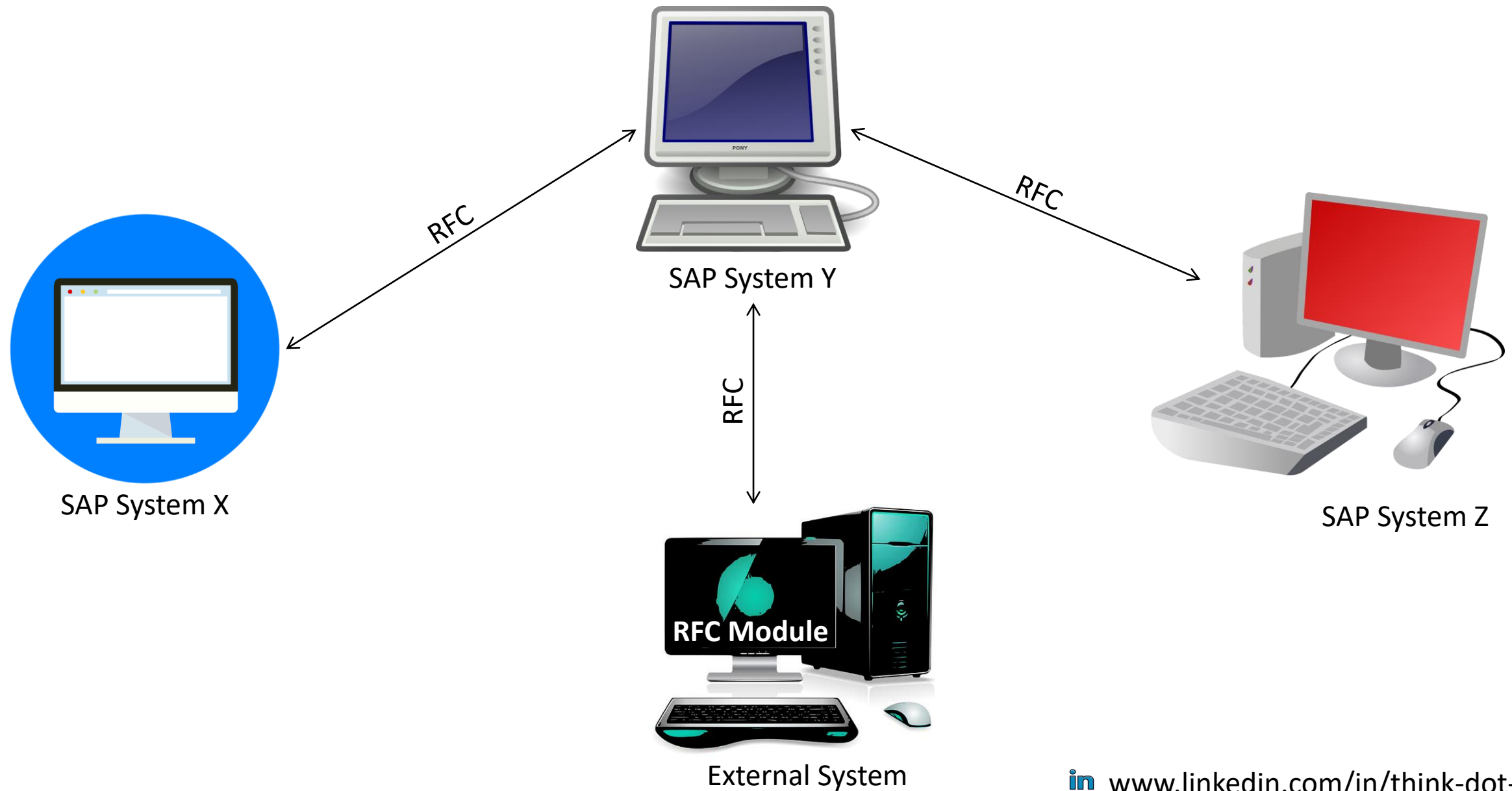
SYNTAX:

```
CALL FUNCTION 'FUNCTION_NAME'  
  DESTINATION 'DESTINATION_NAME'  
  EXPORTING  
    parameter1 = value1  
    parameter2 = value2  
  IMPORTING  
    result1 = variable1  
    result2 = variable2  
  EXCEPTIONS  
    communication_failure = 1  
    system_failure = 2  
  ...
```

FUNCTION_NAME refers to the name of the remote function being called

DESTINATION_NAME specifies the remote system in which the function is executed.

RFC(Remote Function Call)



RFC(Remote Function Call)

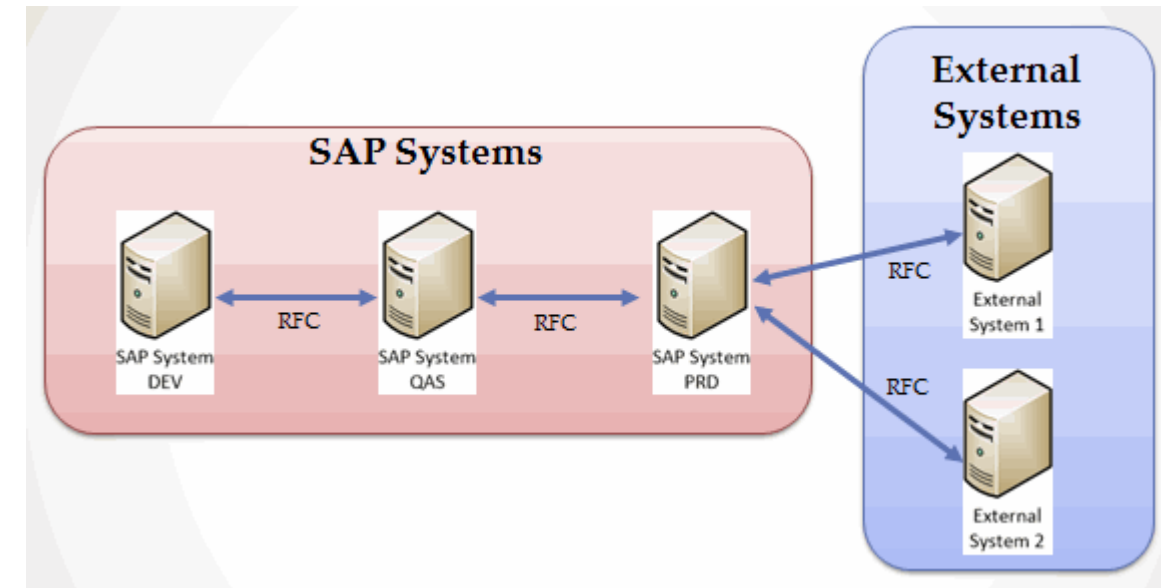
Unraveling the Mystery of RFC - What You Need to Know:

1. When it comes to exchanging data between different systems, SAP uses a special protocol called CPIC (**Common Programming Interface for Communication**). But here's the catch - it's a protocol exclusive to SAP systems. So, to make it easier for programmers to use, SAP developed Remote Function Call (RFC), a communications interface that builds upon the foundations of CPI-C and offers more functionality.
2. With RFC, you can call special subroutines called function modules over a network. These modules work in a similar way to the functions and procedures used in programming languages like C and PASCAL. The beauty of function modules is that they have a defined interface that lets you exchange data, tables, and return codes. You can manage these function modules in a dedicated library called the Function Builder within the R/3 system.
3. The Function Builder (also known as transaction SE37) is a powerful environment that makes it easy to program, document, and test function modules. And when it's time to make a remote call, the R/3 system automatically generates the extra code (RFC stub) needed to make it happen.
4. **Here's the kicker** - RFC connections can be used across the entire system, regardless of which client you're in. That means if you define an RFC connection in client 000, you can use it in client 100 without any modifications. And if you need to maintain your RFC connections, you can do it quickly and easily using transaction SM59.
5. The R/3 system also comes equipped with an RFC-SDK (**Software Development Kit**) that uses extensive C libraries to let external programs connect to the R/3 system. The only thing to keep in mind is that when making a remote call, you need to specify the target server using a special parameter called the destination.

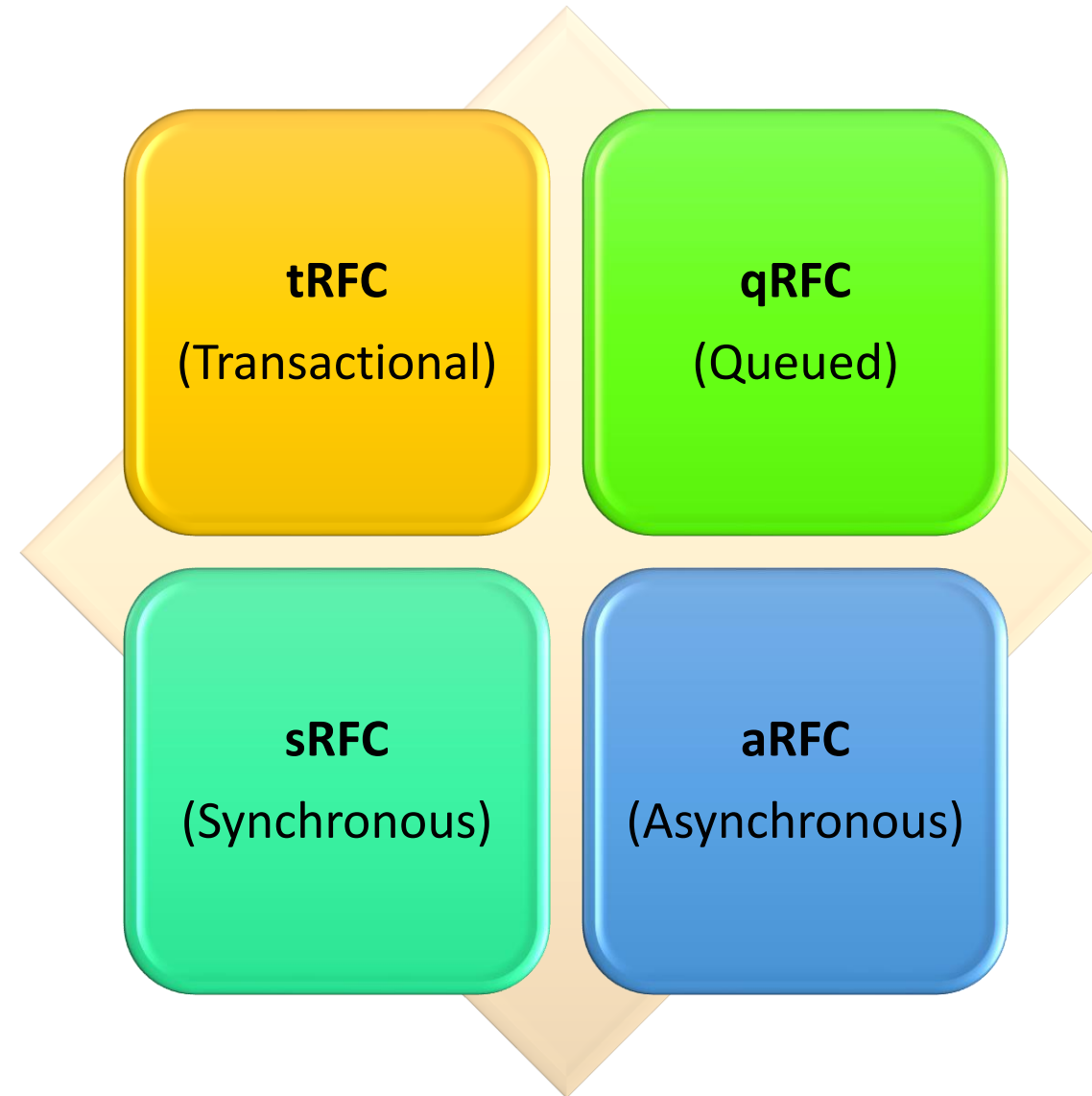
RFC(Remote Function Call)

RFC, or Remote Function Call, offers several advantages, some of which include:

1. **Reduced programming effort:** RFC reduces the efforts of programmers by enabling them to avoid the re-development of modules and methods at remote systems.
2. **Easy data conversion:** RFC is capable of converting data into a format that is understandable by the remote (target) system.
3. **Efficient routine calling:** RFC can call up certain routines that are necessary to start communication with the remote system.
4. **Error handling:** RFC can handle errors that might occur during the communication process.
5. **Increased flexibility:** RFC connections can be used across the entire system, which means that an RFC connection defined in one client can be used from another client without any difference.



RFC(Remote Function Call)



RFC(Remote Function Call)

Synchronous RFC

Synchronous communication is like a conversation between two people who are both actively present and listening to each other. Similarly, in the world of SAP, synchronous Remote Function Call (sRFC) requires both the client and server systems to be available and ready to communicate or transfer data. This is the most common type of RFC and is used when immediate results are required after the execution of the RFC.

When you use sRFC, the resources of the source system wait for the target system to acknowledge receipt of the data or message. This ensures that the data is delivered with acknowledgment (ACKD) and is consistent and reliable for communication. In other words, both systems synchronize their communication so that data is sent and received in a reliable and orderly manner.

However, there's a catch: If the target system is not available, the source system resources have to wait until the target system becomes available. This may cause the processes of the source system to go into Sleep/RFC/CPIC Mode at the target system, and it may block these resources. So it's important to make sure that both systems are available before using sRFC.

Overall, sRFC is commonly used for communication between systems, such as transferring data between different SAP systems. It can also be used for communication between the SAP Web Application Server and SAP GUI. Synchronous RFC can help ensure reliable and consistent data exchange, but it's important to ensure that both systems are available and ready to communicate.

RFC(Remote Function Call)

Asynchronous
RFC

Imagine you're sending a package to your friend. You could choose to send it through a service that requires a signature upon delivery, like FedEx. This means your friend needs to be present to receive the package and sign for it, and you'll receive confirmation once the package is delivered. This is similar to synchronous communication in the SAP world.



On the other hand, you could choose to send the package through regular mail, like a postcard. You don't require a signature upon delivery, so your friend doesn't need to be present when the package is delivered, and you won't receive confirmation that it was delivered. This is similar to asynchronous communication in SAP.

In asynchronous communication, the source system doesn't wait for the target system to acknowledge the message or data. It sends the data and doesn't care if it was received or not. This is useful when you don't need the result immediately or if the target system is busy. However, it's not reliable, since the data might be lost if the target system is unavailable.

Asynchronous communication is useful for communication between systems and for parallel processing, where you want to run tasks simultaneously without waiting for each other to finish. It's like sending multiple postcards to different friends, without waiting for each friend to receive their postcard before sending the next one.

RFC(Remote Function Call)

Transactional RFC

Imagine you're sending a parcel to your friend. You can either send it via regular mail or registered mail. The regular mail is like asynchronous RFC, where you just send the parcel and don't wait for any acknowledgment or confirmation. You know it may get lost or delayed, but you're willing to take that risk. On the other hand, registered mail is like transactional RFC. You pay extra for it, but it ensures that your parcel will be delivered securely, and you'll get a receipt with a unique transaction ID (TID) to track it. Even if there are some issues during the delivery, like the parcel being sent multiple times due to network problems, the registered mail system will handle it and make sure that the parcel is delivered only once, without any data loss.

Now, let's relate it to SAP. Transactional RFC is a special type of asynchronous RFC that ensures transaction-like handling of processing steps. It's like the registered mail of SAP communication. The remote system doesn't need to be available at the time of execution, and the called function module is executed only once, even if the data is sent multiple times due to some network issue. The tRFC component stores the called RFC function, together with the corresponding data, in the SAP database under a unique transaction ID (TID), which makes it secure and reliable. Similar to aRFC, it doesn't wait at the target system, but writes the data into aRFC Tables with a transaction ID (SM58) if the system is not available. The scheduler RSARFCSE picks it up every 60 seconds to ensure that the transaction is completed successfully. So, if you want to ensure secure communication between systems, use transactional RFC just like you would use registered mail to send your important parcels!

RFC(Remote Function Call)

Queued RFC

Imagine you have several transactions that need to be executed in a specific order, but some of them can take longer to process. How do you make sure that they are executed in the order you want them to be? That's where qRFC comes in!

qRFC, or Queued RFC, is an extension of tRFC (Transactional RFC). It ensures that the individual steps are processed in sequence, but with the added benefit of using queues to serialize the transactions. Think of it as lining up for a rollercoaster ride where you must wait for the person in front of you to finish their ride before it's your turn.

The inbound and outbound queues make sure that multiple Logical Units of Work (LUWs) or transactions are processed in the order specified by the application. It guarantees that several transactions are executed in a predefined order, making it the perfect choice for applications that require a defined processing sequence.

But why stop at just one rollercoaster ride? qRFC has another fantastic feature that allows you to execute multiple transactions in parallel, making it faster and more efficient. With qRFC, you can line up multiple rollercoasters, and they will all move at the same time, but the ones that take longer will be allowed to finish before the ones that take less time.

qRFC is recommended for applications that require a specific order of processing, but it is also perfect for applications that need to execute multiple transactions in parallel. It ensures reliable communication, consistency, and speed, making it a valuable tool for any SAP programmer.

RFC(Remote Function Call)

For hand-on example

Please visit: <https://abapskill.com/abap/rfc-function-module-in-abap-with-example/>

Some interview questions related to RFC:

1. What is RFC?
2. What are the different types of RFC?
3. What are the advantages of using RFC?
4. What is the difference between synchronous and asynchronous RFC?
5. What is the difference between tRFC and qRFC?
6. How do you debug an RFC?
7. What is the difference between CALL FUNCTION and RFC function module?
8. How do you create an RFC connection?
9. Can you explain the steps involved in creating a custom RFC function module?
10. How do you troubleshoot RFC connection issues?
11. What are the different ways to monitor RFC communication in SAP?
12. How do you perform load balancing in RFC?
13. Can you explain the concept of RFC destinations?
14. How do you handle errors in RFC communication?
15. What are the best practices for designing and developing RFC function modules?



Difference between BAPI and RFC in SAP ABAP***UNDERSTANDING:***

RFC and BAPI are both methods for communication between different SAP systems or between SAP and non-SAP systems. However, they serve different purposes. RFC stands for Remote Function Call, and it is a general term for any function module that can be called remotely. It can be used for any kind of data exchange, and it allows for both synchronous and asynchronous communication.

On the other hand, BAPI stands for Business Application Programming Interface. It is a specific type of RFC that is designed for business transactions. BAPIs are stored in the Business Object Repository (BOR) and are used to access and manipulate data in SAP business objects. They are standardized interfaces, so they make it easier for developers to access data from different SAP systems without needing to know the technical details of each system.

Think of RFC as a general purpose tool, like a Swiss Army Knife. It has many different functions and can be used for a variety of tasks. BAPI, on the other hand, is like a specialized tool, like a wrench or screwdriver. It's designed for a specific purpose, and it makes that task easier and more efficient.

So, while RFC is a general term for remote function calls, BAPI is a specific type of RFC that is designed for business transactions. Both are important tools for communication between different systems, but they serve different purposes.

Difference between BAPI and RFC in SAP ABAP

- BAPI stands for Business Application Programming Interface, which is a type of RFC enabled function module that provides external access to an SAP business application. On the other hand, RFC stands for Remote Function Call, which is a protocol for communication between applications of different systems in the SAP environment.
- BAPIs can be wrapped in Business Objects, which is a reusable object-oriented programming interface for SAP business applications. However, RFCs cannot be wrapped in Business Objects.
- BAPIs can be compared to CGI applications, which are commonly used for creating dynamic web content. On the other hand, RFCs can be compared to HTTP, which is a protocol for communication between web servers and clients.
- All BAPIs are RFC enabled, but not all RFCs are BAPIs.
- BAPIs do not use exceptions in their interface, while RFCs can have exceptions defined in their interface.

RFCs are often used for technical integrations between SAP systems or between SAP and non-SAP systems, whereas BAPIs are more commonly used for business-level integrations between SAP components. In other words, RFCs are typically used for data exchange between systems or applications, while BAPIs are used for accessing SAP business objects and performing specific business tasks.

For example, if you need to exchange data between two SAP systems, you might use an RFC. But if you need to create a sales order in SAP from a non-SAP system, you might use a BAPI.

Difference between BAPI and RFC in SAP ABAP***INTERVIEW QUESTIONS:***

1. What is RFC and how does it work?
2. What is the difference between a synchronous RFC and an asynchronous RFC?
3. How do you create an RFC destination in SAP?
4. What is a BAPI and how is it different from an RFC?
5. What are the advantages of using BAPI over RFC?
6. How do you find a list of available BAPIs in SAP?
7. What is the difference between a BAPI and a function module?
8. Can a BAPI be called from non-SAP systems? If so, how?
9. How do you create a new BAPI in SAP?
10. What are the different types of BAPIs?
11. What are the benefits of using RFCs and BAPIs for system integration?
12. How do you test an RFC or BAPI?
13. What are the limitations of using RFC and BAPI?
14. How do you troubleshoot errors in RFC and BAPI?
15. What are the different types of RFC interfaces in SAP?
16. How do you establish an RFC connection between two SAP systems?
17. What are the different types of RFC destinations?
18. What is a registered server program in SAP?
19. How do you set up a registered server program in SAP?
20. How do you configure security for RFC and BAPI calls?



Difference between BAPI and RFC in SAP ABAP***INTERVIEW QUESTIONS:***

21. What is a trusted RFC connection and how is it set up?
22. How do you monitor RFC and BAPI calls in SAP?
23. What is the difference between a BOR and a BAPI?
24. What is the difference between BAPI and IDoc?
25. Can you explain the concept of RFC destination groups?
26. What is the difference between a synchronous and asynchronous up
27. How do you debug an RFC or BAPI?
28. What is the difference between stateless and stateful RFC?
29. What is the role of a transaction code in RFC and BAPI?
30. What is the difference between an inbound and outbound RFC?
31. What are the different types of error handling mechanisms available BAPI?
32. Can you explain the concept of a remote enabled function module?
33. What is the difference between a remote enabled function module a enabled function module?
34. How do you handle exceptions in RFC and BAPI?
35. What is the difference between BAPI and RFC in terms of error handling?
36. How do you transport an RFC or BAPI to a different SAP system?
37. What is the difference between a tRFC and qRFC?
38. What is the role of a gateway server in SAP RFC?



Difference between BAPI and RFC in SAP ABAP***INTERVIEW QUESTIONS:***

39. How do you configure RFC tracing in SAP?
40. What is the difference between RFC and SOAP?
41. What is the difference between a RFC and a web service?
42. What is the difference between RFC and RESTful services?
43. How do you handle large volumes of data in RFC and BAPI?
44. What is the difference between a BAPI and a business object?
45. What is the difference between an API and a BAPI?
46. What is the difference between a BAPI and a transaction?
47. How do you implement error handling in BAPIs?
48. What is the difference between a synchronous and asynchronous BAPI?
49. Can you explain the concept of a change pointer in BAPIs?
50. How do you use BAPIs in SAP Business Workflow?

